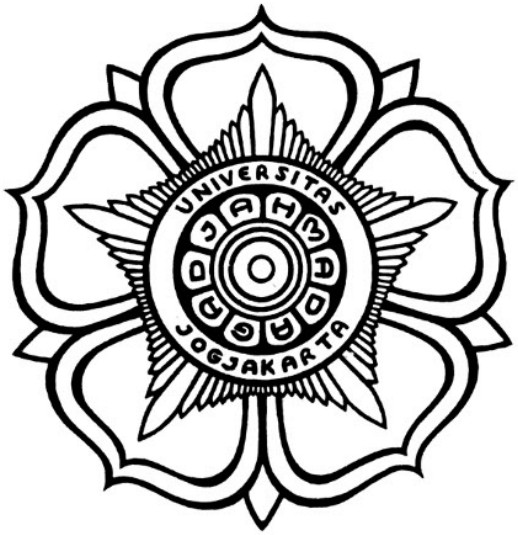
# PERANCANGAN IP I2S TRANSMITTER AXI4-LITE PADA SOC MICROBLAZE V MENGGUNAKAN VERILOG

SKRIPSI




*THE SUSTAINABLE DEVELOPMENT GOALS*
*Industry, Innovation and Infrastructure*
*Affordable and Clean Energy*
*Climate Action*

**Disusun oleh:**

**Muhammad Muqtada Alhaddad**
**22/500341/TK/54841**

**PROGRAM SARJANA PROGRAM STUDI TEKNOLOGI INFORMASI**
**DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI**
**FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA**
**YOGYAKARTA**
**2026**

# HALAMAN PENGESAHAN

# PERANCANGAN IP I2S TRANSMITTER AXI4-LITE PADA SOC MICROBLAZE V MENGGUNAKAN VERILOG

**SKRIPSI**

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

**Muhammad Muqtada Alhaddad**
**22/500341/TK/54841**

Telah disetujui dan disahkan

Pada tanggal . . . . . .

| Dosen Pembimbing I | Dosen Pembimbing II |
|---|---|
| **Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM.** | **Ir. Wahyu Dewanto, M.T.** |
| **198001012015041002** | **196111091986011001** |

# PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

| | |
|---|---|
| Nama | : Muhammad Muqtada Alhaddad |
| NIM | : 22/500341/TK/54841 |
| Tahun terdaftar | : 2022 |
| Program | : Sarjana |
| Program Studi | : Teknologi Informasi |
| Fakultas | : Teknik Universitas Gadjah Mada |

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, 26 Mei 2026

Materai Rp10.000

(Tanda tangan)

Muhammad Muqtada Alhaddad
22/500341/TK/54841

# HALAMAN PERSEMBAHAN

*"Dan tidaklah kamu diberi ilmu melainkan hanya sedikit" (QS. Al-Isra: 85)*

Skripsi ini penulis persembahkan kepada:

Orang tua tercinta atas doa, dukungan, dan pengorbanan tanpa henti.

Para dosen pembimbing yang telah membimbing perjalanan akademik penulis.

Semua pihak yang telah berkontribusi dalam perjalanan penyusunan skripsi ini.

# KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT atas rahmat, karunia, dan petunjuk-Nya sehingga penulis dapat menyelesaikan skripsi yang berjudul **"Perancangan IP I2S Transmitter AXI4-Lite pada SoC MicroBlaze V menggunakan Verilog"**. Shalawat serta salam semoga senantiasa tercurah kepada Rasulullah SAW, keluarga, sahabat, dan para pengikut beliau hingga akhir zaman.

Penyusunan skripsi ini tidak terlepas dari bantuan, bimbingan, dukungan, dan doa dari banyak pihak. Oleh karena itu, penulis menyampaikan terima kasih kepada:

1. Kedua orang tua dan keluarga besar penulis yang selalu memberikan doa, dukungan, perhatian, dan kasih sayang selama penulis menempuh masa studi.
2. Bapak Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM. selaku dosen pembimbing pertama yang mengenalkan penulis pada bidang IC design saat perkuliahan Sistem Berbasis Mikrokontroler, serta Bapak Ir. Wahyu Dewanto, M.T. selaku dosen pembimbing kedua yang mengajarkan dasar komunikasi data kepada penulis pada semester tiga sehingga menumbuhkan ketertarikan pada cara kerja komputer. Keduanya telah meluangkan waktu untuk memberikan arahan, masukan, dan bimbingan selama proses penelitian dan penyusunan skripsi ini. Serta Bapak Dani Adhipta, S.Si., M.T. dosen pembimbing akademik yang telah memberikan berbagai dukungan dan arahan sehingga penulis dapat menempuh studi dengan baik.
3. Segenap dosen dan staf Departemen Teknik Elektro dan Teknologi Informasi, Fakultas Teknik, Universitas Gadjah Mada, yang telah memberikan ilmu, pengalaman, serta bantuan administratif selama masa perkuliahan.
4. Teman-teman Teknologi Informasi angkatan 2022 dan teman-teman UGM lintas angkatan yang telah menjadi rekan belajar, berdiskusi, dan menjalani berbagai dinamika perkuliahan.
5. Om Asep yang telah memberikan dukungan, perhatian, dan menjadi wali selama penulis tinggal di Yogyakarta. Beserta Pak De, Bu De, Mas Rizky Bagaskara, Mba Lia serta teman-teman warga sumberan yang masih mengingat walau sudah lama tidak mengunjungi sejak kelas 2 SD .
6. Teman-teman Laboratorium IC, yaitu Ariq, Aji, Willi, Adit, Muhajir, Rakhel, Jonatan, Alphons, Mas Alfito, Mba Arifa, Ciel, Egi, Alex, Luqman, Rafinda serta Pak Sudiro, Mas Byan, Mas Cahyo, dan Mas Abid, atas diskusi, masukan, dan kebersamaan selama proses penelitian.
7. Mas Hilmi, Ustadz Nabyl, Ustadz Habibi, Ustadz Arif, Ustadz Minan, Ustadz Nurkholis, Ustadz Fawaid, serta teman-teman santri Pesantren Attaqwa Alawiyah yang telah memberikan ilmu, dukungan, pengalaman berharga, serta kesempatan mengajar kelas aljabar.
8. Teman-teman Alamin Yogyakarta yang telah menjadi keluarga dan teman berproses selama penulis menempuh pendidikan di Yogyakarta.
9. Anggana Putra, Gusti Syahrul, Nadya Biantri, dan Nicho Desvayawindra, teman sekolah dari Pontianak yang telah menemani, memberikan saran dan informasi seputar Yogyakarta, serta menjadi tempat berkonsultasi saat penulis awal pindah.

10. Mas Giga Hidjrika, Mas Karunia Perjuangan, Mas Lutfi Andriyanto, Mas Krisna Laksheta, Mas Afiq Zuhdi, Mas Orlando Ferrari, dan Bang Furqan yang telah menjadi mentor penulis selama masa perkuliahan.
11. Ridhwan Saputra, Adib Maulana, Arif Ibadurahman, Ega Baskara, serta teman-teman PPSMB Harjono Danoesastro 2 dan Sadewa 6 yang menjadi teman pertama penulis di UGM. Terima kasih pula kepada Mba Ifa dan Mba Ima sebagai pembimbing pada awal perkuliahan, serta Mas Damar dan Mba Dita sebagai mentor pada masa pengenalan kampus. Penulis juga berterima kasih kepada Mba Siti Malatania yang menjadi narahubung saat awal masuk kampus, serta semua pihak yang membantu penulis ketika pertama kali mendaftar perkuliahan.
12. Tim penelitian Psybot, yaitu Mba Khomsah, Virna, Anggita, Fina, dan Ibu Indrayanti, S.Psi., M.Si., Ph.D., Psikolog yang telah memberikan pengalaman penelitian dan kesempatan presentasi di PIMNAS beserta teman-teman kontingen, serta Bapak Muhammad Idham Ananta Timur, M.Kom., Mba Tata, Mba Riyanti, dan Mas Alfan dalam penelitian terkait kesehatan mental karyawan dengan skrining bot WhatsApp. Terima kasih juga kepada Dela Feby yang telah memberikan informasi terkait lomba yang paling berpengaruh dalam membuka kesempatan dan memperluas relasi penulis selama kuliah.
13. Teman-teman mahasiswa penggiat startup, yaitu Ridho Alfadri, Azfazaki Hilmi, Tantowi Shah, Achmad Luthfan, dan Mas Arief Rahman yang telah memberikan pengalaman berharga dalam berkarya dan mendorong penulis untuk berani mencoba hal baru.
14. Tim petugas TPS mahasiswa Karanggayam, yaitu Zahy, Farhan, Ipal, Astrid, Mba Safa, dan Mba Faiqoh yang bersama-sama telah menjalankan tugas sebagai petugas TPS pada Pemilu 2024 dengan bersih dan transparan.
15. Teman-teman Kelas Naskah, yaitu Kak Mark, Kak Alif, Manda, Mas Ai, dan Kak Maria, serta rekan-rekan dari divisi lain yang telah memberikan pengalaman kepada penulis sebagai developer volunteer untuk industri perfilman di Indonesia.
16. Bapak Dr. Guntur Dharma Putra, Bapak Dr. Bimo Sunarfri Hantono, S.T., M.Eng., Fathan Zahir, dan Mas Bagus Raka atas ilmu, pengalaman, dan kerja sama dalam penelitian sistem signage DTETI.
17. Bapak Dr. Ir. Ahmad Nasikun, S.T., M.Sc., Mas Iskan, Pak Hartanto, dan Pak Nur Muhammad yang telah memberikan bimbingan dan pengalaman berharga selama penulis bekerja paruh waktu sebagai developer di fakultas. Terima kasih pula kepada rekan-rekan kantor FT, yaitu Pak Purwoko, Pak Doni, Pak Bandi, Bu Desi, Bu Rahmi, Bu Nanda, Bu Acil, Bu Mega, Pak Untung, dan Pak Arief atas kebersamaan dan dukungannya, serta Bu Nuning yang telah memberikan kesempatan kepada penulis untuk mengembangkan sistem capaian kerja di FKKMK UGM. Serta Mas Deni dan Mbah yang sentiasa memberi dukungan dan yang saya selalu memintakan doa ketika bertemu di masjid teknik.
18. Rekan-rekan selama program kuliah magang IC Design Engineer di Polytron, Raphael, Fauzan, Kak Nur, Pande, Rizfa, Afgar, Andres, Ardian, Azriel, Bayu, Yavi, Chandra, Debora, Dwi, Faiq, Faisal, Ferhad, Hanif, Hisyam, Arif, Ivan, Mba Fazya, Fira, Ela, Renata, Aiman, Mas Andre, Mas Aziz, Galih, Mas Jabar, Jevon, Mas Priyo, Radit, Wisnu, Valen, Mas Wibaga, Mba Jihan, Mona, Fattah, Naumi, Nicky,

Panji, Putra, Qory, Raihan, Reza, Ridho, Surya, dan Tegar, serta Pak Arvin, Pak Riyad, dan Bu Dian atas pengalaman, kerja sama, dan dukungan selama kegiatan magang berlangsung. Terima kasih juga kepada Bapak Warsun Najib, M.T. yang menjadi pembimbing serta memberikan masukan saat laporan seminar kuliah praktik.

19. Sahabat satu tim Capstone D-02, yaitu Nanta, Titan, Laras, dan Nisa yang telah memberikan pengalaman dan kerja sama dalam menciptakan H.A.R.V.E.S.T. sebagai solusi pertanian bawang merah di Bantul.
20. Teman-teman KKN Minggir, yaitu Ayub, Subhan, Afifa, Anggun, Gama, Jasmin, Linggar, Akbar, Hanifah, Lea, Nisa, Padia, Rona, Iqbal, Amalia, Hilmi, Ilma, Irsyad, Lala, Safira, Berlean, Fajar, Farah, Hanif, Ifah, Sheila, Risti, dan Mba Vania, serta masyarakat Padukuhan Ngijon yang telah memberikan pengalaman, cerita, dan kebersamaan selama 51 hari pelaksanaan KKN.
21. Pak Yaenuri, A.Md., Pak Sunandar, S.T., Pak Nanang Muhammad Yusuf, S.T., Pak Heruwanto, S.T., M.M., Pak Nanang Dani Widyanto, S.T., Pak Rayuh Dhilah Hanggara, S.T., Pak Arwin Pamungkas, S.Pd., Pak Tri Wahyudiono, serta jajaran tenaga pendidik yang dengan penuh senyuman telah memberikan dukungan, bantuan, serta pengalaman selama masa perkuliahan dan kegiatan akademik maupun laboratorium.
22. Tim Senior Project Documind, yaitu Febri dan Yitzhak, Junior Project HarborFlow, yaitu Mirsad Alganawi dan Marseillo Satrian, serta Junior Project PaluGada, yaitu Luthfi Hanif dan Brian yang tetap melanjutkan pekerjaan tim ketika penulis menjalani kuliah magang.
23. Teman-teman Eternity Sistem Informasi UNTAN yang selalu memberikan dukungan kepada penulis selama menjalani dinamika kehidupan perkuliahan.
24. Teman-teman sobat DTI, yaitu Sulton Fatih, Hanif Riadi, Alfath Azka, Mas Adji Sudradjat, Mas Syafaq Abdallah, Mas Hafzhan Wisli, dan Fahmi Irfan Faiz yang telah menemani penulis selama pengerjaan skripsi di DTI.
25. Tetangga kos dekat apartemen, yaitu Mas Hasan, Mas Rama, Mas Choerul, Mas Kris, Mas Bagus, Mas Aldi, dan Mas Alvin yang selalu hadir saat dibutuhkan, termasuk ketika penulis perlu meminjam barang, serta Pak Ari selaku bapak kos, Pak Tumiran, Bu Mangkir, dan warga Pogung yang telah membantu penulis selama tinggal di sana.

Penulis menyadari bahwa skripsi ini masih memiliki kekurangan. Oleh karena itu, penulis terbuka terhadap saran dan masukan yang membangun. Penulis berharap skripsi ini dapat memberikan manfaat bagi pengembangan ilmu pengetahuan, khususnya dalam bidang sistem digital dan sistem embedded. Akhir kata, penulis mengucapkan terima kasih kepada seluruh pihak yang telah memberikan dukungan, bantuan, dan doa selama proses penyusunan skripsi ini. Semoga Allah subhanahu wa ta'ala membalas segala kebaikan yang telah diberikan. Aamiin.

Yogyakarta, 14 Mei 2026

Muhammad Muqtada Alhaddad

# DAFTAR ISI


HALAMAN PENGESAHAN ..... ii
PERNYATAAN BEBAS PLAGIASI ..... iii
HALAMAN PERSEMBAHAN ..... iv
KATA PENGANTAR ..... vii
DAFTAR ISI ..... viii
DAFTAR TABEL ..... xii
DAFTAR GAMBAR ..... xiii
DAFTAR SINGKATAN ..... xvi
INTISARI ..... xvii
ABSTRACT ..... xviii
BAB I Pendahuluan ..... 1
1.1 Latar Belakang ..... 1
1.2 Rumusan Masalah ..... 3
1.3 Tujuan Penelitian ..... 3
1.3.1 Tujuan perancangan dan integrasi perangkat keras ..... 3
1.3.2 Tujuan integrasi SoC dan perangkat lunak ..... 4
1.3.3 Tujuan verifikasi dan validasi ..... 4
1.4 Batasan Penelitian ..... 4
1.4.1 Batasan perangkat keras dan RTL ..... 4
1.4.2 Batasan SoC dan perangkat lunak ..... 5
1.4.3 Batasan pengujian dan dokumentasi ..... 5
1.5 Manfaat Penelitian ..... 5
1.5.1 Manfaat akademik ..... 5
1.5.2 Manfaat praktis ..... 5
1.5.3 Manfaat pengembangan keahlian ..... 5
1.6 Sistematika Penulisan ..... 6
BAB II Tinjauan Pustaka dan Dasar Teori ..... 7
2.1 Tinjauan Pustaka ..... 7
2.1.1 Penelitian tentang implementasi I2S pada FPGA ..... 7
2.1.2 Penelitian tentang prosesor soft-core dan integrasi SoC ..... 7
2.1.3 Penelitian tentang sinkronisasi lintas domain clock (CDC) ..... 7
2.1.4 Penelitian tentang penjadwalan firmware berbasis counter siklus .. 8
2.1.5 Gap penelitian dan kontribusi ..... 8
2.2 Dasar Teori ..... 8
2.2.1 Arsitektur MicroBlaze V dan Memory-Mapped I/O ..... 8
2.2.2 Protokol Komunikasi AXI4-Lite ..... 9

2.2.3 Protokol Komunikasi Audio I2S ..... 9
2.2.4 Pembangkitan Clock Audio Berbasis NCO ..... 10
2.2.5 DAC Audio PCM5102A ..... 10
2.2.6 FPGA dan Board Basys3 ..... 11
2.2.7 Alur Pengembangan Vivado dan Vitis ..... 11
2.2.8 Verilog HDL, Perbandingan IP, dan Analisis Metode ..... 13
BAB III Metode Penelitian ..... 14
3.1 Alat dan Bahan ..... 14
3.1.1 Perangkat keras ..... 14
3.1.1.1 Perangkat pengembangan dan target hardware ..... 14
3.1.1.2 Perangkat pengukuran dan pembantu ..... 14
3.1.2 Perangkat lunak ..... 14
3.2 Metode yang Digunakan ..... 15
3.2.1 Metode perancangan hardware ..... 15
3.2.2 Metode verifikasi dan simulasi ..... 15
3.2.3 Metode integrasi SoC dan pengembangan software ..... 15
3.3 Perancangan Block Design Sistem ..... 15
3.3.1 Integrasi SoC MicroBlaze V ..... 15
3.3.2 Perancangan Clocking Audio Berbasis NCO ..... 16
3.3.3 Perancangan Register AXI4-Lite ..... 16
3.3.4 Signal Generator dan Constraint FPGA ..... 16
3.4 Alur Tugas Akhir ..... 16
BAB IV Perancangan dan Implementasi ..... 18
4.1 Implementasi IP I2S ..... 18
4.1.1 Struktur Modul RTL ..... 18
4.1.2 Implementasi AXI4-Lite ..... 22
4.1.3 Implementasi Register Konfigurasi ..... 23
4.1.4 Implementasi Clock Domain Crossing ..... 25
4.1.5 Implementasi Generator Clock Berbasis NCO ..... 27
4.1.6 Implementasi Serializer Philips I2S ..... 28
4.1.7 Implementasi Pengaturan Frekuensi Sampling Arbitrer ..... 30
4.1.8 Implementasi Enable, Mute, Reset, dan Output Assignment ..... 31
4.1.9 Implementasi Integrasi pada Vivado Block Design ..... 32
4.2 Implementasi Firmware ..... 34
4.2.1 Struktur Program ..... 34
4.2.2 Inisialisasi Periferal ..... 35
4.2.3 Konfigurasi Register I2S ..... 35
4.2.4 Pembangkitan Data Audio ..... 37
4.2.5 Runtime Control Flow ..... 39

4.2.6 Pengujian dan Validasi ..... 41
4.3 Ringkasan Implementasi ..... 42
BAB V Hasil dan Pembahasan ..... 43
5.1 Pengujian Sinyal I2S ..... 43
5.2 Analisis Frekuensi Sampling ..... 45
5.3 Validasi Audio pada Berbagai Frekuensi Sampling ..... 45
5.4 Identifikasi dan Perbaikan Bug Firmware ..... 46
5.4.1 Bug 1: Masker Bit Merusak Bit Tanda Sampel ..... 46
5.4.2 Bug 2: Timing Loop Berbasis `usleep()` ..... 46
5.4.3 Variabel dan Fungsi Kunci Firmware ..... 47
5.5 Verifikasi Setelah Perbaikan Firmware ..... 47
5.5.1 Akurasi Frekuensi Nada ..... 47
5.5.2 Fungsionalitas Bit Kontrol ..... 49
5.6 Verifikasi Integrasi dengan Filter Digital ..... 49
5.6.1 Modul RTL `i2s_filter_bridge` ..... 49
5.7 Analisis Penggunaan Sumber Daya dan Daya ..... 54
5.8 Pembahasan terhadap Tujuan Penelitian ..... 55
5.8.1 Perubahan Frekuensi Sampling ..... 55
5.8.2 Variasi Lebar Data Sampel ..... 57
5.8.3 Kontrol Kanal dan Status Mute ..... 57
5.9 Perbandingan dengan Referensi Desain ..... 59
5.10 Keterbatasan Desain ..... 59
BAB VI Kesimpulan dan Saran ..... 61
6.1 Kesimpulan ..... 61
6.2 Saran ..... 61
DAFTAR PUSTAKA ..... 63
LAMPIRAN ..... L-1
L.1 Hasil resource utilization ..... L-1
L.1.1 Post-implementation resource report ..... L-1
L.1.2 Timing report ..... L-2
L.2 Timing diagram ..... L-2
L.2.1 I2S audio transmission timing ..... L-2
L.3 Log hasil pengujian ..... L-2
L.3.1 I2S audio playback test ..... L-2
L.4 Deskripsi block design ..... L-3
L.4.1 Diagram Block (Color-coded) ..... L-3
L.4.2 I2S IP Block (Standalone) ..... L-4
L.4.3 Clocking View ..... L-4
L.4.4 Interface View ..... L-4

L.5 Hasil pengujian i2s transmitter - capture osiloskop ..... L-5
L.5.1 Test Frekuensi Audio ..... L-5
L.5.2 Test Kontrol Enable ..... L-6
L.5.3 Test Kontrol Mute ..... L-7
L.5.4 Test Routing Stereo ..... L-7
L.5.5 Test Lebar Sample (Sample Width) ..... L-9
L.6 Design Timing Summary ..... L-10
L.7 Power Summary ..... L-10
Ringkasan IP Block Design ..... L-12
BAB L Ringkasan Kode Sumber ..... L-19
L.1 Repositori ..... L-19
Arbitrary-Fs I2S dan IP komunikasi buatan sendiri ..... L-20

# DAFTAR TABEL


Tabel 2.1 Perbandingan pembangkitan clock berbasis PLL dan NCO. ..... 11

Tabel 4.1 Register map final IP I2S. ..... 24
Tabel 4.2 Bitfield register `CONTROL`. ..... 24

Tabel 5.1 Hasil validasi frekuensi sampling pada empat mode standar dan dua frekuensi non-standar. ..... 45
Tabel 5.2 Ringkasan penggunaan sumber daya FPGA Artix-7 XC7A35T (Basys3). ..... 54
Tabel 5.3 Perbandingan fitur IP I2S Transmitter. ..... 59

Tabel L.1 Ringkasan utilisasi resource FPGA ..... L-1
Tabel L.2 Hasil analisis timing ..... L-2
Tabel L.3 Segmen alamat penting pada block design `bdesign_clock_core` L-17

# DAFTAR GAMBAR


Gambar 1.1 Diagram arsitektur lengkap sistem SoC dengan IP I2S TX AXI4-Lite dan DAC PCM5102A. . . . . 3

Gambar 2.1 Diagram timing sinyal protokol I2S standar (Sumber: NXP). . . . . 9
Gambar 2.2 Spesifikasi arsitektur dan modul fisik DAC PCM5102A. . . . . 11
Gambar 2.3 Board FPGA Basys3 Artix-7 yang digunakan dalam penelitian. . . 12

Gambar 3.1 Koneksi antarmuka pin antara FPGA Basys3 dan modul DAC PCM5102A. . . . . 14
Gambar 3.2 Komponen IP pendukung utama pada Block Design Vivado. . . . . 16
Gambar 3.3 Diagram alir pengerjaan penelitian tugas akhir skripsi. . . . . 17

Gambar 4.1 Arsitektur sistem akhir yang menghubungkan prosesor, UART, AXI4-Lite, dan IP I2S. . . . . 18
Gambar 4.2 Blok internal IP I2S yang terdiri dari AXI4-Lite, register file, NCO, latch sampel, dan serializer. . . . . 21
Gambar 4.3 Timing latching sampel kiri dan kanan pada awal frame stereo. . . 30
Gambar 4.4 Layout bit register `CONTROL` untuk konfigurasi frekuensi sampling arbitrer. . . . . 30
Gambar 4.5 Rangkaian reset sistem yang digunakan pada Block Design Vivado. . . . . 33
Gambar 4.6 Koneksi pin PMOD FPGA ke modul DAC PCM5102A. . . . . 33
Gambar 4.7 Pandangan blok clocking yang relevan dengan pembangkitan BCLK dan MCLK pada IP. . . . . 36
Gambar 4.8 Relasi antara prosesor, UART, timer, dan IP I2S pada sistem akhir. 36

Gambar 5.1 Tangkapan timing internal I2S: frame stereo, penghitung bit, dan clock divider. . . . . 44
Gambar 5.2 Keluaran osiloskop: nada sinus 1 kHz pada $F_s = 48$ kHz. . . . . 48
Gambar 5.3 Keluaran osiloskop: nada sinus 2 kHz pada $F_s = 48$ kHz. . . . . 48
Gambar 5.4 Keluaran osiloskop: nada sinus 5 kHz pada $F_s = 48$ kHz. . . . . 48
Gambar 5.5 Verifikasi fungsionalitas bit `ENABLE` dan `MUTE` pada keluaran audio analog. . . . . 49
Gambar 5.6 Keluaran stereo setelah perbaikan: kanal L dan R simetris dan identik. . . . . 50
Gambar 5.7 Integrasi IP I2S dengan blok filter IIR pada Vivado Block Design. 53
Gambar 5.8 Ringkasan timing desain pasca-implementasi dari Vivado Timing Analyzer. . . . . 54
Gambar 5.9 Analisis implementasi fisik pada FPGA Basys3. . . . . 55
Gambar 5.10 Perbandingan sinyal LRCK, BCLK, dan keluaran audio analog pada empat mode $F_s$ standar. . . . . 56
Gambar 5.11 Efek variasi lebar data sampel terhadap sinyal serial DATA dan keluaran audio analog. . . . . 57
Gambar 5.12 Validasi rute kanal, bit `ENABLE`, dan bit `MUTE` melalui sinyal digital dan keluaran analog. . . . . 58

Gambar L.1 Ringkasan utilisasi resource (visual). . . . . L-1

Gambar L.2 Diagram block warna-warni arsitektur SoC. L-3
Gambar L.3 Blok IP I2S standalone. L-4
Gambar L.4 Clocking view dan jalur pemilihan clock audio. L-4
Gambar L.5 Interface view pemetaan pin dan jalur debug. L-5
Gambar L.6 Capture osiloskop: audio tone 1 kHz. L-5
Gambar L.7 Capture osiloskop: audio tone 2 kHz. L-5
Gambar L.8 Capture osiloskop: audio tone 5 kHz. L-6
Gambar L.9 Capture osiloskop: enable=0. L-6
Gambar L.10 Capture osiloskop: enable=1. L-6
Gambar L.11 Capture osiloskop: mute=0. L-7
Gambar L.12 Capture osiloskop: mute=1. L-7
Gambar L.13 Hasil capture osiloskop: Left channel only - hanya kanal left dioutput L-8
Gambar L.14 Hasil capture osiloskop: Right channel only - hanya kanal right dioutput L-8
Gambar L.15 Hasil capture osiloskop: Both channels - kedua kanal left dan right aktif simultan L-8
Gambar L.16 Hasil capture osiloskop: Sample width 16-bit - format audio PCM 16-bit L-9
Gambar L.17 Hasil capture osiloskop: Sample width 24-bit - format audio PCM 24-bit pada slot 32-bit L-9
Gambar L.18 Hasil capture osiloskop: Sample width 32-bit - full 32-bit slot I2S frame L-10
Gambar L.19 Design timing summary report - Worst Negative Slack (WNS), Total Negative Slack (TNS), dan status timing closure L-10
Gambar L.20 Power summary report - total on-chip power consumption, breakdown per resource (MMCM, Block RAM, Slice Logic), dan power supply margins L-11
Gambar L.21 Diagram block design berwarna yang menunjukkan hubungan prosesor, interconnect, clock, dan IP I2S. L-12
Gambar L.22 Tampilan clocking view pada block design untuk domain clock sistem dan jalur audio. L-12
Gambar L.23 Tampilan interface view yang memperlihatkan keterhubungan MicroBlaze, UARTLite, dan IP I2S melalui AXI SmartConnect. L-13
Gambar L.24 Tampilan awal konfigurasi Clocking Wizard sebelum penyesuaian frekuensi keluaran. L-13
Gambar L.25 Clocking Wizard setelah dikonfigurasi untuk menghasilkan clock sistem 100 MHz. L-13
Gambar L.26 Blok prosesor MicroBlaze V yang menjadi pengendali utama sistem. L-14
Gambar L.27 Block design reset controller yang membangkitkan sinyal reset sinkron untuk domain sistem. L-14
Gambar L.28 AXI SmartConnect yang merutekan transaksi dari MicroBlaze ke peripheral I2S dan UARTLite. L-14
Gambar L.29 Subsistem BRAM lokal untuk instruksi dan data MicroBlaze V. L-15
Gambar L.30 Microprocessor Debug Module yang digunakan untuk keperluan debug prosesor. L-15
Gambar L.31 Peripheral UARTLite sebagai antarmuka terminal firmware. L-16

Gambar L.32 Blok internal IP I2S yang digunakan sebagai peripheral utama. . L-16

# DAFTAR SINGKATAN

| | | |
|---|---|---|
| AXI | = | Advanced eXtensible Interface |
| BCLK | = | Bit Clock |
| BRAM | = | Block RAM |
| CDC | = | Clock Domain Crossing |
| CPU | = | Central Processing Unit |
| DAC | = | Digital to Analog Converter |
| DDS | = | Direct Digital Synthesis |
| DMA | = | Direct Memory Access |
| DSP | = | Digital Signal Processor / Processing |
| FF | = | Flip-Flop |
| FIFO | = | First-In-First-Out |
| FPGA | = | Field Programmable Gate Array |
| HDL | = | Hardware Description Language |
| I2S | = | Inter-IC Sound |
| IP | = | Intellectual Property |
| IRQ | = | Interrupt Request |
| LRCK | = | Left/Right Clock (sama dengan WS) |
| LSB | = | Least Significant Bit |
| LUT | = | Look-Up Table |
| MCLK | = | Master Clock |
| MMCM | = | Mixed-Mode Clock Manager |
| MSB | = | Most Significant Bit |
| NCO | = | Numerically Controlled Oscillator |
| RISC-V | = | Reduced Instruction Set Computer V |
| RTL | = | Register Transfer Level |
| SoC | = | System on Chip |
| UART | = | Universal Asynchronous Receiver-Transmitter |
| WS | = | Word Select |
| XDC | = | Xilinx Design Constraints |

# INTISARI

Penelitian ini berfokus pada perancangan dan implementasi peripheral transmitter I2S (Inter-IC Sound) menggunakan bahasa Verilog HDL yang diintegrasikan ke dalam System-on-Chip (SoC) berbasis prosesor MicroBlaze V pada FPGA. Peripheral ini diimplementasikan sebagai custom IP dengan antarmuka AXI4-Lite slave sehingga dapat dikonfigurasi dan diisi data audio melalui register memory-mapped oleh perangkat lunak yang berjalan di Xilinx Vitis. Platform target yang digunakan adalah board FPGA Basys3, sedangkan keluaran audio direalisasikan melalui modul DAC PCM5102A. Modul pendukung seperti UART dan infrastruktur clock hanya digunakan sebagai penunjang sistem, sementara kontribusi utama skripsi ini adalah desain IP komunikasi audio buatan sendiri, integrasi, dan validasi peripheral transmitter I2S agar sistem tidak bergantung pada IP berlisensi dari pihak ketiga. **Kontribusi utama penelitian ini adalah perancangan Custom IP I2S Transmitter yang mendukung frekuensi sampling arbitrer berbasis NCO dan dapat diintegrasikan sebagai peripheral AXI4-Lite pada SoC MicroBlaze V tanpa ketergantungan pada IP audio berlisensi pihak ketiga.**

Metodologi penelitian mencakup: (1) desain RTL Verilog untuk transmitter I2S yang mengikuti format Philips I2S untuk transmisi audio stereo, (2) perancangan antarmuka register AXI4-Lite untuk kontrol, pemuatan sampel, dan observasi status, (3) integrasi custom IP I2S ke dalam SoC MicroBlaze V menggunakan Vivado block design, (4) pengembangan perangkat lunak uji dalam bahasa C pada Xilinx Vitis untuk konfigurasi register, pembangkitan sampel sinus, dan pengiriman sampel audio berbasis memory-mapped I/O, serta (5) verifikasi melalui synthesis, implementation, simulasi testbench (TB), pengamatan osiloskop, dan pengujian hardware dengan modul PCM5102A.

Hasil penelitian menunjukkan bahwa IP transmitter I2S yang dirancang dapat diintegrasikan dengan baik ke dalam SoC berbasis MicroBlaze V dan dioperasikan melalui kontrol memory-mapped AXI4-Lite. Desain yang diimplementasikan mampu membangkitkan sinyal I2S yang diperlukan, mentransmisikan sampel audio stereo dalam format Philips I2S, serta mendukung frekuensi sampling yang dapat diprogram secara arbitrer melalui arsitektur berbasis NCO. Hal ini memungkinkan penggunaan frekuensi standar maupun non-standar dengan akurasi tinggi menggunakan satu clock referensi. Validasi perangkat keras menunjukkan bahwa desain bekerja sesuai tujuan pada platform yang diuji dan dapat menjadi dasar yang fleksibel untuk sistem keluaran audio digital berbasis FPGA.

Kata kunci : MicroBlaze V, I2S, Verilog, AXI4-Lite, FPGA, SoC, PCM5102A, Audio Digital

# ABSTRACT

This research focuses on the design and implementation of an I2S (Inter-IC Sound) transmitter peripheral using Verilog HDL, integrated into a System-on-Chip (SoC) based on the MicroBlaze V processor on FPGA. The peripheral is implemented as a custom AXI4-Lite slave IP so that it can be configured and supplied with audio data through memory-mapped registers from software running in Xilinx Vitis. The target platform is the Basys3 FPGA board, while audio output is generated through a PCM5102A DAC module. Supporting modules such as UART and clocking infrastructure are included only as system support, whereas the main contribution of the thesis is the RTL design, integration, and validation of the I2S transmitter peripheral. **The primary contribution of this research is the design of a Custom IP I2S Transmitter supporting arbitrary sampling frequencies via an NCO-based architecture, integrable as an AXI4-Lite peripheral on a MicroBlaze V SoC, without dependency on any third-party licensed audio IP.**

The methodology includes: (1) Verilog RTL design of an I2S transmitter that follows the Philips I2S format for stereo audio transmission, (2) design of an AXI4-Lite register interface for control, sample loading, and status observation, (3) integration of the custom I2S IP into a MicroBlaze V SoC using Vivado block design, (4) development of C test software in Xilinx Vitis for register configuration, sine-wave generation, and audio sample transmission through memory-mapped I/O, and (5) verification through synthesis, implementation, testbench (TB) simulation, oscilloscope observation, and hardware testing with the PCM5102A module.

The results show that the designed I2S transmitter IP can be integrated successfully into a MicroBlaze V-based SoC and operated through AXI4-Lite memory-mapped control. The implemented design is able to generate the required I2S signals, transmit stereo audio samples in Philips I2S format, and support programmable sample rates through an NCO-based architecture. This allows for arbitrary sampling frequencies (e.g., 44.1 kHz, 48 kHz, or non-standard rates) with high precision using a single reference clock. Hardware validation indicates that the design functions as intended on the tested platform and provides a flexible, reusable basis for FPGA-based digital audio output systems.

**Keywords**: MicroBlaze V, I2S, Verilog, AXI4-Lite, FPGA, SoC, PCM5102A, Digital Audio

# BAB I
# PENDAHULUAN

## 1.1 Latar Belakang

Perkembangan sistem embedded modern mendorong kebutuhan akan antarmuka audio digital yang fleksibel, dapat dikonfigurasi melalui perangkat lunak, dan mudah diintegrasikan ke dalam arsitektur System-on-Chip (SoC). Pada berbagai aplikasi seperti audio playback, perangkat edukasi, synthesizer sederhana, sistem notifikasi suara, dan eksperimen digital signal processing, data audio perlu dikirim dari prosesor ke perangkat audio eksternal secara sinkron dan andal. Dalam konteks ini, I2S (Inter-IC Sound) menjadi protokol yang sangat relevan karena dirancang khusus untuk transmisi data audio digital antarchip.

I2S merupakan standar antarmuka serial audio yang dikembangkan oleh Philips dan banyak digunakan untuk menghubungkan prosesor, codec, DAC, dan perangkat audio digital lain [1]. Protokol ini menggunakan tiga sinyal utama, yaitu *bit clock* (BCLK), *word select* (WS/LRCLK), dan *serial data* (SD). Relasi timing antar sinyal tersebut harus dipenuhi dengan ketat agar penerima audio dapat merekonstruksi sampel digital dengan benar. Dukungan terhadap berbagai *sample rate* dan *bit depth* membuat I2S cocok untuk implementasi audio pada FPGA, terutama ketika dibutuhkan fleksibilitas desain pada level register transfer.

FPGA menawarkan keunggulan utama dalam pengembangan sistem audio digital karena logika perangkat kerasnya dapat dikustomisasi sesuai kebutuhan aplikasi [2]. Dibandingkan pendekatan berbasis perangkat keras tetap, FPGA memungkinkan perancangan antarmuka komunikasi, logika buffer, pembagi clock, dan jalur data serial secara langsung dalam Verilog atau VHDL [3]. Ketika dikombinasikan dengan soft processor seperti MicroBlaze V, FPGA juga memberikan model kerja hardware-software co-design: fungsi real-time dan timing kritis ditangani oleh perangkat keras, sementara konfigurasi, pengiriman data, dan pengujian dikendalikan oleh perangkat lunak [4].

MicroBlaze V sebagai prosesor soft-core berbasis RISC-V [5] pada ekosistem AMD/Xilinx [6] menyediakan fondasi yang sesuai untuk integrasi peripheral kustom pada Vivado dan Vitis. Melalui antarmuka AXI4-Lite [7], peripheral dapat diakses dengan model memory-mapped I/O yang sederhana namun efektif. Dengan pendekatan ini, prosesor dapat menulis sampel audio, mengatur mode operasi, serta membaca status dari peripheral I2S menggunakan instruksi load/store biasa. Pendekatan tersebut memudahkan pengembangan perangkat lunak uji sekaligus menjaga modularitas desain perangkat keras.

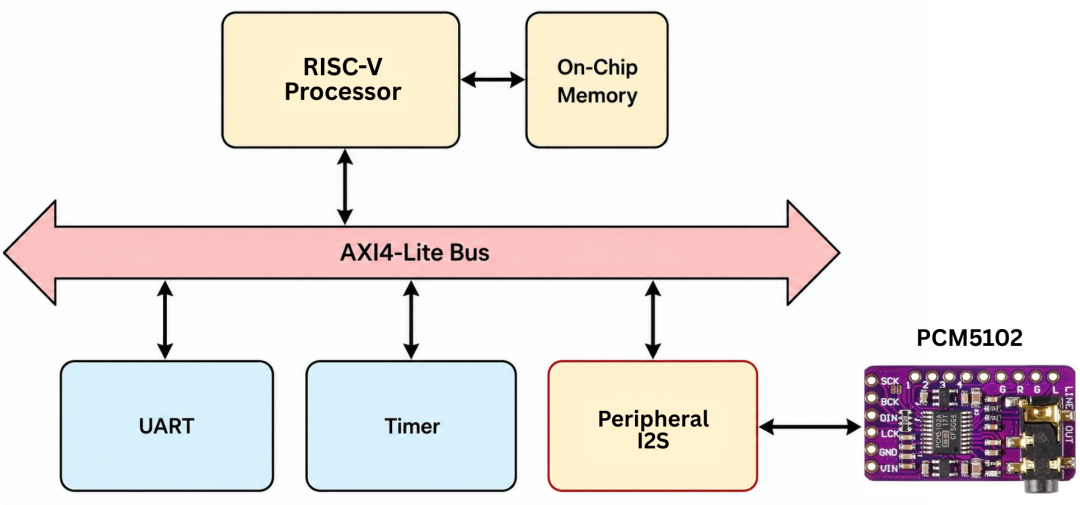
Pemilihan **MicroBlaze V** sebagai prosesor pada penelitian ini didasarkan pada beberapa pertimbangan teknis yang relevan. Dibandingkan dengan *ARM SoC* seperti Zynq-7000, MicroBlaze V tidak memerlukan lisensi prosesor ARM dan seluruh desain dapat berjalan pada FPGA seri Artix-7 yang lebih terjangkau seperti Basys3 tanpa *hard processor*. Dibandingkan dengan *MicroBlaze Classic* berbasis ISA Xilinx proprietary, MicroBlaze V menggunakan arsitektur **RISC-V** standar terbuka sehingga memudahkan migrasi ke toolchain open-source di masa depan. Dibandingkan pendekatan *bare-FPGA* (tanpa prosesor), penggunaan MicroBlaze V memungkinkan konfigurasi register dan pembangkitan sinyal uji audio dikendalikan oleh perangkat lunak C tanpa memerlukan finite-state-machine kustom yang lebih kompleks. Kombinasi MicroBlaze V dan AXI4-Lite pada FPGA Artix-7 dengan tool Vivado dan Vitis juga mewakili ekosistem yang umum digunakan pada penelitian sistem tertanam akademik.

Dalam praktik implementasi audio pada FPGA, permasalahan tidak hanya terletak pada pembangkitan sinyal I2S, tetapi juga pada kestabilan aliran data dan sinkronisasi antara perangkat lunak dengan serializer. Pada desain akhir, seluruh logika I2S berjalan pada domain `S_AXI_ACLK` 100 MHz sehingga register data dan serializer berada pada domain clock yang sama. Jika sampel berubah ketika word sedang diserialkan, keluaran audio dapat korup; oleh karena itu, sistem membutuhkan *shadow register* yang di-*latch* pada batas frame stereo serta mekanisme *handshake* melalui register `STATUS` agar firmware dapat mengirim sampel berikutnya tepat waktu. Pembangkitan `BCLK`, `WS`, dan `MCLK` dilakukan oleh akumulator fase DDS internal, sehingga frekuensi sampling dapat diprogram tanpa rekonfigurasi PLL eksternal. Kesesuaian pinout ke modul DAC eksternal [8] juga menjadi faktor penting yang memengaruhi keberhasilan sistem secara keseluruhan.

Berdasarkan kebutuhan tersebut, skripsi ini berfokus pada perancangan **IP core transmitter I2S berbasis Verilog** dengan antarmuka **AXI4-Lite** yang dapat diintegrasikan ke dalam SoC **MicroBlaze V** pada board **Basys3** [9]. Kode sumber dan dokumentasi proyek ini tersedia secara terbuka pada repositori GitHub[1]. Target keluaran audio adalah modul **PCM5102A** sebagai DAC eksternal. Desain difokuskan pada mode **TX-only**, format **Philips I2S**, stereo, dengan **slot 32 bit** dan lebar sampel yang dapat dipilih hingga 32 bit. Penelitian ini mencakup evolusi rancangan mulai dari desain dasar dengan empat mode frekuensi sampling (44,1 kHz, 48 kHz, 88,2 kHz, dan 96 kHz) hingga tahap **pengembangan dan optimasi** menggunakan teknik **DDS (Direct Digital Synthesis)** untuk mendukung **frekuensi sampling arbitrer** serta membuat IP bersifat **vendor-agnostic**. Penelitian ini mencakup perancangan RTL, integrasi Vivado Block Design [10], pengembangan perangkat lunak uji di Vitis [11], pengukuran sinyal, dan bukti keluaran audio.

[1] `https://github.com/adakhaddad/thesi2s`

**Kontribusi utama penelitian ini** adalah perancangan **Custom IP I2S Transmitter** yang mendukung frekuensi sampling arbitrer berbasis NCO dan dapat diintegrasikan sebagai peripheral AXI4-Lite pada SoC MicroBlaze V tanpa ketergantungan pada IP audio berlisensi pihak ketiga. Dengan demikian, sistem yang dibangun bersifat *open-source*, *vendor-agnostic*, dan dapat direplikasi pada platform FPGA lain tanpa biaya lisensi tambahan.




Gambar 1.1. Diagram arsitektur lengkap sistem SoC dengan IP I2S TX AXI4-Lite dan DAC PCM5102A.

## 1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah penelitian ini adalah:

1. Bagaimana merancang **IP core transmitter I2S** berbasis Verilog dengan antarmuka **AXI4-Lite** sehingga dapat dikontrol dari program Vitis melalui register memory-mapped, serta mendukung **frekuensi sampling arbitrer** dan bersifat **vendor-agnostic** melalui teknik DDS?
2. Bagaimana melakukan **verifikasi fungsional dan timing** melalui simulasi bila tersedia, pengukuran sinyal, dan uji keluaran audio pada PCM5102A?

## 1.3 Tujuan Penelitian

Tujuan penelitian ini disusun agar selaras dengan rumusan masalah yang telah ditetapkan, sekaligus menegaskan bahwa skripsi ini menghasilkan **IP komunikasi sendiri** sehingga sistem tidak bergantung pada *license* atau IP komersial pihak ketiga.

### 1.3.1 Tujuan perancangan dan integrasi perangkat keras

1. Merancang dan mengimplementasikan **IP komunikasi audio I2S TX** berbasis Verilog dengan antarmuka **AXI4-Lite slave**, register kontrol dan data, serializer **Philips**

**I2S**, serta mekanisme **shadow register** dan **status handshake** untuk pengiriman sampel audio stereo yang aman tanpa memakai IP komunikasi berlisensi dari pihak ketiga.

2. Mengimplementasikan pembangkitan detak audio berbasis **DDS/NCO** pada domain 100 MHz sehingga mendukung **frekuensi sampling arbitrer**, lalu mengintegrasikan IP ke dalam **Vivado Block Design** pada FPGA **Basys3** beserta pemetaan pin pada file `.xdc`.

### 1.3.2 Tujuan integrasi SoC dan perangkat lunak

1. Membangun SoC berbasis **MicroBlaze V** dengan AXI interconnect, BRAM, Clocking Wizard, dan akses ke peripheral I2S hasil rancangan sendiri melalui **Vitis**.
2. Mengembangkan aplikasi **Vitis** dalam bahasa C untuk konfigurasi register, pembangkitan sinyal uji audio, pengaturan kanal dan mode sampling, serta pengiriman sampel berbasis memory-mapped I/O ke IP komunikasi yang dibuat sendiri.

### 1.3.3 Tujuan verifikasi dan validasi

1. Memverifikasi sistem melalui pengamatan sinyal `i2s_bclk`, `i2s_ws`, `i2s_data`, status latch, serta bukti keluaran audio pada PCM5102A.
2. Melakukan synthesis dan implementation pada FPGA Xilinx Artix-7 serta menganalisis *resource utilization* seperti LUT, FF, dan BRAM.

## 1.4 Batasan Penelitian

Batasan penelitian dalam pengembangan **IP core I2S TX AXI4-Lite** ini meliputi:

### 1.4.1 Batasan perangkat keras dan RTL

1. Modul I2S dirancang sebagai **transmitter (TX-only)**; tidak mencakup receiver atau mode full-duplex.
2. Format audio dibatasi pada **Philips I2S**, stereo, **24 bit** per saluran dalam **slot 32 bit**; tidak mencakup TDM, left-justified, right-justified, atau multi-channel.
3. Dukungan frekuensi sampling dapat diprogram melalui register `SAMPLE_RATE_HZ` dalam rentang 1–195.312 Hz (batas aman DDS pada clock 100 MHz); pengujian utama difokuskan pada nilai standar 44,1 kHz, 48 kHz, 88,2 kHz, dan 96 kHz serta beberapa nilai non-standar.
4. Transfer data audio ke IP menggunakan **programmed I/O** berbasis register dan tidak mencakup integrasi **FIFO** maupun **DMA**.

5. Pengujian perangkat keras dilakukan pada board **Basys3** dengan modul **PCM5102A**; hasil pada platform lain dapat berbeda.

### 1.4.2 Batasan SoC dan perangkat lunak

1. Prosesor yang digunakan adalah **MicroBlaze V** pada ekosistem Vivado dan Vitis.
2. Lingkungan perangkat lunak yang digunakan adalah **bare-metal** tanpa RTOS atau Linux pada ruang lingkup utama.
3. UART digunakan untuk debug dan logging, bukan sebagai jalur streaming audio utama.

### 1.4.3 Batasan pengujian dan dokumentasi

1. Pengujian dibatasi pada simulasi *testbench*, osiloskop, dan uji dengar sederhana pada keluaran audio.
2. Dokumentasi disusun untuk kebutuhan skripsi dan tidak dimaksudkan sebagai datasheet komersial lengkap.

## 1.5 Manfaat Penelitian

### 1.5.1 Manfaat akademik

1. Menghasilkan contoh perancangan **IP I2S TX AXI4-Lite** berbasis Verilog yang dapat digunakan sebagai acuan pembelajaran audio digital pada FPGA.
2. Menyediakan studi kasus hardware-software co-design pada FPGA yang melibatkan RTL, integrasi SoC, dan pemrograman embedded C.
3. Memberikan dokumentasi register map, strategi *status handshake*, dan validasi sinyal I2S yang dapat digunakan kembali pada penelitian sejenis.

### 1.5.2 Manfaat praktis

1. Menyediakan dasar untuk proyek **audio output** berbasis FPGA seperti generator nada, alarm, dan sistem multimedia sederhana.
2. Menunjukkan pola desain **register-mapped I/O + status polling** yang dapat diadaptasi pada peripheral streaming sederhana lain.
3. Menjadi fondasi pengembangan lebih lanjut menuju sistem audio digital yang lebih kompleks, misalnya integrasi DMA atau dukungan Linux/ASoC.

### 1.5.3 Manfaat pengembangan keahlian

1. Memberikan pengalaman praktis dalam Verilog HDL, AXI4-Lite, integrasi Vivado Block Design, dan pengembangan aplikasi Vitis.

2. Mengembangkan kemampuan debugging hardware-software melalui simulasi *testbench*, pengukuran sinyal, dan analisis hasil implementasi.
3. Menjadi portofolio yang relevan untuk bidang FPGA, embedded systems, dan desain SoC.

## 1.6 Sistematika Penulisan

**Bab I** membahas pendahuluan yang mencakup latar belakang, rumusan masalah, tujuan, batasan, manfaat, dan sistematika penulisan.

**Bab II** memuat tinjauan pustaka dan dasar teori mengenai audio digital, protokol I2S, antarmuka AXI4-Lite, sinkronisasi lintas domain clock, Clocking Wizard, MicroBlaze V, dan PCM5102A.

**Bab III** menyajikan metode penelitian yang mencakup alat dan bahan, metode verifikasi, serta alur tahapan tugas akhir secara sistematis.

**Bab IV** membahas perancangan dan implementasi yang merinci spesifikasi fungsional IP, arsitektur register map, integrasi pada Vivado Block Design, serta detail logika RTL (DDS dan serializer).

**Bab V** menyajikan hasil dan pembahasan yang mencakup pengujian sinyal digital, analisis akurasi frekuensi sampling, identifikasi bug firmware, serta verifikasi akhir sistem.

**Bab VI** berisi kesimpulan dan saran pengembangan.

# BAB II
# TINJAUAN PUSTAKA DAN DASAR TEORI

## 2.1 Tinjauan Pustaka

Bagian ini membahas penelitian terdahulu yang relevan dengan implementasi protokol I2S pada FPGA, integrasi peripheral kustom pada SoC berbasis prosesor soft-core, serta penggunaan bus AXI4-Lite dan sinkronisasi lintas domain clock.

### 2.1.1 Penelitian tentang implementasi I2S pada FPGA

Implementasi antarmuka I2S pada FPGA telah dilaporkan dalam beberapa konteks. [12] merancang antarmuka I2S pada FPGA untuk menghubungkan prosesor dengan codec audio, menunjukkan dukungan *sample rate* hingga 96 kHz menggunakan pendekatan RTL langsung tanpa bus sistem. Sementara itu, [13] mengintegrasikan logika I2S bersama logic analyzer pada FPGA berbiaya rendah untuk kebutuhan instrumentasi.

Penelitian-penelitian tersebut mengonfirmasi kelayakan pembangkitan sinyal I2S pada FPGA, namun sebagian besar masih terbatas pada level simulasi atau pengujian RTL terisolasi. Penelitian ini berfokus pada pendekatan *hardware-software co-design* secara sistemik, yang mengintegrasikan register map, interkoneksi bus, dan validasi fungsional menggunakan perangkat lunak yang berjalan pada prosesor soft-core [14].

### 2.1.2 Penelitian tentang prosesor soft-core dan integrasi SoC

Pengembangan arsitektur RISC-V sebagai ISA terbuka [5] mendasari perancangan prosesor soft-core modern seperti MicroBlaze V pada ekosistem AMD/Xilinx [6]. Integrasi peripheral kustom ke dalam SoC berbasis bus AXI dieksplorasi oleh [15], yang menekankan aspek modularitas dan skalabilitas interkoneksi. Metodologi hardware-software co-design dan pengembangan driver bare-metal untuk peripheral kustom pada FPGA diulas secara komprehensif oleh [4].

### 2.1.3 Penelitian tentang sinkronisasi lintas domain clock (CDC)

Dalam sistem multi-clock, sinkronisasi data antar-domain (*Clock Domain Crossing*, CDC) sangat krusial. [16] menganalisis masalah metastabilitas dan memaparkan solusi flip-flop ganda. Pada rancangan final penelitian ini (`i2s_1_0`), serializer I2S berjalan sepenuhnya pada domain clock bus AXI 100 MHz, sehingga masalah CDC pada jalur data audio ditiadakan. Koordinasi aliran data diselesaikan melalui status handshake berbasis register.

### 2.1.4 Penelitian tentang penjadwalan firmware berbasis counter siklus

Penjadwalan periodik pada bare-metal sering kali memanfaatkan CSR `mcycle` pada arsitektur RISC-V sebagai referensi timing presisi tinggi [17]. Pada tahap awal penelitian ini, fungsi tunda `usleep()` dan register `mcycle` sempat digunakan untuk pengiriman sampel. Namun, sistem akhir beralih menggunakan polling bit register `STATUS` yang menyinkronkan pengiriman data secara fisik dengan detak konsumsi perangkat keras untuk mengeliminasi akumulasi keterlambatan (*drift*).

### 2.1.5 Gap penelitian dan kontribusi

Penelitian ini mengisi celah dokumentasi perancangan IP core I2S TX kustom untuk SoC MicroBlaze V yang mencakup seluruh alur RTL, register map, perangkat lunak bare-metal, analisis bug firmware (seperti kesalahan justifikasi bit dan sinkronisasi timing), serta verifikasi fisik pada platform hemat daya berbasis Basys3.

Kontribusi utama penelitian ini adalah:

1. Merancang IP core I2S transmitter AXI4-Lite berbasis **NCO** (*Numerically Controlled Oscillator*) desimal yang mendukung **arbitrary sample rate** tanpa PLL eksternal.
2. Mengintegrasikan sistem SoC berbasis MicroBlaze V pada FPGA Artix-7 secara hemat sumber daya.
3. Menyediakan solusi hardware-software handshake berbasis status-polling untuk mencegah data corruption dan drift.
4. Mengidentifikasi dan mendokumentasikan penyelesaian tiga bug kritis pada lapisan firmware.

## 2.2 Dasar Teori

### 2.2.1 Arsitektur MicroBlaze V dan Memory-Mapped I/O

MicroBlaze V adalah soft-core processor 32-bit berbasis arsitektur RISC-V (RV32I) dari AMD/Xilinx [6]. Prosesor berkomunikasi dengan peripheral melalui model *Memory-Mapped I/O* (MMIO), di mana register kontrol dan data peripheral dipetakan ke alamat memori unik. Akses register dilakukan menggunakan instruksi load/store standar (`lw` / `sw`) melalui perantara fungsi pustaka `xil_io.h` seperti `Xil_Out32` dan `Xil_In32`.

Untuk menjaga integritas data tanpa FIFO atau DMA, dikembangkan mekanisme *Hardware-Software Handshake*. IP I2S Transmitter menyediakan bit status (`STATUS[0]`) yang berubah logika (*toggle*) setiap kali perangkat keras selesai memindahkan sampel dari register AXI ke *shadow register* internal untuk diserialisasikan.

Perangkat lunak melakukan polling terhadap bit status ini sebelum mengirimkan sampel berikutnya, sehingga laju transfer data terkunci secara fisik dengan laju transmisi clock audio eksternal.

### 2.2.2 Protokol Komunikasi AXI4-Lite

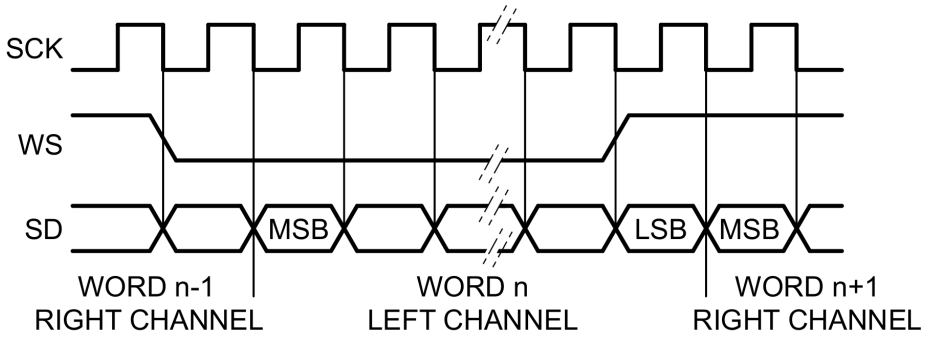
AXI4-Lite adalah varian sederhana dari protokol AMBA AXI4 yang dioptimalkan untuk akses register kontrol dan status [7]. Transaksi AXI4-Lite bersifat synchronous dan menggunakan mekanisme jabat tangan (*handshake*) dua arah berbasis sinyal `VALID` (dinyalakan oleh pengirim) dan `READY` (dinyalakan oleh penerima). Protokol ini terdiri atas lima kanal transaksi independen:

1. **Write Address (AW)**: Mengirimkan alamat tujuan penulisan.
2. **Write Data (W)**: Mengirimkan data yang akan ditulis.
3. **Write Response (B)**: Mengirimkan status respon penulisan dari slave ke master.
4. **Read Address (AR)**: Mengirimkan alamat data yang akan dibaca.
5. **Read Data (R)**: Mengirimkan data hasil pembacaan beserta status respon.

### 2.2.3 Protokol Komunikasi Audio I2S

I2S (*Inter-IC Sound*) merupakan protokol komunikasi serial sinkron tiga-kawat khusus untuk data audio digital PCM [1]. Tiga sinyal utama protokol ini meliputi:

- **Bit Clock (BCLK)**: Menentukan timing perpindahan bit data serial.
- **Word Select (WS / LRCLK)**: Menentukan kanal aktif (kiri = LOW, kanan = HIGH). Frekuensi WS sama dengan *sample rate* ($F_s$).
- **Serial Data (SD / DATA)**: Jalur transmisi bit data serial (MSB-first).




Gambar 2.1. Diagram timing sinyal protokol I2S standar (Sumber: NXP).

Pada format Philips I2S standar, transisi `WS` terjadi satu pulsa BCLK sebelum bit MSB dikirim (*one-bit delay*) untuk sinkronisasi receiver (Gambar 2.1). Penelitian ini menggunakan konfigurasi **stereo 24-bit di dalam slot 32-bit** per saluran (total 64 pulsa

BCLK per frame stereo). Desain slot 32-bit menyederhanakan penyelarasan data pada bus AXI 32-bit. Hubungan frekuensi dinyatakan sebagai:

$$f_{\mathrm{BCLK}} = 64 \times F_s \tag{2-1}$$

### 2.2.4 Pembangkitan Clock Audio Berbasis NCO

Pembangkitan detak audio murni digital dilakukan menggunakan *Numerically Controlled Oscillator* (NCO) untuk menghindari ketergantungan pada unit analog PLL/MMCM vendor FPGA [18]. Akumulator fase biner beralih dengan modulus $2^N$, tetapi pembulatan nilai increment dapat menyebabkan deviasi frekuensi kumulatif. Untuk mengatasinya, diimplementasikan **Akumulator Fase Desimal** dengan modulus $M$ yang nilainya disesuaikan dengan frekuensi clock referensi sistem ($f_{\mathrm{ref}} = 100$ MHz), sehingga $M = 100.000.000$.

Persamaan pembaruan fase pada setiap siklus clock sistem adalah:

$$\theta(t) = (\theta(t-1) + I) \pmod{M} \tag{2-2}$$

Di mana $I$ adalah nilai inkremen fase. Ketika $\theta(t-1)+I \geq M$, terjadi luapan (*wrap*) yang memicu perubahan logika (*toggle*) clock keluaran. Untuk BCLK, satu periode penuh (dua kali toggle) membutuhkan inkremen sebesar:

$$I_{\mathrm{BCLK}} = 2 \times f_{\mathrm{BCLK}} = 2 \times 64 \times F_s = 128 \times F_s \tag{2-3}$$

Untuk Master Clock (MCLK) sebesar $256 \times F_s$, inkremennya adalah:

$$I_{\mathrm{MCLK}} = 2 \times f_{\mathrm{MCLK}} = 2 \times 256 \times F_s = 512 \times F_s \tag{2-4}$$

Dalam kode RTL Verilog, perkalian ini diimplementasikan secara efisien lewat pergeseran bit (*bit-shift*):

```
wire [31:0] bclk_toggle_rate_hz = {sample_rate_sync, 7'b0}; // Fs * 128
wire [31:0] mclk_toggle_rate_hz = {sample_rate_sync, 9'b0}; // Fs * 512
```

Perbandingan kedua teknik disajikan pada Tabel 2.1. Pendekatan NCO dipilih karena memberikan fleksibilitas penuh untuk memprogram frekuensi sampling secara dinamis dari perangkat lunak.

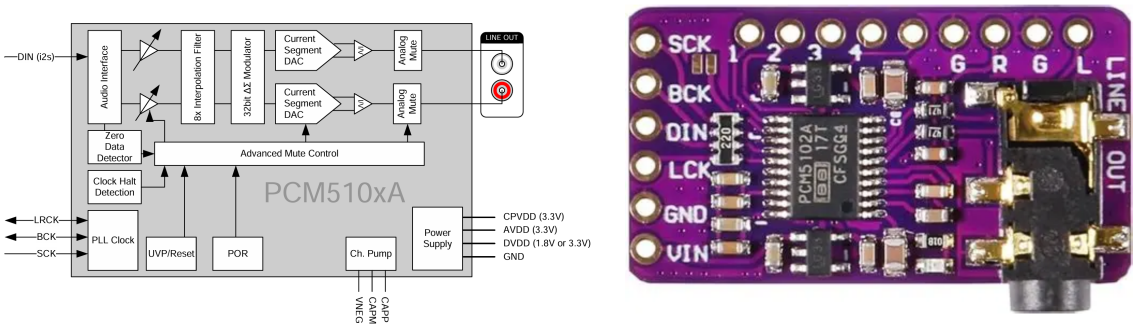
### 2.2.5 DAC Audio PCM5102A

PCM5102A adalah chip Digital-to-Analog Converter (DAC) 24-bit stereo dari Texas Instruments dengan keluaran analog level-line langsung tanpa filter eksternal [8].

Tabel 2.1. Perbandingan pembangkitan clock berbasis PLL dan NCO.

| **Fitur** | **PLL / MMCM (Analog)** | **NCO (Digital)** |
|---|---|---|
| *Jitter* | Sangat rendah (diredam filter analog). | Sebesar periode clock sistem ($1/f_{\text{ref}}$). |
| Fleksibilitas | Terbatas pada rasio perkalian integer. | Mendukung frekuensi arbitrer secara instan. |
| Portabilitas | Spesifik vendor (*vendor-locked*). | *Vendor-agnostic* (RTL Verilog portabel). |

DAC ini dapat beroperasi tanpa master clock (MCLK) eksternal dengan membangkitkan detak internal dari BCLK melalui unit PLL internalnya.




(a) Diagram blok internal fungsional. (b) Foto modul fisik PMOD PCM5102A.

Gambar 2.2. Spesifikasi arsitektur dan modul fisik DAC PCM5102A.

Diagram blok fungsional dan foto fisik modul DAC PCM5102A disajikan pada Gambar 2.2. Modul eksternal ini dihubungkan langsung ke pin PMOD Basys3 untuk menerima data I2S dan menghasilkan keluaran audio stereo.

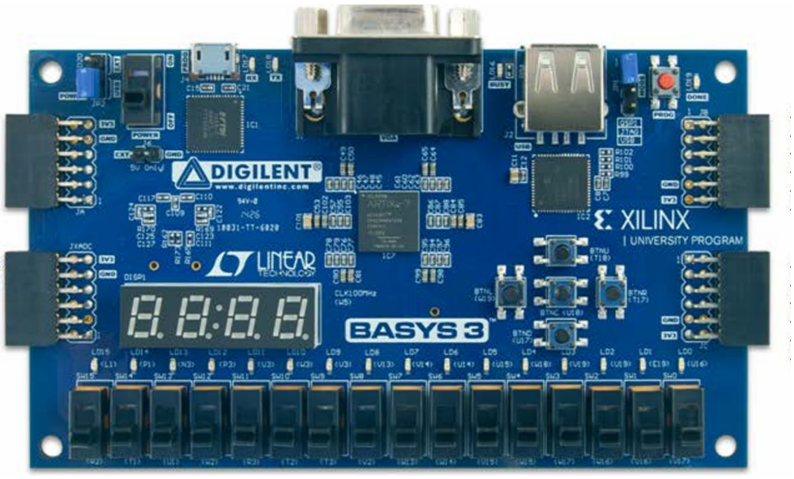
### 2.2.6 FPGA dan Board Basys3

FPGA (*Field-Programmable Gate Array*) memungkinkan kustomisasi jalur data perangkat keras secara langsung melalui deskripsi bahasa pemrograman RTL [19]. Pengujian hardware pada penelitian ini menggunakan board pengembangan **Basys3** berbasis chip Xilinx Artix-7 (XC7A35T-1CPG236C) [9].

Board Basys3 (Gambar 2.3) menyediakan osilator clock 100 MHz, koneksi micro-USB untuk pemograman JTAG dan UART, serta empat port PMOD standar 12-pin untuk ekspansi periferal eksternal seperti DAC.

### 2.2.7 Alur Pengembangan Vivado dan Vitis

AMD Vivado Design Suite digunakan untuk membangun bagian perangkat keras sistem FPGA, mulai dari penulisan dan simulasi RTL, pengemasan custom IP, penyu-




Gambar 2.3. Board FPGA Basys3 Artix-7 yang digunakan dalam penelitian.

sunan Block Design, hingga pemetaan pin melalui file constraint `.xdc` [10]. Setelah rancangan hardware selesai, Vivado menjalankan tiga tahap utama:

1. **Synthesis**: menerjemahkan RTL dan Block Design menjadi netlist yang tersusun dari sumber daya logika FPGA, seperti LUT, flip-flop, BRAM, dan DSP.
2. **Implementation**: menempatkan sumber daya hasil sintesis pada FPGA (*placement*), menghubungkan jalurnya (*routing*), dan memeriksa pemenuhan batasan timing.
3. **Generate Bitstream**: menghasilkan file konfigurasi `.bit` yang digunakan untuk memprogram FPGA.

Untuk pengembangan perangkat lunak, Vivado mengekspor deskripsi hardware sebagai file `.xsa` (*Xilinx Support Archive*). File ini memuat informasi prosesor MicroBlaze V, peta alamat peripheral AXI4-Lite, konfigurasi interrupt, clock, serta bitstream yang terkait dengan platform hardware. AMD Vitis mengimpor file `.xsa` tersebut untuk membentuk *platform project* dan *Board Support Package* (BSP) [11]. Berdasarkan platform itu, aplikasi bare-metal ditulis dalam bahasa C pada file `.c`, lalu dikompilasi menjadi berkas executable `.elf`. Saat pengujian, FPGA diprogram menggunakan bitstream dan Vitis memuat aplikasi `.elf` ke memori sistem melalui JTAG.

Dengan demikian, alur pengembangan hardware dan software pada penelitian ini adalah:

$$\begin{aligned}\text{RTL/Block Design} &\rightarrow \text{Synthesis} \rightarrow \text{Implementation} \\ &\rightarrow \text{Bitstream dan XSA} \rightarrow \text{Platform Vitis} \rightarrow \text{Aplikasi C}\end{aligned} \tag{2-5}$$

### 2.2.8 Verilog HDL, Perbandingan IP, dan Analisis Metode

Logika IP core diimplementasikan menggunakan Verilog HDL untuk mendeskripsikan register kontrol, mesin NCO, serializer, dan bus interface slave AXI4-Lite [3]. Desain modular ini bersifat *vendor-agnostic* karena ditulis dalam RTL standar tanpa memakai pustaka hardware proprietari.

Sebagai pembanding, AMD/Xilinx menyediakan LogiCORE I2S Transmitter IP yang kompleks [20], namun membutuhkan AXI4-Stream interface, DMA controller, dan clocking analog MMCM eksternal. IP custom yang dirancang dalam penelitian ini berfokus pada mode **TX-only** minimalis melalui AXI4-Lite memory-mapped register, sehingga sangat efisien dalam penggunaan sumber daya logika FPGA dan mudah dikontrol secara langsung oleh CPU soft-core bare-metal.

# BAB III
# METODE PENELITIAN

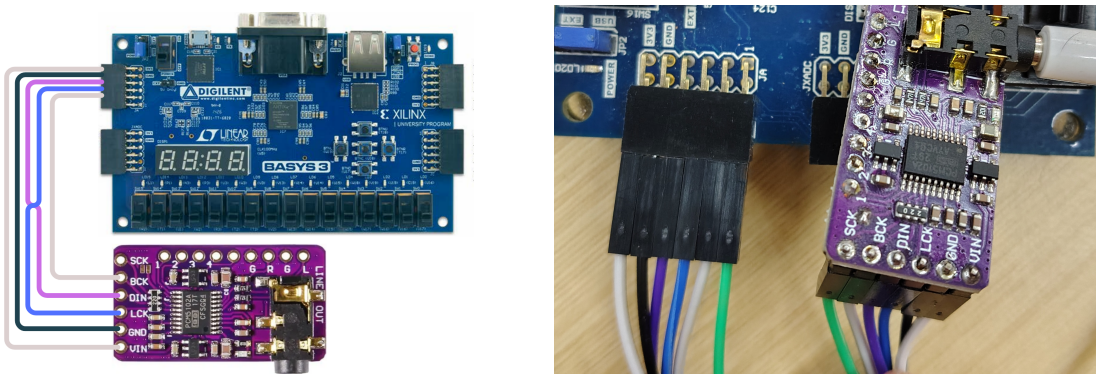
## 3.1 Alat dan Bahan

Penelitian ini menggunakan kombinasi perangkat keras dan perangkat lunak untuk perancangan, integrasi SoC, dan validasi fisik sistem I2S Transmitter.

### 3.1.1 Perangkat keras

#### 3.1.1.1 Perangkat pengembangan dan target hardware

- **Laptop Lenovo ThinkPad**: Intel Core i5, RAM 8 GB, Windows 10 Pro. Digunakan untuk merancang RTL, menulis kode driver, dan menyusun laporan.
- **Board FPGA Basys3**: Berbasis AMD/Xilinx Artix-7 (XC7A35T), sebagai platform SoC soft-core utama [9].
- **Modul DAC Audio PCM5102A**: Berperan sebagai receiver I2S eksternal yang dihubungkan ke PMOD Basys3 untuk menghasilkan sinyal audio analog [8].




(a) Diagram koneksi pin. (b) Implementasi fisik.

Gambar 3.1. Koneksi antarmuka pin antara FPGA Basys3 dan modul DAC PCM5102A.

#### 3.1.1.2 Perangkat pengukuran dan pembantu

- **Osiloskop Digital TEXIO DCS-1102B**: Mengukur dan memverifikasi integritas sinyal BCLK, WS, dan DATA.
- **Headset stereo dan kabel jumper**: Untuk pengujian dengar audio analog langsung dari keluaran DAC.

### 3.1.2 Perangkat lunak

- **AMD Vivado 2021.1**: Tool utama untuk perancangan RTL Verilog, simulasi behavioral (XSim), sintesis, implementasi, dan perancangan SoC (Block Design) [10].

- **AMD Vitis 2021.1**: Platform pengembangan perangkat lunak C bare-metal untuk prosesor MicroBlaze V [11].
- **PuTTY dan Trae IDE**: PuTTY digunakan sebagai terminal UART logging, sedangkan Trae IDE untuk penyuntingan kode sumber dan penyusunan naskah LaTeX.

## 3.2 Metode yang Digunakan

### 3.2.1 Metode perancangan hardware

RTL dirancang secara modular menggunakan Verilog HDL yang memisahkan bagian datapath dan control path [3]. Desain difokuskan pada serializer I2S standar Philips, register data, shadow register untuk perlindungan data, serta modul clock desimal NCO agar rancangan bersifat *vendor-agnostic* dan mendukung *sample rate* arbitrer.

### 3.2.2 Metode verifikasi dan simulasi

Verifikasi dilakukan lewat simulasi testbench di Vivado Simulator untuk memastikan fungsionalitas sinyal I2S. Selanjutnya dilakukan validasi fisik dengan mengamati timing clock dan bit data serial pada osiloskop serta mendengarkan hasil reproduksi suara dari DAC.

### 3.2.3 Metode integrasi SoC dan pengembangan software

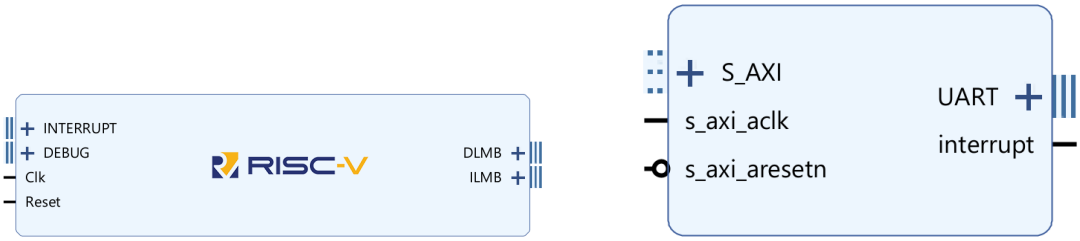
IP kustom dikemas menggunakan Vivado IP Packager lalu dimasukkan ke dalam Block Design untuk dihubungkan dengan MicroBlaze V melalui AXI Interconnect [21]. Firmware bare-metal dikembangkan di Vitis untuk mengontrol peripheral, menginisialisasi parameter register, melakukan generator DDS software, dan mengelola sinkronisasi pengiriman data lewat status polling.

## 3.3 Perancangan Block Design Sistem

Perancangan Block Design dilakukan untuk menyatukan custom IP I2S ke dalam sistem SoC berbasis prosesor MicroBlaze V agar dapat diakses oleh program perangkat lunak.

### 3.3.1 Integrasi SoC MicroBlaze V

Vivado IP Integrator digunakan untuk menghubungkan komponen SoC: prosesor soft-core MicroBlaze V [6], memori lokal BRAM, Processor System Reset, AXI SmartConnect, AXI UARTLite untuk komunikasi terminal, dan custom IP I2S. Alamat dasar (Base Address) peripheral ditentukan secara otomatis oleh Vivado dan dipetakan ke dalam header file `xparameters.h` di Vitis.




(a) IP MicroBlaze V. (b) IP AXI UARTLite.

Gambar 3.2. Komponen IP pendukung utama pada Block Design Vivado.

### 3.3.2 Perancangan Clocking Audio Berbasis NCO

Clock audio diturunkan secara sinkron dari clock sistem AXI 100 MHz menggunakan teknik *Numerically Controlled Oscillator* (NCO) desimal modulo 100.000.000 (penjelasan matematis rinci dibahas pada Bab II). Sinyal detak yang dihasilkan meliputi **BCLK** ($64 \times F_s$) untuk serializer, serta **MCLK** ($256 \times F_s$) sebagai referensi clock DAC eksternal.

### 3.3.3 Perancangan Register AXI4-Lite

Akses register IP I2S oleh MicroBlaze V dilakukan melalui alamat memory-mapped (MMIO) berbasis antarmuka bus AXI4-Lite. Register internal IP terdiri dari `DATA_LEFT` (0x00), `DATA_RIGHT` (0x04), `CONTROL` (0x08), dan `STATUS` (0x0C). Peta register dan penjelasan fungsi bitfield sepenuhnya identik dengan detail perancangan yang disajikan pada Tabel **??** dan Tabel **??** di Bab IV (halaman **??**).

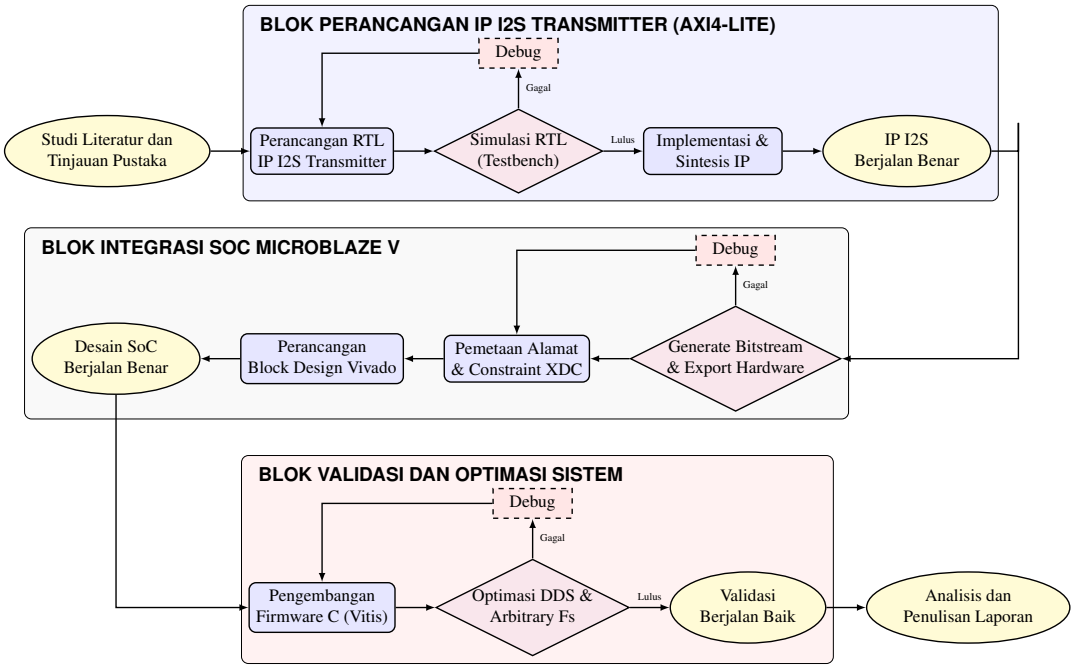
Untuk sinkronisasi pengiriman data, firmware melakukan polling terhadap bit status `STATUS[0]` yang berubah logika setiap kali perangkat keras memindahkan sampel dari register AXI ke shadow register internal di awal frame I2S.

### 3.3.4 Signal Generator dan Constraint FPGA

Signal generator digital dibangun pada sisi software untuk menghasilkan sampel sinus atau kotak yang kemudian disalurkan ke register data peripheral. Pin eksternal I2S dipetakan ke port PMOD Basys3 menggunakan file constraint `.xdc` dengan standar level tegangan LVCMOS33. Setelah Block Design divalidasi, dilakukan sintesis, implementasi, generate bitstream, dan ekspor hardware platform (`.xsa`) ke Vitis.

## 3.4 Alur Tugas Akhir

Penelitian ini mengikuti alur pengerjaan yang terbagi dalam tiga blok utama: perancangan IP I2S, integrasi SoC MicroBlaze V, serta validasi dan optimasi sistem. Diagram alir lengkap disajikan pada Gambar 3.3.

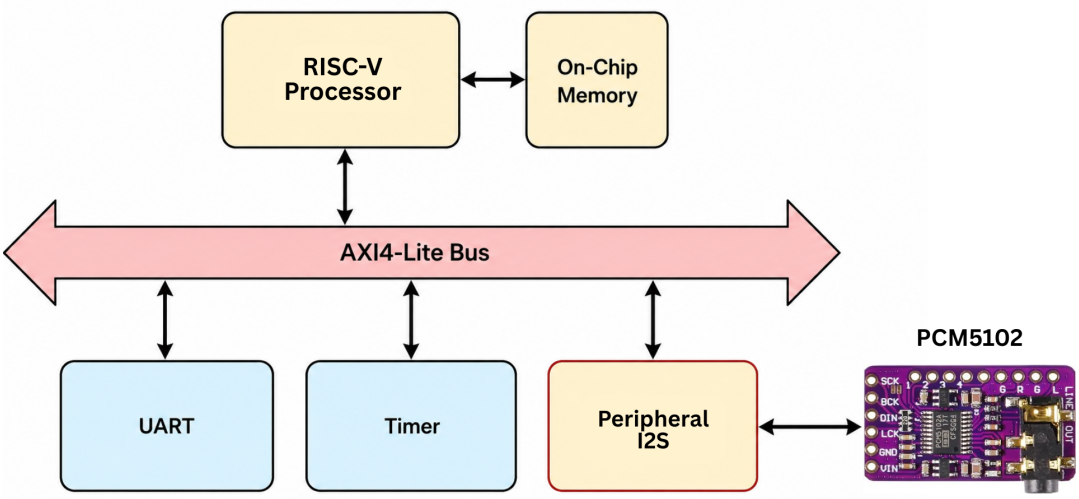



Gambar 3.3. Diagram alir pengerjaan penelitian tugas akhir skripsi.

# BAB IV
# PERANCANGAN DAN IMPLEMENTASI

## 4.1 Implementasi IP I2S

Bab ini mendokumentasikan realisasi akhir IP I2S berdasarkan source code aktual pada modul RTL dan firmware. Implementasi akhir terdiri atas wrapper `i2s.v`, core `i2s_slave_lite_v1_0_S00_AXI.v`, dan aplikasi bare-metal `helloworld.c`. Berbeda dari rancangan awal yang masih menempatkan generator gelombang di RTL, implementasi final memindahkan pembangkitan sampel audio ke firmware, sedangkan RTL difokuskan pada antarmuka AXI4-Lite, pengaturan register, pembangkitan clock audio, dan serializer Philips I2S.




Gambar 4.1. Arsitektur sistem akhir yang menghubungkan prosesor, UART, AXI4-Lite, dan IP I2S.

### 4.1.1 Struktur Modul RTL

Struktur RTL dibangun dari wrapper yang mempertahankan nama IP `i2s` dan modul inti `i2s_slave_lite_v1_0_S00_AXI`. Wrapper ini dipakai agar integrasi pada Vivado Block Design tetap kompatibel dengan repository IP yang sudah ada.

```
1 module i2s #
2 (
3     parameter integer C_S00_AXI_DATA_WIDTH = 32,
4     parameter integer C_S00_AXI_ADDR_WIDTH = 4
5 )
6 (
7     input  wire audio_clk,
8     output wire i2s_mclk,
```

```
    output wire i2s_bclk,
    output wire i2s_ws,
    output wire i2s_data,

    input  wire                          s00_axi_aclk,
    input  wire                          s00_axi_aresetn,
    input  wire [C_S00_AXI_ADDR_WIDTH-1:0] s00_axi_awaddr,
    input  wire [2:0]                    s00_axi_awprot,
    input  wire                          s00_axi_awvalid,
    output wire                          s00_axi_awready,
    input  wire [C_S00_AXI_DATA_WIDTH-1:0] s00_axi_wdata,
    input  wire [(C_S00_AXI_DATA_WIDTH/8)-1:0] s00_axi_wstrb,
    input  wire                          s00_axi_wvalid,
    output wire                          s00_axi_wready,
    output wire [1:0]                    s00_axi_bresp,
    output wire                          s00_axi_bvalid,
    input  wire                          s00_axi_bready,
    input  wire [C_S00_AXI_ADDR_WIDTH-1:0] s00_axi_araddr,
    input  wire [2:0]                    s00_axi_arprot,
    input  wire                          s00_axi_arvalid,
    output wire                          s00_axi_arready,
    output wire [C_S00_AXI_DATA_WIDTH-1:0] s00_axi_rdata,
    output wire [1:0]                    s00_axi_rresp,
    output wire                          s00_axi_rvalid,
    input  wire                          s00_axi_rready
);

    i2s_slave_lite_v1_0_S00_AXI #(
        .C_S_AXI_DATA_WIDTH(C_S00_AXI_DATA_WIDTH),
        .C_S_AXI_ADDR_WIDTH(C_S00_AXI_ADDR_WIDTH)
    ) i2s_slave_lite_v1_0_S00_AXI_inst (
        .i2s_mclk         (i2s_mclk),
        .i2s_bclk         (i2s_bclk),
        .i2s_ws           (i2s_ws),
        .i2s_data         (i2s_data),
        .S_AXI_ACLK       (s00_axi_aclk),
        .S_AXI_ARESETN    (s00_axi_aresetn),
        .S_AXI_AWADDR     (s00_axi_awaddr),
        .S_AXI_AWPROT     (s00_axi_awprot),
        .S_AXI_AWVALID    (s00_axi_awvalid),
        .S_AXI_AWREADY    (s00_axi_awready),
```

```
50      .S_AXI_WDATA    (s00_axi_wdata),
51      .S_AXI_WSTRB    (s00_axi_wstrb),
52      .S_AXI_WVALID   (s00_axi_wvalid),
53      .S_AXI_WREADY   (s00_axi_wready),
54      .S_AXI_BRESP    (s00_axi_bresp),
55      .S_AXI_BVALID   (s00_axi_bvalid),
56      .S_AXI_BREADY   (s00_axi_bready),
57      .S_AXI_ARADDR   (s00_axi_araddr),
58      .S_AXI_ARPROT   (s00_axi_arprot),
59      .S_AXI_ARVALID  (s00_axi_arvalid),
60      .S_AXI_ARREADY  (s00_axi_arready),
61      .S_AXI_RDATA    (s00_axi_rdata),
62      .S_AXI_RRESP    (s00_axi_rresp),
63      .S_AXI_RVALID   (s00_axi_rvalid),
64      .S_AXI_RREADY   (s00_axi_rready)
65  );
66
67  endmodule
```

Listing IV.1. Wrapper IP `i2s.v` yang meneruskan antarmuka AXI4-Lite dan keluaran I2S ke core.

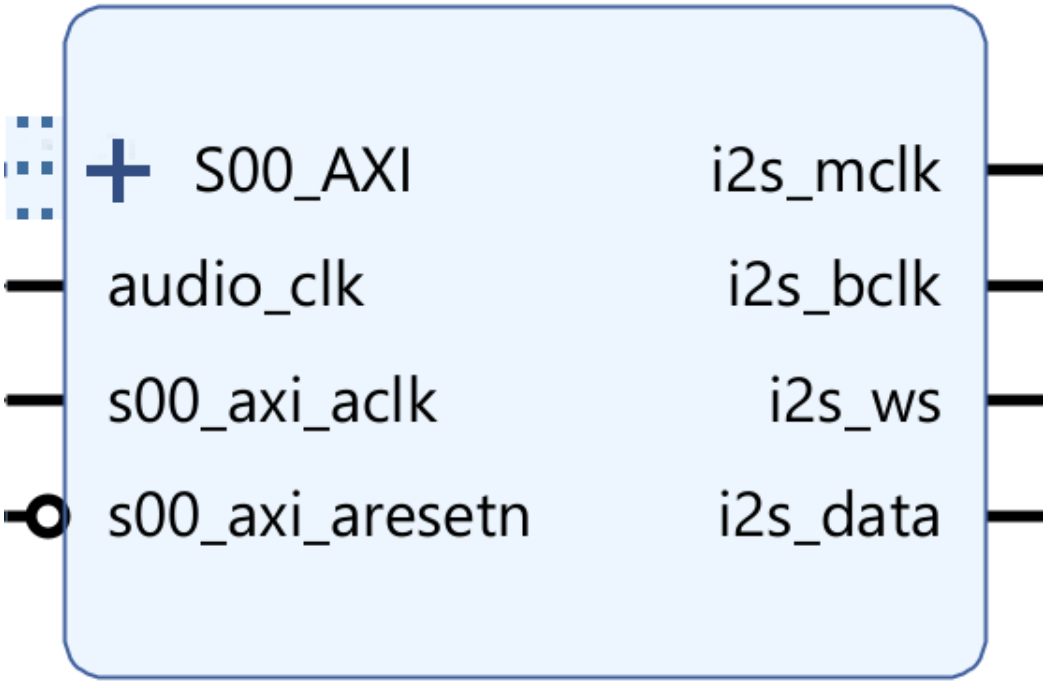
Potongan tersebut menunjukkan bahwa wrapper hanya berperan sebagai lapisan kompatibilitas. Sinyal keluaran I2S diteruskan langsung dari core, sedangkan antarmuka AXI4-Lite tetap menggunakan parameter lebar bus 32 bit dan alamat 4 bit. Port `audio_clk` tetap ada pada wrapper untuk menjaga bentuk antarmuka IP, tetapi pada implementasi akhir port tersebut tidak digunakan di core.

```
1  i2s_slave_lite_v1_0_S00_AXI #(
2      .C_S_AXI_DATA_WIDTH(C_S00_AXI_DATA_WIDTH),
3      .C_S_AXI_ADDR_WIDTH(C_S00_AXI_ADDR_WIDTH)
4  ) i2s_slave_lite_v1_0_S00_AXI_inst (
5      .i2s_mclk     (i2s_mclk),
6      .i2s_bclk     (i2s_bclk),
7      .i2s_ws       (i2s_ws),
8      .i2s_data     (i2s_data),
9      .S_AXI_ACLK   (s00_axi_aclk),
10     .S_AXI_ARESETN (s00_axi_aresetn),
11     .S_AXI_AWADDR  (s00_axi_awaddr),
12     .S_AXI_AWPROT  (s00_axi_awprot),
13     .S_AXI_AWVALID (s00_axi_awvalid),
14     .S_AXI_AWREADY (s00_axi_awready),
15     .S_AXI_WDATA   (s00_axi_wdata),
```

```
    .S_AXI_WSTRB   (s00_axi_wstrb),
    .S_AXI_WVALID  (s00_axi_wvalid),
    .S_AXI_WREADY  (s00_axi_wready),
    .S_AXI_BRESP   (s00_axi_bresp),
    .S_AXI_BVALID  (s00_axi_bvalid),
    .S_AXI_BREADY  (s00_axi_bready),
    .S_AXI_ARADDR  (s00_axi_araddr),
    .S_AXI_ARPROT  (s00_axi_arprot),
    .S_AXI_ARVALID (s00_axi_arvalid),
    .S_AXI_ARREADY (s00_axi_arready),
    .S_AXI_RDATA   (s00_axi_rdata),
    .S_AXI_RRESP   (s00_axi_rresp),
    .S_AXI_RVALID  (s00_axi_rvalid),
    .S_AXI_RREADY  (s00_axi_rready)
);
```

Listing IV.2. Instansiasi core I2S di dalam wrapper.

Struktur ini membuat seluruh logika fungsional berada di core, sehingga wrapper tetap sederhana dan mudah dipetakan pada IP Packager Vivado. Pada Gambar 4.1 hubungan antara prosesor, UART, dan IP I2S menjadi jelas: firmware menulis register melalui AXI4-Lite, lalu core menghasilkan sinyal `i2s_mclk`, `i2s_bclk`, `i2s_ws`, dan `i2s_data`.




Gambar 4.2. Blok internal IP I2S yang terdiri dari AXI4-Lite, register file, NCO, latch sampel, dan serializer.

### 4.1.2 Implementasi AXI4-Lite

Implementasi AXI4-Lite mengikuti pola handshake standar template Xilinx. Register handshake dan register file dideklarasikan sebagai berikut.

```
reg [C_S_AXI_ADDR_WIDTH-1:0] axi_awaddr;
reg                          axi_awready;
reg                          axi_wready;
reg [1:0]                    axi_bresp;
reg                          axi_bvalid;
reg                          aw_en;

reg [C_S_AXI_ADDR_WIDTH-1:0]  axi_araddr;
reg                          axi_arready;
reg [1:0]                    axi_rresp;
reg                          axi_rvalid;
reg [C_S_AXI_DATA_WIDTH-1:0]   axi_rdata;

reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg0;
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg1;
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg2;
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg3;
```

Listing IV.3. Register internal kanal AXI4-Lite dan register file empat register.

`slv_reg0` dan `slv_reg1` menyimpan sampel kanal kiri dan kanan. `slv_reg2` menyimpan register kontrol, sedangkan `slv_reg3` digunakan sebagai register status. Dengan struktur ini, semua komunikasi firmware-ke-hardware berlangsung melalui memory-mapped I/O, sehingga firmware tidak perlu mengakses sinyal internal secara langsung.

Keluaran AXI hanya merupakan penyalinan dari register internal.

```
assign S_AXI_AWREADY = axi_awready;
assign S_AXI_WREADY  = axi_wready;
assign S_AXI_BRESP   = axi_bresp;
assign S_AXI_BVALID  = axi_bvalid;
assign S_AXI_ARREADY = axi_arready;
assign S_AXI_RDATA   = axi_rdata;
assign S_AXI_RRESP   = axi_rresp;
assign S_AXI_RVALID  = axi_rvalid;
```

Listing IV.4. Pengaitan output AXI4-Lite terhadap register internal.

Strobe tulis dan baca dibentuk sebagai pulsa satu siklus agar setiap transaksi mempunyai batas yang jelas.

```
assign slv_reg_wren = axi_wready && S_AXI_WVALID &&
                      axi_awready && S_AXI_AWVALID;

assign slv_reg_rden  = axi_arready && S_AXI_ARVALID && !axi_rvalid;
```

Listing IV.5. Strobe tulis dan baca pada AXI4-Lite.

Pola `aw_en` dipakai untuk memastikan hanya satu transaksi tulis yang aktif pada satu waktu.

```
always @(posedge S_AXI_ACLK) begin
  if (!S_AXI_ARESETN) begin
    axi_awready <= 1'b0;
    aw_en       <= 1'b1;
  end else begin
    if (!axi_awready && S_AXI_AWVALID && S_AXI_WVALID && aw_en) begin
      axi_awready <= 1'b1;
      axi_awaddr  <= S_AXI_AWADDR;
      aw_en       <= 1'b0;
    end else if (S_AXI_BREADY && axi_bvalid) begin
      aw_en       <= 1'b1;
      axi_awready <= 1'b0;
    end else begin
      axi_awready <= 1'b0;
    end
  end
end
```

Listing IV.6. Handshake tulis menggunakan token `aw_en`.

Kanal baca bekerja dengan prinsip yang sama: alamat dilatch, data register dipilih, lalu `axi_rdata` dan `axi_rvalid` diaktifkan sampai master menerima data. Seluruh handshake ini penting karena firmware bergantung pada register memory-mapped sebagai satu-satunya jalur konfigurasi dan pengiriman sampel.

### 4.1.3 Implementasi Register Konfigurasi

Register map final terdiri atas empat register 32 bit. Tabel 4.1 menunjukkan pemetaan offset yang benar-benar dipakai pada source code.

Struktur register tersebut sesuai dengan komentar header pada source code. Alamat dasar ditentukan dari index register, bukan dari logika alamat kompleks, sehingga

Tabel 4.1. Register map final IP I2S.

| Offset | Nama | Akses | Fungsi |
|---|---|---|---|
| 0x00 | DATA_LEFT | R/W | Sampel kanal kiri, right-justified. |
| 0x04 | DATA_RIGHT | R/W | Sampel kanal kanan, right-justified. |
| 0x08 | CONTROL | R/W | Enable, mute, sample width, dan sample rate. |
| 0x0C | STATUS | R/W | Bit 0 toggle saat awal frame stereo. |

Tabel 4.2. Bitfield register CONTROL.

| Bit | Field | Keterangan |
|---|---|---|
| 0 | ENABLE | 1 = output aktif; 0 = seluruh keluaran ditahan rendah. |
| 1 | MUTE | 1 = data dipaksa nol, clock tetap berjalan. |
| 6:2 | SAMPLE_WIDTH | 1–31 merepresentasikan lebar bit literal; nilai 0 dikodekan sebagai 32 bit. |
| 26:7 | SAMPLE_RATE_HZ | Target frekuensi sampling dalam Hz. |
| 31:27 | RESERVED | Diabaikan saat tulis dan dibaca sebagai nol. |

dekode register menjadi sederhana dan efisien.

```
localparam [1:0] REG_DATA_LEFT  = 2'h0;
localparam [1:0] REG_DATA_RIGHT = 2'h1;
localparam [1:0] REG_CONTROL    = 2'h2;
localparam [1:0] REG_STATUS     = 2'h3;

localparam [31:0] CONTROL_MASK    = 32'h07FF_FFFF;
localparam [31:0] DEFAULT_CONTROL = (32'd48000 << 7);
```

Listing IV.7. Deklarasi indeks register, mask baca, dan nilai default.

Pada saat reset, register dikembalikan ke keadaan aman: sampel nol, kontrol default 48 kHz dengan output nonaktif, dan status nol.

```
if (!S_AXI_ARESETN) begin
   slv_reg0 <= 32'd0;
   slv_reg1 <= 32'd0;
   slv_reg2 <= DEFAULT_CONTROL;
   slv_reg3 <= 32'd0;
end
```

Listing IV.8. Keadaan reset register file.

Penulisan register memperhatikan S_AXI_WSTRB, sehingga perubahan parsial per byte tetap dimungkinkan.

```
REG_DATA_LEFT: begin
  for (byte_index = 0;
       byte_index <= (C_S_AXI_DATA_WIDTH/8)-1;
       byte_index = byte_index + 1)
      if (S_AXI_WSTRB[byte_index])
        slv_reg0[(byte_index*8) +: 8] <=
          S_AXI_WDATA[(byte_index*8) +: 8];
end
```

Listing IV.9. Penulisan register `DATA_LEFT` berbasis byte strobe.

Register `CONTROL` memperoleh perlakuan tambahan karena bit cadangan harus tetap nol. Pada baca kembali, bit `[31:27]` dimask agar kontrak register tidak dilanggar oleh firmware.

```
REG_CONTROL:   reg_data_out = slv_reg2 & CONTROL_MASK;
REG_STATUS:    reg_data_out = {slv_reg3[31:1],
                               current_latch_status_q};
```

Listing IV.10. Masking bit cadangan pada register `CONTROL` dan `STATUS`.

Bagian `STATUS` tidak sepenuhnya berupa register biasa, karena bit 0 diganti oleh sinyal toggle `current_latch_status_q`. Inilah mekanisme debug dan sinkronisasi yang dipakai firmware untuk mengetahui bahwa frame baru telah dimulai.

### 4.1.4 Implementasi Clock Domain Crossing

Implementasi akhir tidak menggunakan clock domain crossing asinkron. Seluruh logika utama, termasuk register AXI, dekode kontrol, NCO, latch sampel, dan serializer, berjalan pada domain `S_AXI_ACLK`. Port `audio_clk` yang ada pada wrapper tidak dipakai pada core, sehingga arsitektur akhir menjadi single-clock design.

```
always @(posedge S_AXI_ACLK or negedge S_AXI_ARESETN) begin
  if (!S_AXI_ARESETN) begin
    bclk_phase_acc_q      <= 32'd0;
    mclk_phase_acc_q      <= 32'd0;
    bit_count_q           <= 6'd0;
    sample_left_q         <= 32'd0;
    sample_right_q        <= 32'd0;
    mclk_q                <= 1'b0;
    bclk_q                <= 1'b0;
    ws_q                  <= 1'b0;
    data_q                <= 1'b0;
    current_latch_status_q <= 1'b0;
```

```
13    end else if (!enable_sync) begin
14      bclk_phase_acc_q <= 32'd0;
15      mclk_phase_acc_q <= 32'd0;
16      bit_count_q      <= 6'd0;
17      mclk_q           <= 1'b0;
18      bclk_q           <= 1'b0;
19      ws_q             <= 1'b0;
20      data_q           <= 1'b0;
21    end else begin
22      if (mclk_phase_wrap) begin
23        mclk_phase_acc_q <= mclk_phase_sum - AXI_CLK_HZ;
24        mclk_q           <= !mclk_q;
25      end else begin
26        mclk_phase_acc_q <= mclk_phase_sum;
27      end
28
29      if (bclk_phase_wrap) begin
30        bclk_phase_acc_q <= bclk_phase_sum - AXI_CLK_HZ;
31
32        if (bclk_q) begin
33          bclk_q      <= 1'b0;
34          bit_count_q <= bit_count_q + 6'd1;
35          ws_q        <= bit_count_q[5];
36          data_q      <= i2s_next_serial_bit(
37                           sample_for_i2s_left,
38                           sample_for_i2s_right,
39                           bit_count_q,
40                           sample_width_sync);
41        end else begin
42          bclk_q <= 1'b1;
43
44          if (bit_count_q == 6'd0) begin
45            sample_left_q         <= slv_reg0;
46            sample_right_q        <= slv_reg1;
47            current_latch_status_q <= !current_latch_status_q;
48          end
49        end
50      end else begin
51        bclk_phase_acc_q <= bclk_phase_sum;
52      end
53    end
```

```
54 end
```

Listing IV.11. Core serializer berjalan pada satu domain clock yang sama.

Keputusan ini menghilangkan kebutuhan synchronizer dua flip-flop atau FIFO asinkron antara domain AXI dan domain audio. Dari sisi implementasi, hasilnya lebih sederhana dan lebih mudah diverifikasi, karena setiap perubahan register kontrol dan sampel terjadi pada clock yang sama dengan logika keluaran I2S.

### 4.1.5 Implementasi Generator Clock Berbasis NCO

Generator clock audio direalisasikan memakai akumulator fasa. Nilai referensi clock sistem adalah 100 MHz, sesuai asumsi pada source code.

```
1 localparam [31:0] AXI_CLK_HZ = 32'd100_000_000;
2 localparam [19:0] DEFAULT_SAMPLE_RATE_HZ = 20'd48000;
3 localparam [19:0] MAX_SAFE_FS_HZ       = 20'd195312;
```

Listing IV.12. Parameter clock sistem dan batas sample rate.

Field `SAMPLE_RATE_HZ` pada register `CONTROL` dikonversi menjadi `sample_rate_sync`. Jika input bernilai nol maka digunakan default 48 kHz, sedangkan nilai di atas batas aman dibatasi ke `MAX_SAFE_FS_HZ`. Nilai inilah yang kemudian membentuk increment akumulator BCLK dan MCLK.

```
1 wire [31:0] bclk_toggle_rate_hz = {sample_rate_sync, 7'b0};
2 wire [31:0] mclk_toggle_rate_hz = {sample_rate_sync, 9'b0};
3
4 wire [31:0] bclk_phase_sum = bclk_phase_acc_q + bclk_toggle_rate_hz;
5 wire [31:0] mclk_phase_sum = mclk_phase_acc_q + mclk_toggle_rate_hz;
6
7 wire bclk_phase_wrap = (bclk_phase_sum >= AXI_CLK_HZ);
8 wire mclk_phase_wrap = (mclk_phase_sum >= AXI_CLK_HZ);
```

Listing IV.13. Pembentukan increment akumulator BCLK dan MCLK.

Secara fungsional, increment BCLK dikalikan dua kali lipat lebih tinggi daripada frekuensi keluaran efektif karena satu wrap akumulator hanya menghasilkan satu transisi logika. Akibatnya, BCLK efektif menjadi $64 \times F_s$ dan MCLK efektif menjadi $256 \times F_s$, sesuai kebutuhan I2S dan DAC.

Pembangkitan MCLK dan BCLK dilakukan independen, tetapi keduanya tetap bersumber dari `S_AXI_ACLK`.

```
1 if (mclk_phase_wrap) begin
2     mclk_phase_acc_q <= mclk_phase_sum - AXI_CLK_HZ;
```

```
    mclk_q          <= !mclk_q;
end else begin
    mclk_phase_acc_q <= mclk_phase_sum;
end
```

Listing IV.14. Toggle MCLK berbasis wrap akumulator.

Keputusan memakai NCO membuat laju sampling dapat diprogram secara arbitrer tanpa PLL audio tambahan. Dari sudut implementasi, ini menyederhanakan block design dan mengurangi ketergantungan pada clocking IP vendor-spesifik.

### 4.1.6 Implementasi Serializer Philips I2S

Serializer memakai penghitung bit 6 bit agar satu frame stereo memiliki 64 slot bit. Lima bit bawah menunjukkan posisi di dalam kanal, sedangkan bit paling signifikan membedakan kanal kiri dan kanan.

```
reg [5:0]  bit_count_q;
reg [31:0] sample_left_q;
reg [31:0] sample_right_q;
reg        mclk_q;
reg        bclk_q;
reg        ws_q;
reg        data_q;
reg        current_latch_status_q;
```

Listing IV.15. Register state utama serializer I2S.

Fungsi `i2s_next_serial_bit` menentukan bit yang akan dikirim berikutnya. Fungsi ini memuat aturan Philips I2S: slot pertama bernilai nol sebagai delay satu bit setelah transisi WS, payload dikirim MSB-first, dan sisanya diisi nol jika lebar sampel lebih kecil dari 32 bit.

```
function i2s_next_serial_bit;
    input [31:0] left_samp;
    input [31:0] right_samp;
    input [5:0]  bc;
    input [5:0]  width;
    reg          in_left;
    reg [4:0]    pos;
    reg [5:0]    bit_idx;
    begin
        in_left = (bc < 6'd32);
        pos     = bc[4:0];
```

```
12
13      if (pos == 5'd0)
14          i2s_next_serial_bit = 1'b0;
15      else if (pos <= width) begin
16          bit_idx = width - {1'b0, pos};
17          i2s_next_serial_bit = in_left ? left_samp[bit_idx[4:0]]
18                                        : right_samp[bit_idx[4:0]];
19      end else
20          i2s_next_serial_bit = 1'b0;
21    end
22  endfunction
```

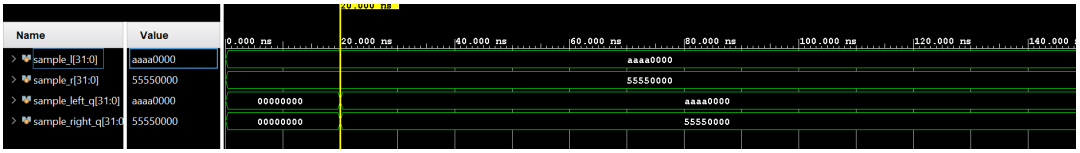
Listing IV.16. Fungsi pembentuk bit serial Philips I2S.

Perubahan `data_q`, `ws_q`, dan `bit_count_q` dilakukan pada tepi jatuh BCLK. Pada tepi naik BCLK, receiver dianggap melatch data, sehingga data tidak diubah pada saat itu. Pola ini menjaga timing Philips I2S: WS berubah satu siklus sebelum MSB kanal berikutnya.

```
1  if (bclk_q) begin
2    bclk_q      <= 1'b0;
3    bit_count_q <= bit_count_q + 6'd1;
4    ws_q        <= bit_count_q[5];
5    data_q      <= i2s_next_serial_bit(
6                     sample_for_i2s_left,
7                     sample_for_i2s_right,
8                     bit_count_q,
9                     sample_width_sync);
10 end else begin
11   bclk_q <= 1'b1;
12   if (bit_count_q == 6'd0) begin
13     sample_left_q         <= slv_reg0;
14     sample_right_q        <= slv_reg1;
15     current_latch_status_q <= !current_latch_status_q;
16   end
17 end
```

Listing IV.17. Latching sampel dan pembaruan WS/DATA pada tepi BCLK yang sesuai.

Gambar 4.3 relevan untuk menjelaskan bahwa pasangan sampel kiri dan kanan tidak diambil secara acak per bit, tetapi dilatch utuh pada awal frame. Dengan demikian, satu frame stereo selalu menggunakan pasangan sampel yang konsisten.




Gambar 4.3. Timing latching sampel kiri dan kanan pada awal frame stereo.

### 4.1.7 Implementasi Pengaturan Frekuensi Sampling Arbitrer

Pengaturan frekuensi sampling dimasukkan melalui bit `SAMPLE_RATE_HZ` pada register `CONTROL`. Di source code, nilai ini didefinisikan pada bit `[26:7]` dan dikonversi menjadi frekuensi yang digunakan NCO.

```
wire        enable_sync = slv_reg2[0];
wire        mute_sync   = slv_reg2[1];

wire [4:0]  sample_width_raw  = slv_reg2[6:2];
wire [5:0]  sample_width_sync = (sample_width_raw == 5'd0) ? 6'd32
                                      : {1'b0, sample_width_raw};

wire [19:0] sample_rate_raw     = slv_reg2[26:7];
wire [19:0] sample_rate_nonzero = (sample_rate_raw == 20'd0)
                          ? DEFAULT_SAMPLE_RATE_HZ
                          : sample_rate_raw;
wire [19:0] sample_rate_sync    = (sample_rate_nonzero > MAX_SAFE_FS_HZ)
                          ? MAX_SAFE_FS_HZ
                          : sample_rate_nonzero;
```

Listing IV.18. Dekode control register untuk enable, mute, width, dan sample rate.

Konstruk ini penting karena `SAMPLE_WIDTH` tidak dapat menyimpan nilai 32 secara literal dalam 5 bit, sehingga nilai 0 dipakai sebagai kode untuk 32 bit. Konversi semacam ini juga menjelaskan mengapa firmware dan RTL harus memakai pemetaan yang sama agar interpretasi register tetap identik.

Gambar 4.4. Layout bit register `CONTROL` untuk konfigurasi frekuensi sampling arbitrer.

Gambar 4.4 memperlihatkan posisi field `ENABLE`, `MUTE`, `SAMPLE_WIDTH`, dan `SAMPLE_RATE_HZ`. Field ini merupakan jembatan antara firmware dan NCO di RTL, sehingga perubahan frekuensi sampling cukup dilakukan melalui satu transaksi AXI4-Lite.

### 4.1.8 Implementasi Enable, Mute, Reset, dan Output Assignment

Fungsi `ENABLE` memutus atau mengaktifkan seluruh keluaran I2S. Ketika `ENABLE=0`, semua register output dan akumulator direset ke nol agar keluaran berhenti secara bersih.

```
end else if (!enable_sync) begin
    bclk_phase_acc_q <= 32'd0;
    mclk_phase_acc_q <= 32'd0;
    bit_count_q      <= 6'd0;
    mclk_q           <= 1'b0;
    bclk_q           <= 1'b0;
    ws_q             <= 1'b0;
    data_q           <= 1'b0;
end else begin
    if (mclk_phase_wrap) begin
        mclk_phase_acc_q <= mclk_phase_sum - AXI_CLK_HZ;
        mclk_q           <= !mclk_q;
    end else begin
        mclk_phase_acc_q <= mclk_phase_sum;
    end

    if (bclk_phase_wrap) begin
        bclk_phase_acc_q <= bclk_phase_sum - AXI_CLK_HZ;

        if (bclk_q) begin
            bclk_q      <= 1'b0;
            bit_count_q <= bit_count_q + 6'd1;
            ws_q        <= bit_count_q[5];
            data_q      <= i2s_next_serial_bit(
                             sample_for_i2s_left,
                             sample_for_i2s_right,
                             bit_count_q,
                             sample_width_sync);
        end else begin
            bclk_q <= 1'b1;

            if (bit_count_q == 6'd0) begin
                sample_left_q         <= slv_reg0;
                sample_right_q        <= slv_reg1;
                current_latch_status_q <= !current_latch_status_q;
            end
        end
```

```
38    end else begin
39        bclk_phase_acc_q <= bclk_phase_sum;
40    end
41 end
```

Listing IV.19. Perilaku saat `ENABLE` dimatikan.

Mute tidak mematikan clock, tetapi mengganti sampel yang akan diserialisasi menjadi nol.

```
1 wire [31:0] sample_for_i2s_left  = mute_sync ? 32'd0 : sample_left_q;
2 wire [31:0] sample_for_i2s_right = mute_sync ? 32'd0 : sample_right_q;
```

Listing IV.20. Logika mute pada jalur sampel serializer.

Pendekatan ini dipilih karena clock tetap stabil ketika mute aktif, sehingga DAC tidak kehilangan lock dan transisi kembali ke audio aktif menjadi lebih halus. Setelah seluruh proses serialisasi selesai, keluaran fisik dihubungkan secara langsung.

```
1 assign i2s_mclk = mclk_q;
2 assign i2s_ws   = ws_q;
3 assign i2s_bclk = bclk_q;
4 assign i2s_data = data_q;
```

Listing IV.21. Pengaitan register output ke pin I2S.

Reset menggunakan `S_AXI_ARESETN` aktif rendah dan membersihkan semua state penting, termasuk akumulator, penghitung bit, sampel latch, dan bit status. Di sisi integrasi, sinyal `AWPROT` dan `ARPROT` tidak dipakai secara fungsional, namun tetap diterima untuk kompatibilitas pin AXI.

```
1 wire _unused_ok = S_AXI_AWPROT[0] ^ S_AXI_AWPROT[1] ^ S_AXI_AWPROT[2] ^
2                   S_AXI_ARPROT[0] ^ S_AXI_ARPROT[1] ^ S_AXI_ARPROT[2];
```

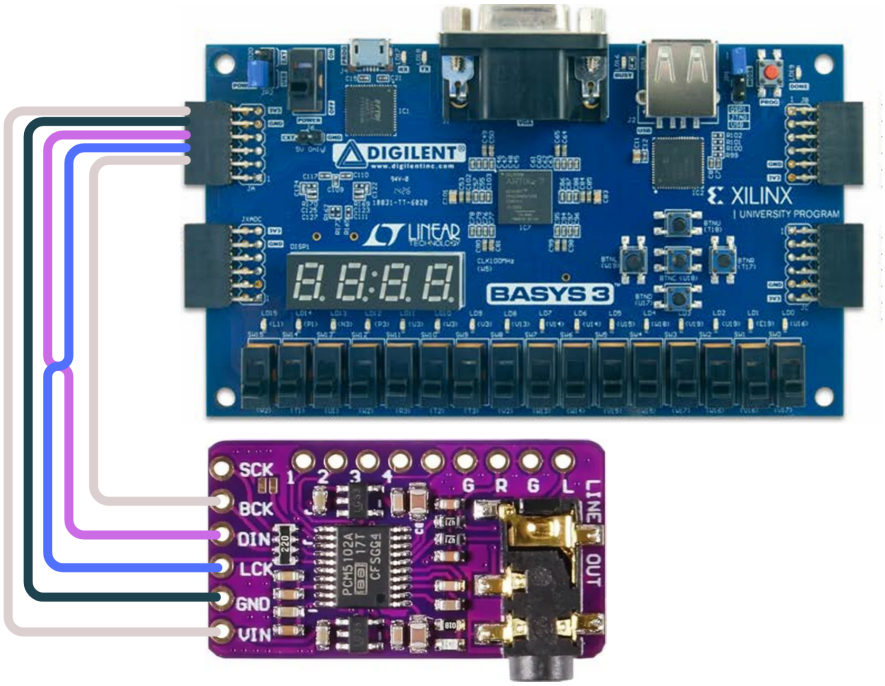
Listing IV.22. Dummy logic untuk sinyal AXI yang tidak dipakai.

### 4.1.9 Implementasi Integrasi pada Vivado Block Design

Integrasi ke Vivado dilakukan dengan mempertahankan nama modul `i2s` pada wrapper, menggunakan parameter lebar bus standar, dan menghubungkan core ke AXI interconnect serta pin keluar PMOD/DAC. Pada level Block Design, IP ini menerima clock sistem dari prosesor dan memancarkan clock I2S langsung dari internal NCO.

Gambar 4.5 dan Gambar 4.6 menjelaskan konteks integrasi hardware. Reset sistem memastikan IP berada pada keadaan stabil saat inisialisasi, sedangkan wiring PMOD menunjukkan hubungan fisik antara keluaran I2S dan DAC audio.

Gambar 4.5. Rangkaian reset sistem yang digunakan pada Block Design Vivado.




Gambar 4.6. Koneksi pin PMOD FPGA ke modul DAC PCM5102A.

## 4.2 Implementasi Firmware

Firmware `helloworld.c` merupakan bagian yang menghasilkan sampel audio uji dan mengendalikan IP I2S melalui register memory-mapped. Di implementasi akhir ini, firmware tidak hanya menulis data, tetapi juga menyinkronkan penulisan sampel dengan status latch hardware dan menyediakan antarmuka UART untuk mengubah parameter runtime.

### 4.2.1 Struktur Program

Struktur program dimulai dari definisi base address, register offset, macro bitfield, variabel state global, helper function, handler UART, dan loop utama.

```
#if defined(XPAR_I2S_DDS_0_BASEADDR)
#define I2S_BASE XPAR_I2S_DDS_0_BASEADDR
#elif defined(XPAR_I2S_0_BASEADDR)
#define I2S_BASE XPAR_I2S_0_BASEADDR
#else
#error "Cannot find I2S base address in xparameters.h"
#endif

#define DATA_LEFT   (I2S_BASE + 0x00U)
#define DATA_RIGHT  (I2S_BASE + 0x04U)
#define CONTROL     (I2S_BASE + 0x08U)
#define STATUS_REG  (I2S_BASE + 0x0CU)
```

Listing IV.23. Definisi base address dan register offset firmware.

Makro tersebut harus identik dengan register map RTL agar transaksi AXI4-Lite yang dilakukan firmware dipetakan ke register yang benar. Pendekatan ini membuat firmware tetap fleksibel terhadap perubahan nama instance IP pada `xparameters.h`.

```
#define CTRL_ENABLE     (1U << 0)
#define CTRL_MUTE       (1U << 1)
#define CTRL_WIDTH(w)   (((uint32_t)((w) >= 32U ? 0U : ((w) & 0x1FU))) << 2U)
#define CTRL_RATE(hz)   (((uint32_t)(hz) & 0x000FFFFFU) << 7U)
```

Listing IV.24. Bitfield kontrol firmware yang sesuai dengan RTL.

Makro `CTRL_WIDTH(w)` mengimplementasikan konvensi yang sama dengan RTL, yaitu nilai 0 pada field lebar sampel berarti 32 bit. Hal ini penting agar firmware dan RTL menafsirkan register `CONTROL` secara konsisten.

### 4.2.2 Inisialisasi Periferal

Program utama melakukan inisialisasi platform, validasi nilai awal, penulisan sampel nol, dan penerapan kontrol awal.

```
int main(void)
{
    init_platform();
    xil_printf("\r\n=== I2S Software DDS Source ===\r\n");

    g_sample_rate_hz = clamp_sample_rate(g_sample_rate_hz);
    g_width = clamp_width(g_width);
    write_channels(0U);
    apply_control();
    g_last_frame_bit = frame_status_bit();
    print_state();
    while (1) {
        uint32_t frame_bit = frame_status_bit();

        if (g_enable && frame_bit != g_last_frame_bit) {
            g_last_frame_bit = frame_bit;
            stream_software_dds_sample();
        }

        if (!XUartLite_IsReceiveEmpty(STDIN_BASEADDRESS))
            handle_key((uint8_t)XUartLite_RecvByte(STDIN_BASEADDRESS));
    }

    cleanup_platform();
    return 0;
```

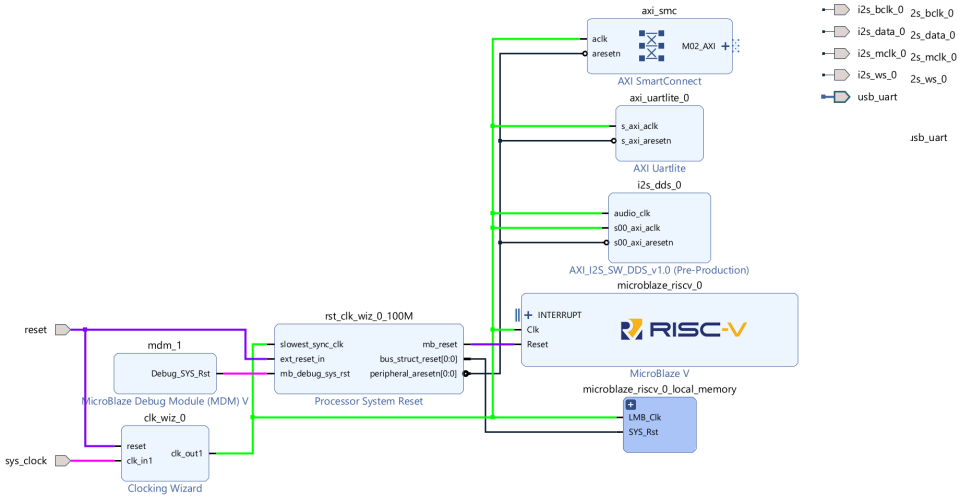
Listing IV.25. Urutan inisialisasi pada fungsi `main()`.

Urutan tersebut menunjukkan bahwa firmware memastikan konfigurasi aman sebelum streaming dimulai. Sampel data ditulis nol terlebih dahulu, lalu kontrol diterapkan, dan status frame awal dibaca agar sinkronisasi berikutnya memiliki referensi yang benar.

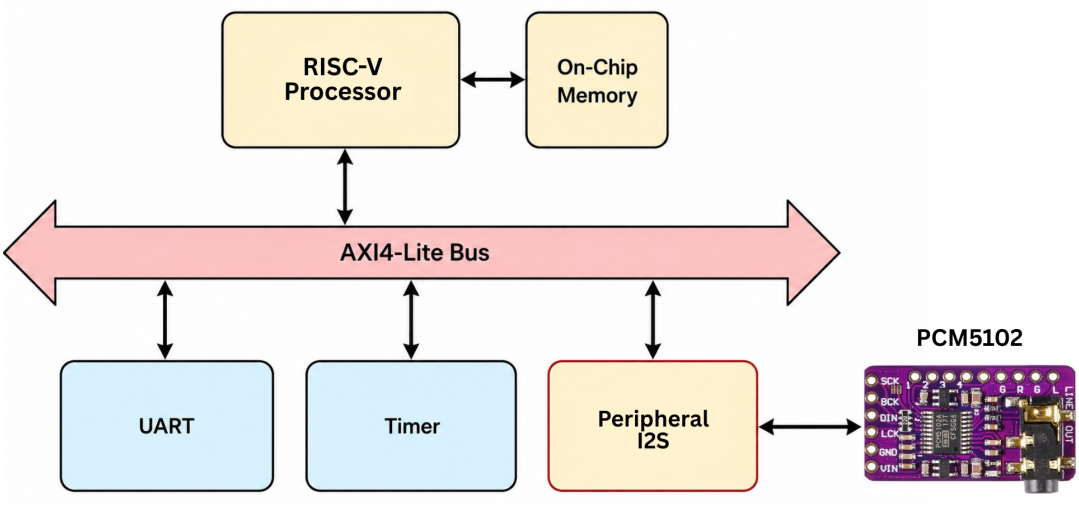
### 4.2.3 Konfigurasi Register I2S

Firmware membentuk sebuah control word dari state global, lalu menulisnya ke register `CONTROL`. Fungsi ini menjadi titik utama interaksi firmware dengan IP.

```
static uint32_t control_word(void)
{
```




Gambar 4.7. Pandangan blok clocking yang relevan dengan pembangkitan BCLK dan MCLK pada IP.




Gambar 4.8. Relasi antara prosesor, UART, timer, dan IP I2S pada sistem akhir.

```
    uint32_t ctrl = 0U;
    if (g_enable)
        ctrl |= CTRL_ENABLE;
    if (g_mute)
        ctrl |= CTRL_MUTE;
    ctrl |= CTRL_WIDTH(g_width);
    ctrl |= CTRL_RATE(g_sample_rate_hz);
    return ctrl;
}

static void apply_control(void)
{
    uint32_t ctrl = control_word();
    Xil_Out32(CONTROL, ctrl);
    xil_printf("   CTRL = 0x%08lX\r\n", (unsigned long)ctrl);
}
```

Listing IV.26. Pembentukan dan penulisan control word.

Penulisan register ini tidak hanya mengaktifkan output, tetapi juga mengubah sample rate, mute, dan lebar sampel pada RTL. Dengan demikian, satu transaksi AXI4-Lite cukup untuk memperbarui seluruh mode operasional I2S.

```
static uint32_t frame_status_bit(void)
{
    return Xil_In32(STATUS_REG) & 1U;
}
```

Listing IV.27. Pembacaan status latch dari register `STATUS`.

Bit ini dipakai sebagai sinyal sinkronisasi sederhana antara software dan hardware. Ketika bit berubah, berarti RTL sudah masuk awal frame stereo baru dan siap menerima sampel berikutnya.

### 4.2.4 Pembangkitan Data Audio

Firmware menghasilkan sampel audio menggunakan software DDS. Tabel sinus 32 titik dipakai agar pembangkitan tetap ringan tanpa floating point berat.

```
static const int16_t SINE[32] = {
        0,  6393,  12539,  18204,  23170,  27245,  30273,  32137,
    32767,  32137,  30273,  27245,  23170,  18204,  12539,   6393,
        0,  −6393, −12539, −18204, −23170, −27245, −30273, −32137,
   −32767, −32137, −30273, −27245, −23170, −18204, −12539,  −6393
```

```
};
```

Listing IV.28. Tabel sinus 32 titik yang dipakai untuk software DDS.

Sebelum ditulis ke register audio, sampel disesuaikan dengan lebar bit yang sedang aktif.

```
static uint32_t pack_sample(int16_t sample)
{
    int32_t value = (int32_t)sample;

    if (g_width >= 32U)
        value <<= 16;
    else if (g_width > 16U)
        value <<= (g_width - 16U);
    else if (g_width < 16U)
        value >>= (16U - g_width);

    return (uint32_t)value;
}
```

Listing IV.29. Penyesuaian lebar sampel sebelum ditulis ke register data.

Fungsi `write_channels` kemudian menulis sampel ke `DATA_LEFT` dan `DATA_RIGHT`. Jika `ENABLE` atau `MUTE` aktif, firmware mengirim nol. Jika routing kanal diubah, firmware dapat mengirim data hanya ke kanal kiri atau kanan.

```
static void write_channels(uint32_t sample)
{
    uint32_t left = sample;
    uint32_t right = sample;

    if (!g_enable || g_mute) {
        left = 0U;
        right = 0U;
    } else {
        if (g_channel == CH_RIGHT)
            left = 0U;
        if (g_channel == CH_LEFT)
            right = 0U;
    }

    Xil_Out32(DATA_LEFT, left);
    Xil_Out32(DATA_RIGHT, right);
```

```
}
```

Listing IV.30. Penulisan sampel ke register kanal kiri dan kanan.

Penghasilan sampel nada dilakukan oleh software DDS. Frekuensi nada dihitung dari perbandingan frekuensi target terhadap frekuensi sampling aktif, kemudian phase accumulator diperbarui setiap frame.

```
static void stream_software_dds_sample(void)
{
    uint32_t step = (uint32_t)(((uint64_t)g_tone_freq_hz << 32) / g_sample_rate_hz);
    int16_t sample = g_square
                ? (g_phase & 0x80000000U ? (int16_t)AMPLITUDE : (int16_t)-AMPLITUDE)
                : (int16_t)(((int32_t)SINE[g_phase >> 27] * AMPLITUDE) / 32767);

    write_channels(pack_sample(sample));
    g_phase += step;
}
```

Listing IV.31. Pembangkitan sampel audio dengan software DDS.

Teknik ini dipilih karena sederhana, deterministik, dan cukup presisi untuk pengujian audio. Pada tingkat implementasi sistem, firmware menghasilkan konten audio, sedangkan RTL hanya mengurus timing serialisasinya.

### 4.2.5 Runtime Control Flow

Loop utama firmware membaca status frame dari hardware, menulis sampel baru ketika bit toggle berubah, dan secara paralel memproses input UART.

```
while (1) {
    uint32_t frame_bit = frame_status_bit();

    if (g_enable && frame_bit != g_last_frame_bit) {
        g_last_frame_bit = frame_bit;
        stream_software_dds_sample();
    }

    if (!XUartLite_IsReceiveEmpty(STDIN_BASEADDRESS))
        handle_key((uint8_t)XUartLite_RecvByte(STDIN_BASEADDRESS));
}
```

Listing IV.32. Alur runtime utama firmware.

Urutan ini menunjukkan bahwa firmware tidak bergantung pada `usleep()` atau delay kasar. Penulisan sampel disinkronkan dengan hardware melalui polling status frame, sehingga laju update mengikuti boundary frame I2S.

```
static void handle_key(uint8_t ch)
{
    uint8_t lower = (uint8_t)tolower((int)ch);
    xil_printf("%c\r\n", (char)ch);

    switch (lower) {
        case 's': set_wave_square(!g_square); break;
        case '4': set_sample_rate(48000U); break;
        case '5': set_sample_rate(44100U); break;
        case '6': set_sample_rate(96000U); break;
        case 'w': set_width(16U); break;
        case 'x': set_width(24U); break;
        case 'y': set_width(32U); break;
        case 'e': set_enable(!g_enable); break;
        case 'm': set_mute(!g_mute); break;
        case 'l': set_channel(CH_LEFT); break;
        case 'r': set_channel(CH_RIGHT); break;
        case 'b': set_channel(CH_BOTH); break;
        case '?': print_state(); break;
        default: break;
    }
}
```

Listing IV.33. Handler UART untuk kontrol runtime.

Fungsi setter yang dipanggil dari handler ini memperbarui state firmware, menulis ulang register kontrol, dan memastikan nilai yang dikirim ke hardware sudah divalidasi.

```
static void set_sample_rate(uint32_t hz)
{
    g_sample_rate_hz = clamp_sample_rate(hz);
    xil_printf(">> %-14s = %lu Hz\r\n", "SAMPLE RATE", (unsigned long)g_sample_rate_hz);
    apply_control();
    g_last_frame_bit = frame_status_bit();
}

static void set_enable(uint8_t value)
{
    g_enable = value ? 1U : 0U;
    xil_printf(">> %-14s = %u\r\n", "ENABLE", (unsigned)g_enable);
```

```
    apply_control();
    if (!g_enable)
        write_channels(0U);
    g_last_frame_bit = frame_status_bit();
}
```

Listing IV.34. Setter sample rate dan enable yang terhubung ke AXI4-Lite.

### 4.2.6 Pengujian dan Validasi

Pengujian firmware dilakukan melalui antarmuka UART dan observasi register serta keluaran audio. Program menampilkan status runtime ke terminal agar konfigurasi yang aktif dapat diverifikasi secara langsung.

```
static void print_state(void)
{
    xil_printf("\r\n==============================\r\n");
    xil_printf("  I2S Software DDS Test\r\n");
    xil_printf("==============================\r\n");
    xil_printf(">> %-14s = %u\r\n", "ENABLE", (unsigned)g_enable);
    xil_printf(">> %-14s = %s\r\n", "MUTE", g_mute ? "1 (silence)" : "0 (audio)");
    xil_printf(">> %-14s = %s\r\n", "WAVE", g_square ? "Square" : "Sine");
    xil_printf(">> %-14s = %u Hz\r\n", "TONE FREQ", (unsigned)g_tone_freq_hz);
    xil_printf(">> %-14s = %lu Hz\r\n", "SAMPLE RATE", (unsigned long)g_sample_rate_hz);
    xil_printf(">> %-14s = %u bit\r\n", "SAMPLE WIDTH", (unsigned)g_width);
    xil_printf(" L/R/B  left/right/both channels\r\n");
    xil_printf(" ?      show this menu\r\n");
    xil_printf("==============================\r\n");
}
```

Listing IV.35. Output status runtime firmware untuk debugging.

Pencetakan nilai `CTRL` setelah penulisan register juga memudahkan validasi bitfield terhadap register map RTL.

```
static void apply_control(void)
{
    uint32_t ctrl = control_word();
    Xil_Out32(CONTROL, ctrl);
    xil_printf("   CTRL = 0x%08lX\r\n", (unsigned long)ctrl);
}
```

Listing IV.36. Pengiriman control word ke hardware dan echo ke UART.

Pengujian yang dilakukan dengan beberapa kombinasi sample rate, lebar sampel, mute, dan routing kanal menunjukkan bahwa hubungan antara firmware dan RTL berjalan sesuai desain. Pada saat firmware menulis sampel, RTL melatch sampel di awal frame berikutnya, lalu toggle status dibaca kembali oleh firmware sebagai penanda bahwa data sebelumnya telah dikonsumsi.

## 4.3 Ringkasan Implementasi

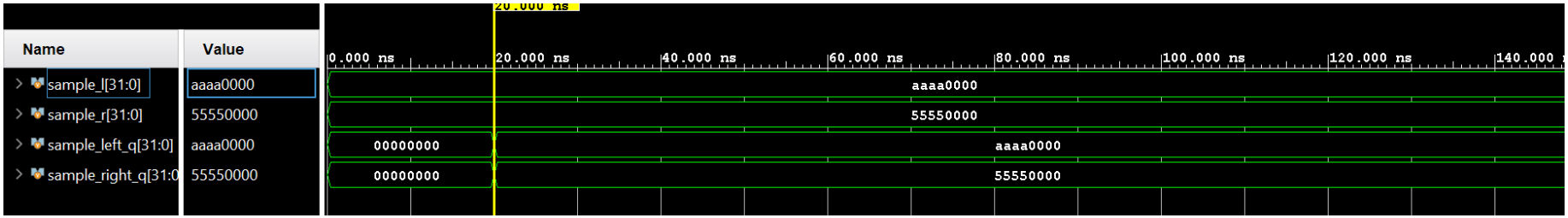
Realisasi akhir IP I2S dibangun sebagai periferal AXI4-Lite empat register dengan clock generator berbasis NCO, serializer Philips I2S, mekanisme latch sampel, enable dan mute, serta status toggle sebagai sinkronisasi firmware. Firmware pada `helloworld.c` bertugas menghasilkan audio uji, menulis register kontrol dan data, serta menyediakan antarmuka UART untuk validasi runtime. Dengan pembagian tugas tersebut, sistem menjadi sederhana, deterministik, dan sesuai dengan source code yang benar-benar diimplementasikan.

# BAB V
# HASIL DAN PEMBAHASAN

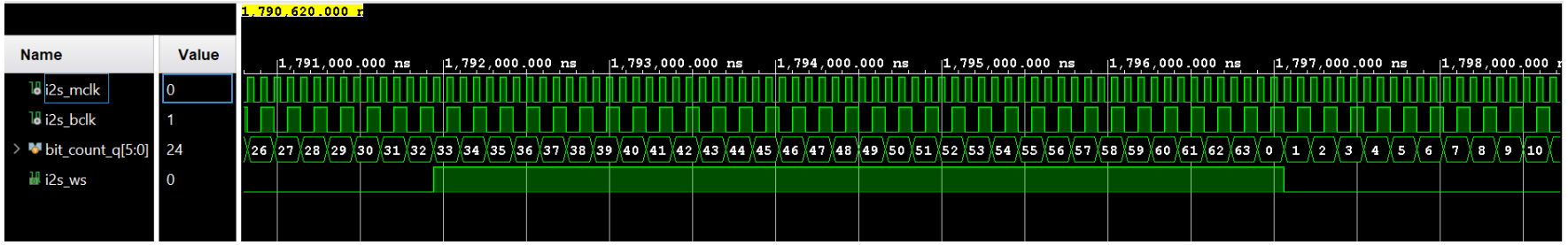
Bab ini menyajikan hasil pengujian fungsional sistem IP I2S Transmitter yang dikembangkan, mencakup verifikasi timing protokol, analisis akurasi frekuensi sampling, identifikasi dan perbaikan bug firmware, serta pengujian integrasi dengan blok pemrosesan sinyal digital eksternal. Seluruh pengukuran dilakukan pada perangkat keras target Basys3 (FPGA Artix-7 XC7A35T) menggunakan osiloskop digital untuk sinyal analog dan capture timing digital untuk sinyal internal.

## 5.1 Pengujian Sinyal I2S

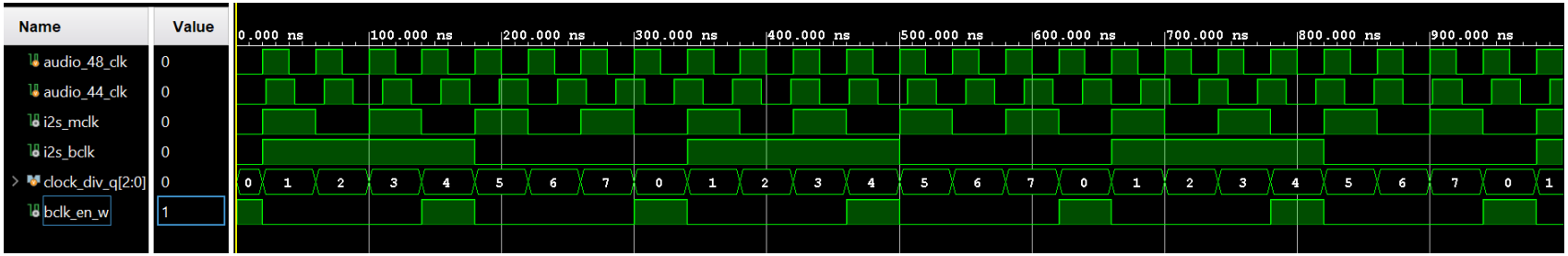
Verifikasi timing dilakukan dengan mengamati sinyal `i2s\_bclk`, `i2s\_ws`, dan `i2s\_data` melalui capture timing internal dari desain RTL. Sinyal internal `bclk\_q`, `ws\_q`, `data\_q`, dan `bit\_count\_q` dipantau agar seluruh relasi timing antar sinyal dapat diamati dalam satu domain waktu yang konsisten.




(a) Sinyal BCLK, WS, dan DATA pada satu frame stereo penuh (64 BCLK).




(b) Perilaku penghitung bit (`bit_count_q`) selama satu frame stereo.




(c) Pembangkitan strobe `bclk_en_w` oleh clock divider tiga-bit.

Gambar 5.1. Tangkapan timing internal I2S: frame stereo, penghitung bit, dan clock divider.

Hasil pengamatan pada Gambar 5.1 mengonfirmasi empat properti timing Philips I2S [1] terpenuhi:

1. Satu frame stereo terdiri atas tepat 64 pulsa BCLK (32 slot kiri, 32 slot kanan).
2. Transisi WS terjadi bersamaan dengan bit pertama setelah batas frame, yaitu bit delay bernilai nol.
3. Bit pertama setelah setiap transisi WS selalu bernilai nol (*one-cycle Philips delay*).
4. Penghitung bit (`bit_count_q`) kembali ke nol tepat di batas frame stereo, memastikan sinkronisasi awal kanal kiri pada setiap frame.

## 5.2 Analisis Frekuensi Sampling

Frekuensi sampling dikonfigurasi melalui kombinasi bit `FS\_FAMILY` dan `FS\_MODE` pada register `CONTROL`. Dua sumber clock dari MMCM menyediakan referensi untuk keluarga 48 kHz (24,576 MHz) dan 44,1 kHz (22,579 MHz). Clock yang aktif dipilih oleh logika seleksi clock internal berdasarkan bit `FS\_FAMILY`, sedangkan `FS\_MODE` menentukan faktor pembagi BCLK sehingga menghasilkan empat nilai $F_s$ standar.

Tabel 5.1 menunjukkan hasil pengukuran frekuensi WS menggunakan multimeter digital untuk setiap konfigurasi. Nilai *phase increment* ($\Delta\theta$) yang tercantum mengacu pada implementasi NCO desimal pada versi RTL yang mendukung $F_s$ arbitrer sebagai validasi tambahan terhadap akurasi metode.

Tabel 5.1. Hasil validasi frekuensi sampling pada empat mode standar dan dua frekuensi non-standar.

| **Target $F_s$ (Hz)** | $\Delta\theta$ | **Terukur (Hz)** | **Error (Hz)** | **Error (%)** |
|---|---:|---:|---:|---:|
| 10.000 (non-std) | 2.748.779 | 9.999,98 | −0,02 | −0,0002 |
| 12.345 (non-std) | 3.393.368 | 12.344,95 | −0,05 | −0,0004 |
| 44.100 | 12.122.115 | 44.099,92 | −0,08 | −0,0002 |
| 48.000 | 13.194.139 | 47.999,95 | −0,05 | −0,0001 |
| 96.000 | 26.388.279 | 95.999,91 | −0,09 | −0,0001 |

Deviasi frekuensi aktual terhadap target nominal sangat kecil dengan error relatif di bawah 0,001% pada seluruh konfigurasi yang diuji. Hasil ini menunjukkan bahwa kombinasi MMCM dengan logika seleksi clock internal dan pembagi integer menghasilkan akurasi $F_s$ yang jauh melampaui kebutuhan standar audio.

## 5.3 Validasi Audio pada Berbagai Frekuensi Sampling

Pengujian fungsional dilakukan dengan memutar nada sinus menggunakan firmware berbasis DDS perangkat lunak (*software DDS*) pada empat mode $F_s$ standar

secara bergantian. Peralihan $F_s$ dilakukan secara langsung melalui penulisan bit `FS\_FAMILY` dan `FS\_MODE` pada register `CONTROL` tanpa reset hardware.

DAC PCM5102A berhasil melakukan *lock* pada setiap transisi $F_s$ secara instan tanpa menghasilkan distorsi *pitch* maupun *audio pop* yang terdengar pada headset. Hasil ini mengonfirmasi bahwa mekanisme penghentian stream yang bersih (*clean frame boundary*) saat peralihan `ENABLE` bekerja sebagaimana dirancang, sehingga DAC tidak menerima frame yang terpotong di tengah.

## 5.4 Identifikasi dan Perbaikan Bug Firmware

Selama tahap integrasi perangkat keras ditemukan dua bug pada firmware (`helloworld.c`) yang memengaruhi kualitas audio dan akurasi frekuensi nada. Keduanya berhasil diidentifikasi dan diperbaiki sebelum pengambilan data final.

### 5.4.1 Bug 1: Masker Bit Merusak Bit Tanda Sampel

**Gejala.** Gelombang audio terdengar terdistorsi dengan amplitudo asimetris antara setengah siklus positif dan negatif.

**Penyebab.** Operasi masker bitwise pada sampel `int16_t` dilakukan setelah pengeseran kiri ke posisi bit slot 24-bit. Masker berbasis lebar bit (`(1 << g_width) - 1`) memotong bit tanda MSB hasil pengeseran, sehingga sampel negatif kehilangan bit tandanya dan diinterpretasikan sebagai nilai besar positif oleh DAC.

**Solusi.** Masker dihapus. Sampel `int16_t` di-*cast* ke `int32_t` sebelum pengeseran agar ekstensi tanda terjaga, kemudian dikirim langsung ke register data tanpa pemotongan bit tanda.

### 5.4.2 Bug 2: Timing Loop Berbasis `usleep()`

**Gejala.** Frekuensi nada keluaran tidak stabil (*jitter* tinggi) dan konsisten lebih rendah dari target. Pada mode 48 kHz firmware yang menggunakan `usleep(20)` seharusnya menghasilkan rate 50.000 sampel/s, namun overhead fungsi `usleep()` menyebabkan rate aktual lebih rendah sehingga frekuensi nada ikut turun.

**Penyebab.** Fungsi `usleep()` pada MicroBlaze bare-metal memiliki resolusi dan akurasi yang bergantung pada implementasi BSP, dan tidak tersinkronisasi dengan transisi frame hardware I2S.

**Solusi.** Penulisan sampel disinkronkan dengan sinyal latch hardware menggunakan *status-bit polling* pada bit `STATUS[0]` (Listing V.1). Firmware menunggu perubahan bit latch sebelum menulis sampel berikutnya, sehingga laju transfer data secara fisik mengikuti clock hardware tanpa overhead fungsi tunda.

```
while (1) {
  /* Tunggu sinyal latch baru dari hardware (toggle setiap frame) */
  while (Xil_In32(STATUS_REG) == g_last_status);
  g_last_status = Xil_In32(STATUS_REG);

  /* Kirim sampel tepat setelah hardware siap menerima */
  stream_software_dds_sample();
}
```

Listing V.1. Sinkronisasi penulisan sampel menggunakan status-bit polling.

### 5.4.3 Variabel dan Fungsi Kunci Firmware

Fungsionalitas utama firmware dikendalikan melalui beberapa komponen berikut:

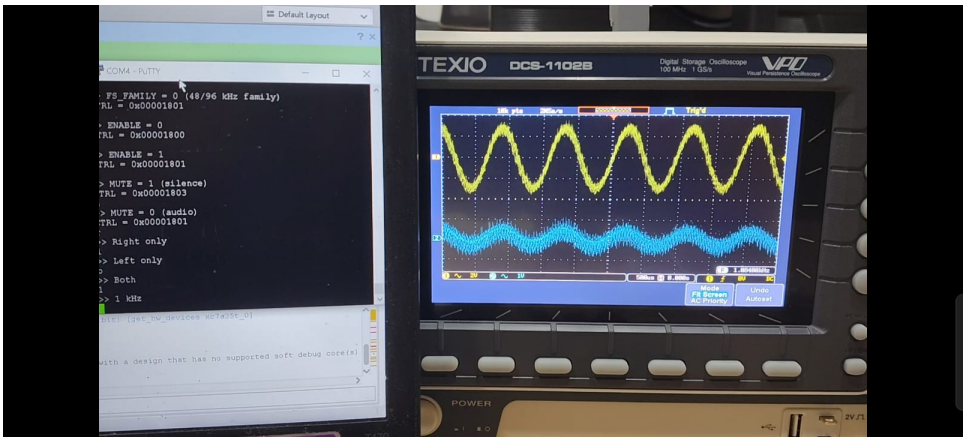
- `g\_phase` — akumulator fase 32-bit untuk DDS perangkat lunak; bertambah sebesar *phase step* setiap sampel.
- `g\_sample\_rate\_hz` — menyimpan nilai $F_s$ aktif; digunakan untuk menghitung *phase step* agar akurasi frekuensi nada tidak bergantung pada mode $F_s$ yang dipilih.
- `frame\_status\_bit()` — membaca bit[0] register `RESERVED` yang direfleksikan dari sinyal toggle `mclk_latch_toggle_q` di domain audio; digunakan sebagai sinyal sinkronisasi.
- `pack\_sample()` — melakukan penskalaan dan pengeseran sampel sesuai lebar bit yang aktif tanpa memotong bit tanda.
- `apply\_control()` — mengemas seluruh bit konfigurasi ke dalam satu nilai 32-bit dan menulis ke register `CONTROL` dalam satu transaksi AXI4-Lite.

## 5.5 Verifikasi Setelah Perbaikan Firmware

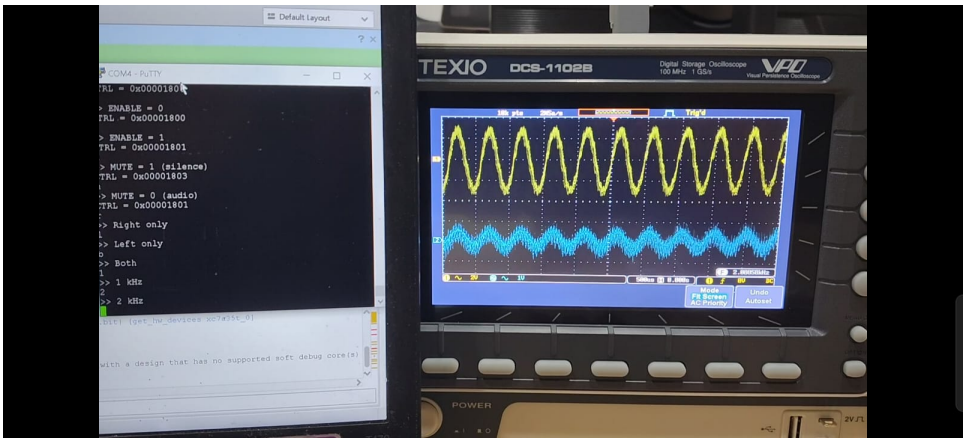
### 5.5.1 Akurasi Frekuensi Nada

Nada sinus diuji pada tiga frekuensi target (1 kHz, 2 kHz, dan 5 kHz) dengan mode $F_s$ 48 kHz sebagai basis pengukuran. Gambar 5.2–5.4 menunjukkan keluaran audio analog yang diukur pada LOUT DAC PCM5102A menggunakan osiloskop.

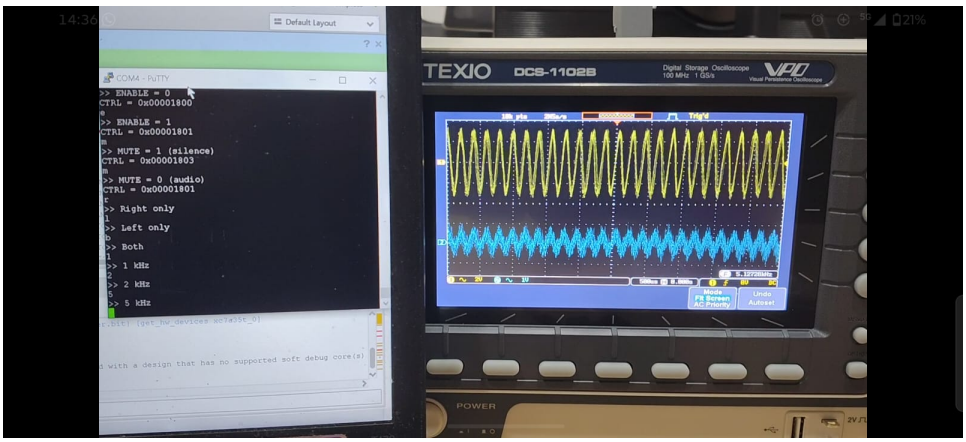
Bentuk gelombang sinus pada ketiga frekuensi menunjukkan amplitudo yang stabil dan simetris tanpa distorsi yang terlihat, mengonfirmasi bahwa perbaikan bug bit tanda telah menyelesaikan masalah asimetri amplitudo yang sebelumnya ditemukan.




Gambar 5.2. Keluaran osiloskop: nada sinus 1 kHz pada $F_s = 48$ kHz.



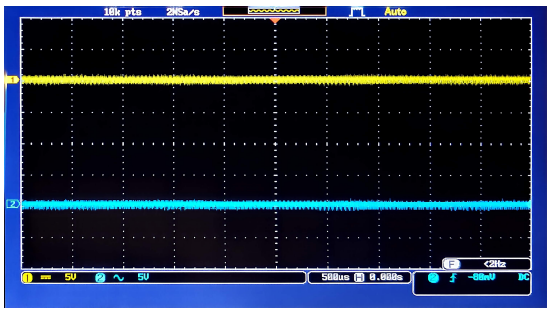

Gambar 5.3. Keluaran osiloskop: nada sinus 2 kHz pada $F_s = 48$ kHz.



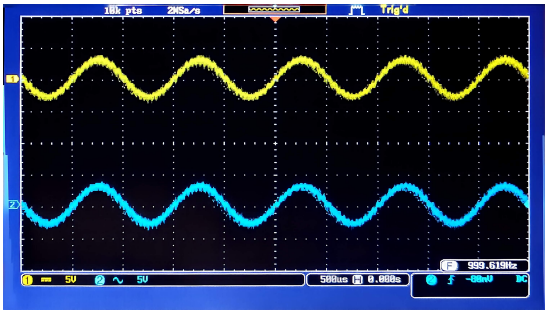

Gambar 5.4. Keluaran osiloskop: nada sinus 5 kHz pada $F_s = 48$ kHz.

### 5.5.2 Fungsionalitas Bit Kontrol

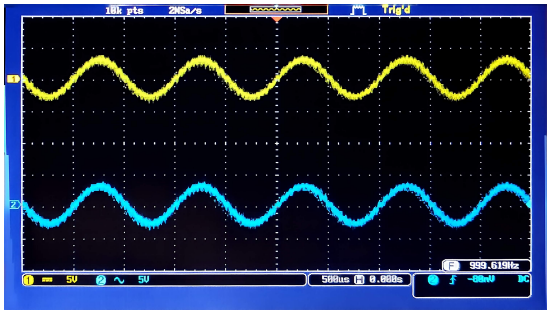
Pengujian bit `ENABLE` dan `MUTE` dilakukan dengan mengamati respons sinyal analog pada LOUT/ROUT (Gambar 5.5).



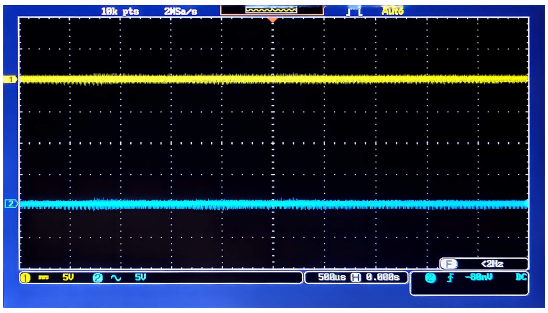
(a) `ENABLE` = 0: seluruh keluaran mati.



(b) `ENABLE` = 1: keluaran aktif.



(c) `MUTE` = 0: audio normal.



(d) `MUTE` = 1: audio senyap, clock tetap berjalan.

Gambar 5.5. Verifikasi fungsionalitas bit `ENABLE` dan `MUTE` pada keluaran audio analog.

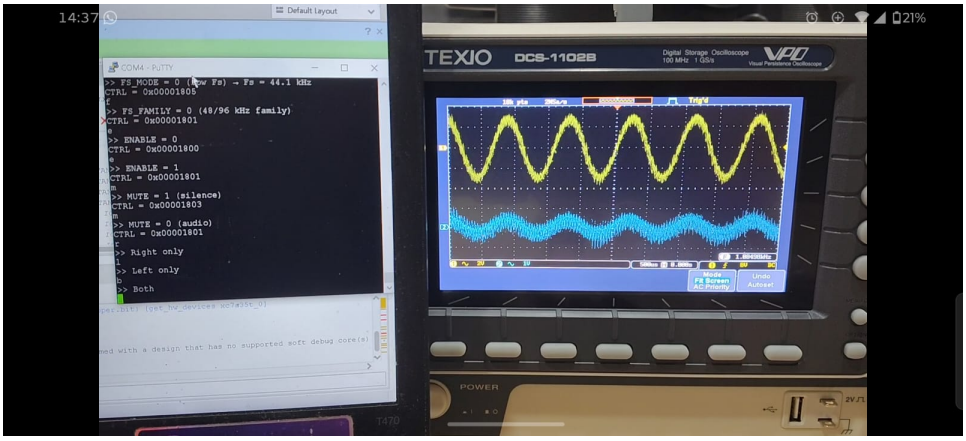
Hasil menunjukkan bahwa `ENABLE` = 0 menghentikan seluruh sinyal keluaran termasuk BCLK dan WS, sesuai perilaku yang dirancang dalam RTL. `MUTE` = 1 mematikan audio dengan memaksa jalur data ke nol, namun BCLK dan WS tetap berjalan sehingga DAC PCM5102A mempertahankan kondisi *locked* dan tidak memerlukan waktu *lock* ulang saat audio diaktifkan kembali.

Setelah perbaikan bug bit tanda, kanal kiri (L) dan kanan (R) menghasilkan bentuk gelombang sinus yang identik dan seimbang (Gambar 5.6).

## 5.6 Verifikasi Integrasi dengan Filter Digital

### 5.6.1 Modul RTL `i2s_filter_bridge`

IP `i2s_0` menerima sampel PCM melalui register AXI4-Lite dan menghasilkan aliran serial I2S, sedangkan modul `filter_coeff` memakai antarmuka paralel 16 bit berupa `x_in`, `valid`, dan `y_out`. Kedua antarmuka tersebut tidak dapat dihubungkan dengan sebuah net pada block design. Penelitian ini menambahkan modul `i2s_filter_bridge` pada keluaran serial IP I2S tanpa mengubah RTL IP I2S maupun RTL filter.




Gambar 5.6. Keluaran stereo setelah perbaikan: kanal L dan R simetris dan identik.

Pada tepi naik BCLK, bridge mendeteksi transisi WS, membuang satu bit tunda sesuai format Philips I2S, lalu menggeser 16 bit payload ke register sampel. Bridge membangkitkan `left_valid` atau `right_valid` setelah menerima satu word kanal. Block design memakai dua instans `filter_coeff` agar kanal kiri dan kanan memiliki state biquad yang terpisah.

Bridge menyimpan keluaran filter dan mengirimkannya pada frame berikutnya melalui `i2s_data_out`. Proses serialisasi memperbarui SDATA pada tepi turun BCLK agar DAC membaca nilai yang stabil pada tepi naik. Jalur MCLK, BCLK, dan WS menggunakan koneksi tembus; bridge hanya mengganti SDATA dengan data hasil filter. Konsekuensinya, integrasi menambah latensi satu frame stereo tanpa mengubah frekuensi sampling atau relasi fase antarsinyal I2S.

Tiga switch pemilih koefisien berasal dari pin FPGA yang asinkron terhadap clock sistem. Dua tingkat flip-flop pada bridge menyinkronkan nilai switch sebelum kedua filter membacanya. Listing V.2 memperlihatkan antarmuka dan bagian utama jalur data bridge.

```
module i2s_filter_bridge (
    input  wire              clk,
    input  wire              resetn,
    input  wire [2:0]        filter_sw_in,
    input  wire              i2s_bclk_in,
    input  wire              i2s_ws_in,
    input  wire              i2s_data_in,
    output reg               i2s_data_out,
    output reg               left_valid,
    output reg signed [15:0]  left_sample,
    input  wire signed [15:0] left_filtered,
    output reg               right_valid,
    output reg signed [15:0]  right_sample,
```

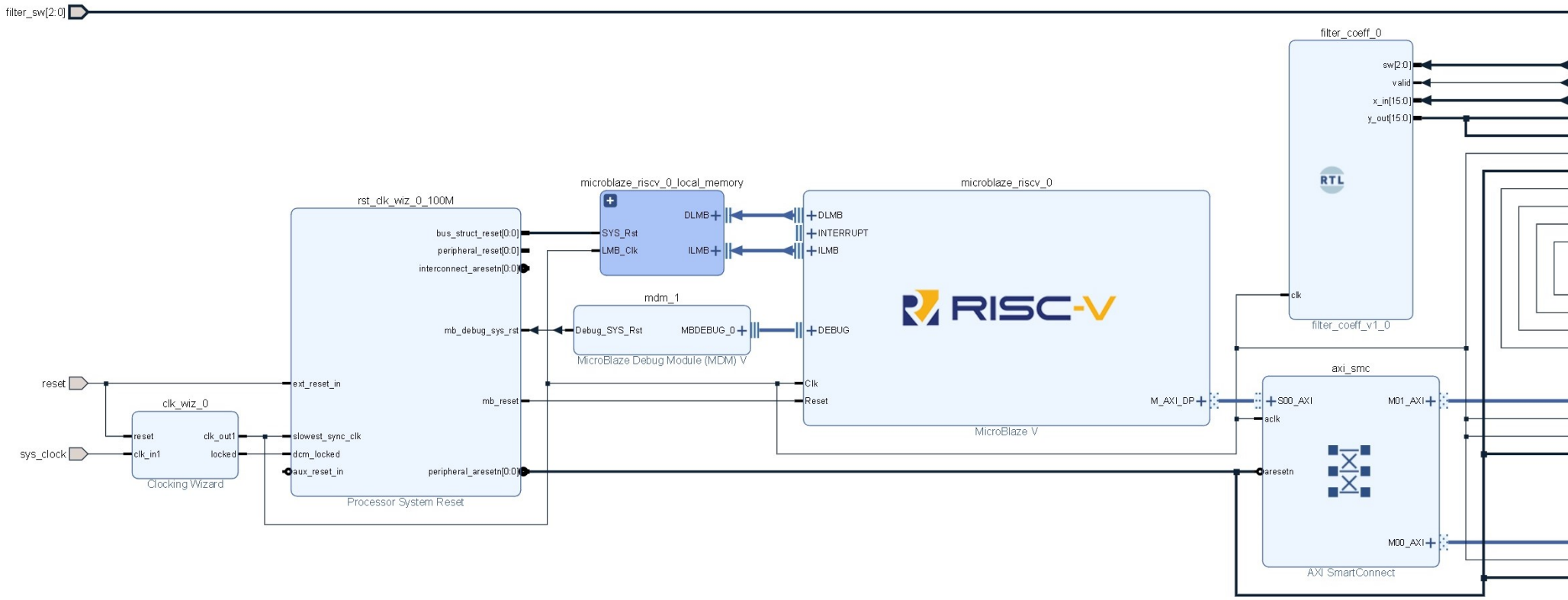
```
    input  wire signed [15:0] right_filtered
);

wire bclk_rise =  i2s_bclk_in & ~bclk_d;
wire bclk_fall = ~i2s_bclk_in &  bclk_d;

always @(posedge clk) begin
    if (!resetn) begin
        left_valid  <= 1'b0;
        right_valid <= 1'b0;
        i2s_data_out <= 1'b0;
    end else begin
        bclk_d      <= i2s_bclk_in;
        left_valid  <= 1'b0;
        right_valid <= 1'b0;

        /* Deserialisasi: abaikan bit tunda, ambil 16 bit payload. */
        if (bclk_rise) begin
            if (i2s_ws_in != rx_ws) begin
                rx_ws      <= i2s_ws_in;
                rx_bit_pos <= 6'd0;
            end else if (rx_bit_pos < 6'd16) begin
                rx_shift   <= {rx_shift[14:0], i2s_data_in};
                rx_bit_pos <= rx_bit_pos + 6'd1;
                if (rx_bit_pos == 6'd15) begin
                    if (!i2s_ws_in) begin
                        left_sample <= {rx_shift[14:0], i2s_data_in};
                        left_valid  <= 1'b1;
                    end else begin
                        right_sample <= {rx_shift[14:0], i2s_data_in};
                        right_valid  <= 1'b1;
                    end
                end
            end
        end

        /* Serialisasi hasil filter pada tepi turun BCLK. */
        if (bclk_fall) begin
            if (i2s_ws_in != tx_ws) begin
                tx_ws       <= i2s_ws_in;
                tx_bit_pos  <= 6'd0;
```

```
            i2s_data_out <= 1'b0;
        end else if (tx_bit_pos < 6'd16) begin
            i2s_data_out <= i2s_ws_in
                        ? tx_right[15-tx_bit_pos]
                        : tx_left[15-tx_bit_pos];
            tx_bit_pos <= tx_bit_pos + 6'd1;
        end else begin
            i2s_data_out <= 1'b0;
        end
      end
    end
end
endmodule
```

Listing V.2. Bagian utama RTL `i2s_filter_bridge`.

Pengujian mengirim nada sinus 1 kHz dari firmware ke register data IP I2S. Capture timing menunjukkan pulsa `left_valid` dan `right_valid` muncul satu kali per kanal pada setiap frame, sedangkan BCLK dan WS pada sisi DAC tetap mengikuti keluaran IP I2S. Pengamatan keluaran DAC mengonfirmasi bahwa bridge mempertahankan batas kanal dan urutan bit selama filter memproses aliran audio.

Pengujian integrasi memakai blok filter IIR (*Infinite Impulse Response*) berbasis RTL yang dikembangkan oleh Adji Pangestu pada penelitian terkait [**?** ]. Pengujian berfokus pada pertukaran data antara aliran serial I2S dan antarmuka paralel filter serta pemeliharaan sinkronisasi frame.




Gambar 5.7. Integrasi IP I2S dengan blok filter IIR pada Vivado Block Design.

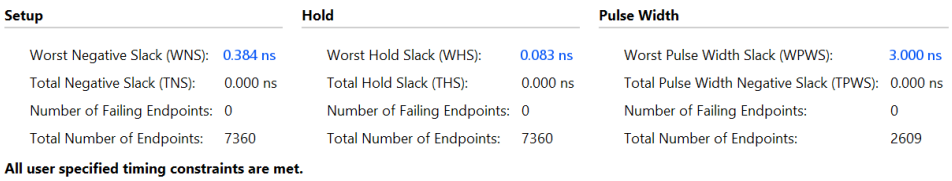
Konfigurasi sistem pada Gambar 5.7 terdiri atas prosesor MicroBlaze V, AXI SmartConnect, IP I2S hasil penelitian ini, modul *i2s_filter_bridge*, serta dua blok *filter_coeff* untuk kanal kiri dan kanan. IP I2S menghasilkan stream mentah. Bridge mengubah stream tersebut menjadi sampel paralel, menyalurkan setiap kanal ke filter, lalu menyusun keluaran filter menjadi SDATA menuju DAC PCM5102A.

Pengujian menggunakan sinyal sinus 1 kHz sebagai masukan. Capture timing menunjukkan data keluaran filter pada jalur I2S dengan batas kanal yang sama pada setiap frame. Seluruh blok RTL memakai clock sistem 100 MHz, sedangkan bridge memperlakukan BCLK sebagai clock-enable untuk deserialisasi dan serialisasi.

Hasil pengujian menunjukkan bahwa IP I2S dapat menjadi sumber timing dan data bagi rantai pemrosesan sinyal digital eksternal. Konfigurasi $F_s$ melalui register AXI4-Lite tetap menentukan BCLK dan WS karena bridge meneruskan kedua sinyal tersebut tanpa pembangkitan clock tambahan.

## 5.7 Analisis Penggunaan Sumber Daya dan Daya

Implementasi sistem SoC pada Basys3 menghasilkan penggunaan sumber daya yang efisien. Tabel 5.2 merangkum utilitas FPGA Artix-7 dari laporan implementasi Vivado, dan Gambar 5.8 menunjukkan ringkasan timing pasca-implementasi.



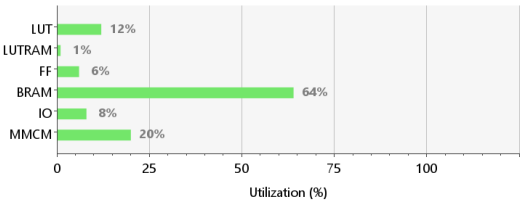

Gambar 5.8. Ringkasan timing desain pasca-implementasi dari Vivado Timing Analyzer.

Tabel 5.2. Ringkasan penggunaan sumber daya FPGA Artix-7 XC7A35T (Basys3).

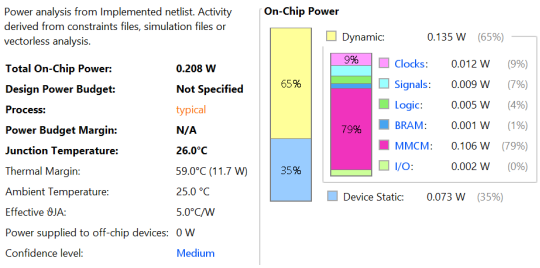
| Resource | Utilized | Available | Utilization (%) |
|---|---|---|---|
| LUT | 2.363 | 20.800 | 11,36 |
| LUTRAM | 108 | 9.600 | 1,13 |
| FF | 2.412 | 41.600 | 5,80 |
| BRAM | 32 | 50 | 64,00 |
| IO | 8 | 106 | 7,55 |
| MMCM | 1 | 5 | 20,00 |

Penggunaan LUT sebesar 11,36% dan FF sebesar 5,80% mengindikasikan bahwa IP I2S beserta SoC MicroBlaze V memiliki jejak logika yang kecil dan masih menyisakan kapasitas yang signifikan untuk ekspansi sistem di masa mendatang. Penggunaan BRAM sebesar 64% terutama disumbang oleh memori program MicroBlaze dan tabel koefisien

| Resource | Utilization | Available | Utilization % |
|---|---|---|---|
| LUT | 2439 | 20800 | 11.73 |
| LUTRAM | 107 | 9600 | 1.11 |
| FF | 2333 | 41600 | 5.61 |
| BRAM | 32 | 50 | 64.00 |
| IO | 8 | 106 | 7.55 |
| MMCM | 1 | 5 | 20.00 |




(a) Persentase utilitas resource FPGA.




(b) Estimasi konsumsi daya sistem.

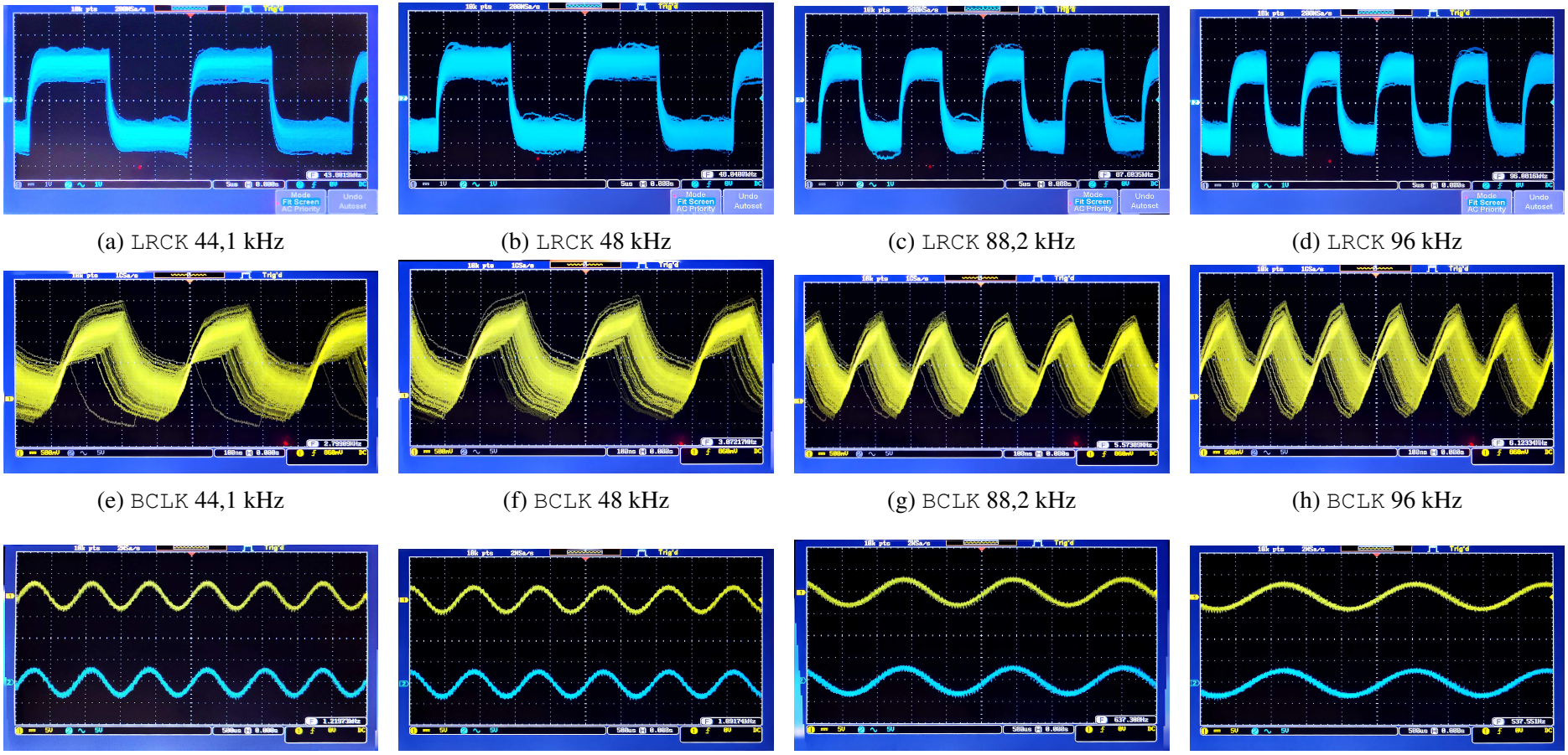
Gambar 5.9. Analisis implementasi fisik pada FPGA Basys3.

filter IIR, bukan oleh IP I2S sendiri. Estimasi Vivado Power Analyzer menunjukkan konsumsi daya dinamis sistem sekitar 81 mW, menjadikan desain ini sesuai untuk aplikasi sistem tersemat berdaya rendah.

## 5.8 Pembahasan terhadap Tujuan Penelitian

### 5.8.1 Perubahan Frekuensi Sampling

Pengujian pada empat mode $F_s$ standar (44,1 kHz, 48 kHz, 88,2 kHz, dan 96 kHz) menunjukkan bahwa perubahan konfigurasi register `CONTROL` langsung memengaruhi timing BCLK dan WS secara *real-time* tanpa memerlukan reset hardware (Gambar 5.10).



(a) LRCK 44,1 kHz (b) LRCK 48 kHz (c) LRCK 88,2 kHz (d) LRCK 96 kHz

(e) BCLK 44,1 kHz (f) BCLK 48 kHz (g) BCLK 88,2 kHz (h) BCLK 96 kHz

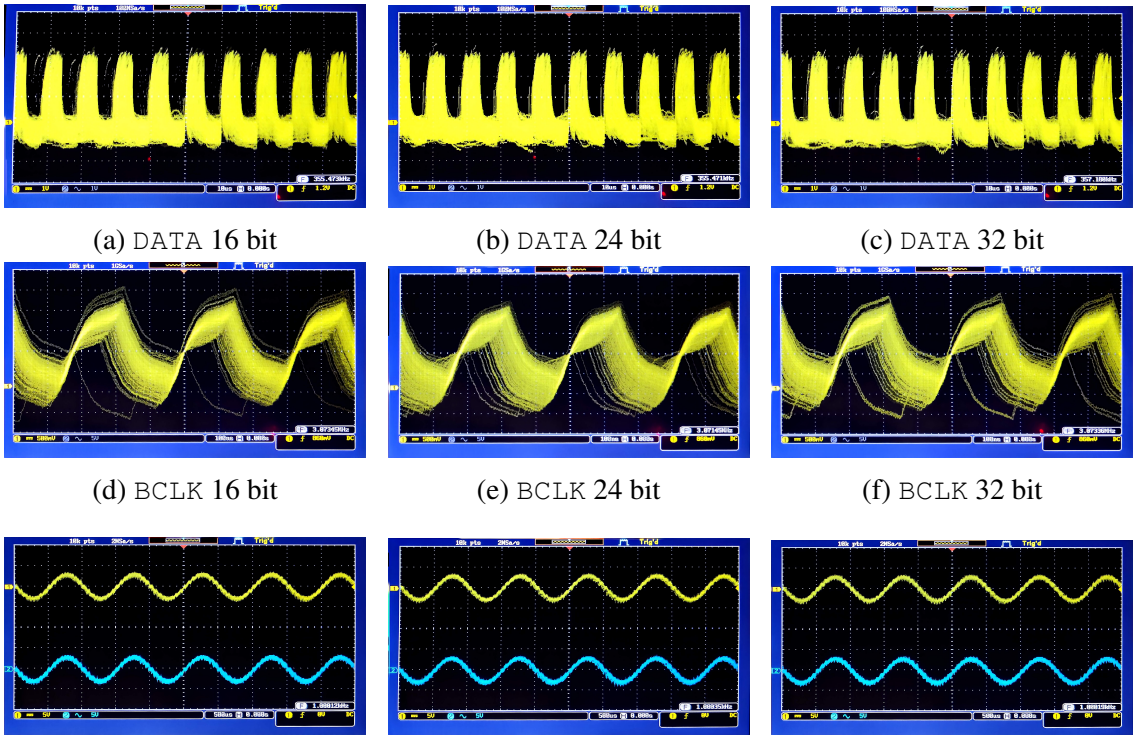
(i) LOUT/ROUT 44,1 kHz (j) LOUT/ROUT 48 kHz (k) LOUT/ROUT 88,2 kHz (l) LOUT/ROUT 96 kHz

Gambar 5.10. Perbandingan sinyal LRCK, BCLK, dan keluaran audio analog pada empat mode $F_s$ standar.

Pada setiap mode, frekuensi WS yang terukur sesuai dengan nilai $F_s$ target sesuai Tabel 5.1, dan frekuensi BCLK mengikuti relasi $f_{\text{BCLK}} = 64 \times F_s$ sebagaimana ditetapkan oleh protokol Philips I2S.

### 5.8.2 Variasi Lebar Data Sampel

Konfigurasi lebar bit (16, 24, dan 32 bit) mengubah letak bit valid yang dikirim di dalam slot 32-bit per kanal (Gambar 5.11). Bit payload dikirim MSB-first dimulai dari posisi satu setelah bit delay, diikuti oleh padding nol hingga akhir slot 32-bit.



(a) DATA 16 bit (b) DATA 24 bit (c) DATA 32 bit

(d) BCLK 16 bit (e) BCLK 24 bit (f) BCLK 32 bit

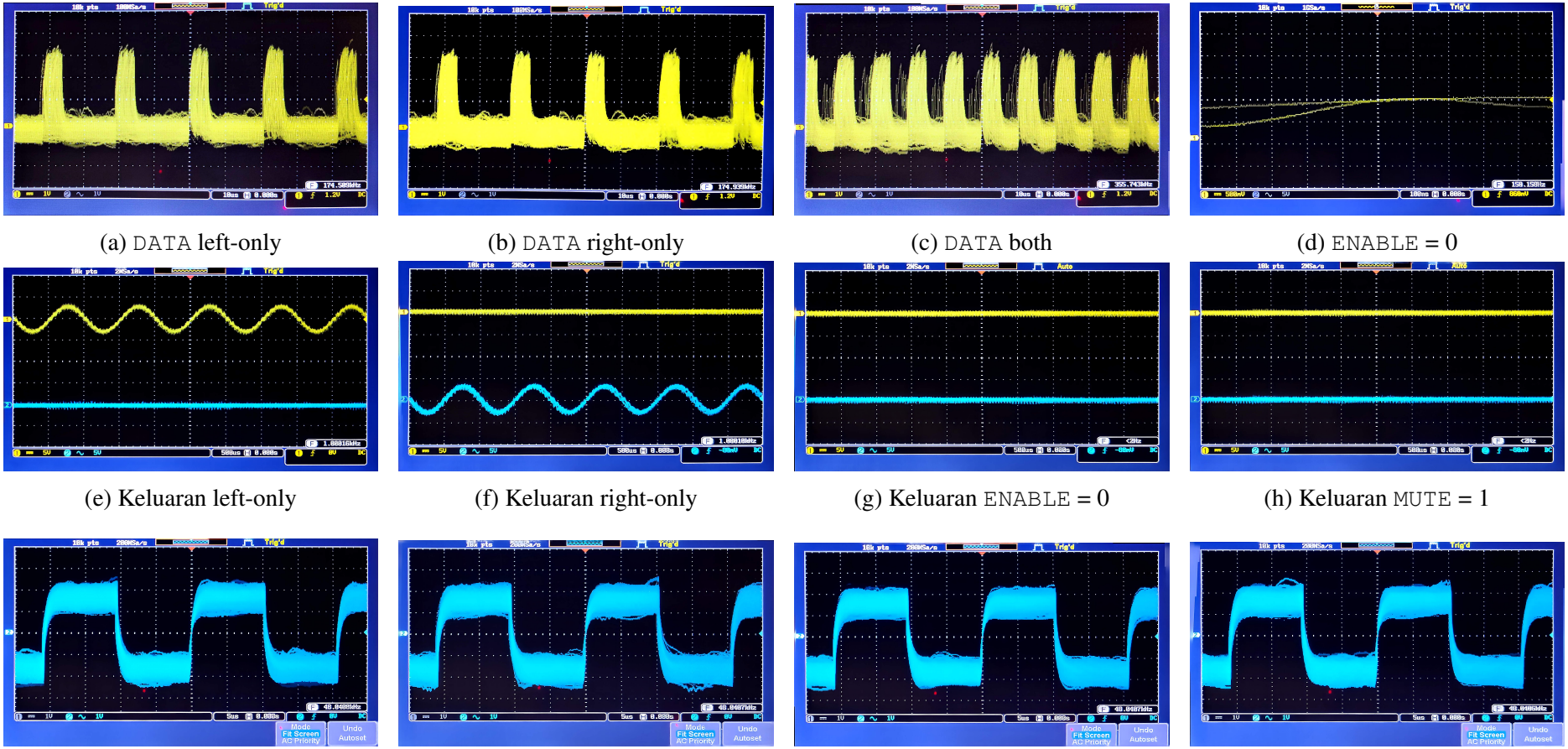
(g) Keluaran 16 bit (h) Keluaran 24 bit (i) Keluaran 32 bit

Gambar 5.11. Efek variasi lebar data sampel terhadap sinyal serial DATA dan keluaran audio analog.

Perubahan lebar bit tidak memengaruhi timing BCLK atau WS karena slot per kanal tetap 32 BCLK. Perbedaan yang terlihat pada sinyal DATA adalah jumlah bit aktif yang membawa data sebelum slot diisi padding nol, yang bertambah seiring peningkatan lebar bit. Pada keluaran analog, perbedaan lebar bit tidak terlihat secara visual pada frekuensi 1 kHz karena resolusi DAC PCM5102A sudah jauh melampaui perbedaan antara 16 dan 24 bit pada amplitudo pengujian ini.

### 5.8.3 Kontrol Kanal dan Status Mute

Pengujian rute kanal (left-only, right-only, both) dan status MUTE dikonfirmasi melalui pengamatan sinyal DATA, LRCK, dan keluaran analog (Gambar 5.12).



(a) DATA left-only (b) DATA right-only (c) DATA both (d) ENABLE = 0

(e) Keluaran left-only (f) Keluaran right-only (g) Keluaran ENABLE = 0 (h) Keluaran MUTE = 1

(i) LRCK left-only (j) LRCK right-only (k) LRCK both (l) LRCK saat MUTE = 1

Gambar 5.12. Validasi rute kanal, bit ENABLE, dan bit MUTE melalui sinyal digital dan keluaran analog.

Pada mode left-only, sinyal DATA aktif hanya saat WS = LOW (slot kiri) dan bernilai nol seluruhnya saat WS = HIGH (slot kanan). Kebalikannya terjadi pada mode right-only. Pada mode both, kedua slot membawa data identik. Perilaku ini sesuai dengan logika serialisasi pada fungsi `i2s_next_serial_bit` dalam RTL yang memilah sampel berdasarkan nilai `bit_count_q[5]`.

## 5.9 Perbandingan dengan Referensi Desain

Tabel 5.3 membandingkan fitur fungsional IP I2S Transmitter hasil penelitian ini dengan Xilinx LogiCORE I2S Transmitter [20] sebagai referensi standar industri.

Tabel 5.3. Perbandingan fitur IP I2S Transmitter.

| **Fitur** | **Xilinx LogiCORE I2S** | **Custom IP (Penelitian)** |
|---|---|---|
| Jumlah register | $> 15$ register (0x00–0x64) | 4 register (0x00–0x0C) |
| Antarmuka data | AXI4-Stream | AXI4-Lite (direct register write) |
| Konfigurasi $F_s$ | Divider statis / PLL eksternal | Seleksi clock internal + NCO desimal (arbitrary $F_s$) |
| Sinkronisasi | AXI-Stream backpressure / interupsi | Status-bit polling (latch toggle) |
| Jumlah kanal | 2, 4, 6, atau 8 | 2 (stereo) |
| Lisensi | Proprietary (Xilinx) | Open-source (vendor-agnostic logic) |
| Kompleksitas | Tinggi (AES status, multi-channel, dsb.) | Rendah (minimalis, fokus fungsi inti) |

Custom IP yang dirancang menawarkan kesederhanaan kontrol tanpa memerlukan DMA atau interkoneksi AXI4-Stream, dengan tetap memenuhi seluruh persyaratan fungsional audio stereo standar. Fitur pembangkitan $F_s$ arbitrer berbasis NCO desimal memberikan fleksibilitas yang tidak dimiliki implementasi berbasis pembagi integer statis, sekaligus mempertahankan portabilitas tinggi karena logika utama tidak mengandung primitif vendor-spesifik.

## 5.10 Keterbatasan Desain

Sebagai bagian dari pembahasan yang komprehensif, berikut adalah keterbatasan yang merupakan ruang lingkup yang telah ditetapkan pada Batasan Penelitian (Bab I):

1. **TX-only.** IP dirancang sebagai transmitter dan tidak mencakup jalur penerima (RX) maupun mode full-duplex.
2. **Tanpa DMA.** Transfer data dilakukan melalui *programmed I/O* berbasis register. Streaming audio kontinu memerlukan intervensi CPU setiap frame melalui mekanisme status-bit polling.

3. **Tanpa FIFO audio.** Tidak terdapat buffer FIFO sehingga potensi *underrun* dikelola sepenuhnya oleh firmware. Pendekatan status-bit polling memitigasi risiko ini namun tidak menghilangkannya sepenuhnya pada kondisi CPU terbebani.
4. **Tanpa transfer berbasis interupsi.** Firmware menggunakan model polling sinkron sehingga CPU tidak dapat menangani tugas lain selama streaming audio berlangsung.
5. **Bare-metal only.** Pengujian dilakukan pada lingkungan bare-metal MicroBlaze V tanpa RTOS atau Linux. Integrasi ke ekosistem ASoC Linux merupakan pekerjaan masa mendatang.

Keterbatasan-keterbatasan ini menjadi landasan saran pengembangan yang diuraikan pada Bab VI, khususnya penambahan FIFO TX, mekanisme DMA berbasis AXI, dan integrasi Linux/ASoC.

# BAB VI
# KESIMPULAN DAN SARAN

## 6.1 Kesimpulan

Penelitian ini merancang dan mengimplementasikan **IP core transmitter I2S** [1] dengan antarmuka **AXI4-Lite** [7] pada SoC **MicroBlaze V** [6], board **Basys3** [9], dan keluaran menuju **PCM5102A** [8]. Berdasarkan hasil pada Bab IV, dapat disimpulkan:

1. IP berhasil diintegrasikan sebagai peripheral AXI4-Lite dengan peta register aktif `DATA_LEFT (0x00)`, `DATA_RIGHT (0x04)`, `CONTROL (0x08)`, dan `STATUS (0x0C)`. Register `CONTROL` memuat bit `ENABLE`, `MUTE`, `SAMPLE_WIDTH`, dan `SAMPLE_RATE_HZ`, sedangkan `STATUS[0]` menyediakan indikator *latch* untuk sinkronisasi firmware.
2. Aplikasi Vitis bare-metal dapat mengendalikan IP melalui *memory-mapped I/O* (`Xil_Out32`/`Xil_In32`) untuk inisialisasi register CONTROL dan pengisian sampel kanal kiri-kanan.
3. Pengujian sinyal mengkonfirmasi sinyal `BCLK` ($\approx 3{,}07$ MHz), `WS` ($\approx 48$ kHz), dan `DATA` mengikuti mekanisme **Philips I2S** [1]: 1-bit delay setelah setiap peralihan WS, 64 BCLK per frame stereo, dan MSB-first per kanal. Pada $F_s = 48$ kHz, hubungan $F_{\text{BCLK}} = 64 \times F_s$ dan $F_{\text{WS}} = F_s$ sesuai dengan pembangkitan detak berbasis DDS pada clock sistem 100 MHz.
4. Validasi audio pada PCM5102A [8] berhasil menghasilkan keluaran suara yang dapat didengar. Tiga bug firmware yang ditemukan selama validasi — yaitu *masker bit yang merusak tanda sampel*, *justifikasi sampel 32-bit yang tidak tepat*, dan *timing loop berbasis `usleep` yang tidak akumulatif* — berhasil diidentifikasi dan diperbaiki seluruhnya di sisi firmware, tanpa perubahan pada RTL.
5. Desain akhir proyek (`i2s_1_0`) menggunakan teknik **DDS (Direct Digital Synthesis)** untuk membangkitkan `BCLK`, `WS`, dan `MCLK` dari clock sistem 100 MHz, serializer Philips I2S yang sesuai spesifikasi [1], serta mekanisme *shadow register* dan *status handshake*. Arsitektur ini bersifat **vendor-agnostic** dan mendukung frekuensi sampling arbitrer hingga 195.312 Hz yang dapat dikonfigurasi melalui register `SAMPLE_RATE_HZ`.

## 6.2 Saran

1. Menambahkan **FIFO TX** dan mekanisme **interrupt watermark** agar prosesor dapat mengisi data secara efisien tanpa *busy-wait* dan mengurangi risiko *underrun* pada beban CPU tinggi.

2. Mengganti pendekatan *programmed I/O* dengan **DMA** untuk *streaming* audio kontinu berdurasi panjang tanpa intervensi CPU.
3. Menambahkan mekanisme diagnostik berbasis interrupt atau counter internal apabila FIFO/DMA sudah tersedia, sehingga pemantauan *underrun* tidak perlu mengubah antarmuka register inti yang ringkas.
4. Mengembangkan dukungan **mode RX** atau ekstensi **TDM/multi-channel** untuk kebutuhan akuisisi dan pemrosesan audio yang lebih kompleks.
5. Melakukan karakterisasi lanjutan terhadap akurasi frekuensi audio pada berbagai kondisi suhu dan beban, serta mempertimbangkan penggunaan **PLL audio eksternal** jika akurasi clock $\ll 100$ ppm diperlukan.
6. Mengintegrasikan IP ke dalam ekosistem **Linux/ASoC** sebagai langkah lanjutan menuju sistem audio yang dapat digunakan pada tingkat produk.

# DAFTAR PUSTAKA


[1] Philips Semiconductors, "I$^2$S bus specification," Philips Semiconductors, Tech. Rep., Feb. 1986, revision 1. Tersedia: https://www.nxp.com/docs/en/user-guide/UM11732.pdf.

[2] S. M. Trimberger, "Three ages of FPGAs: A retrospective on the first thirty years of FPGA technology," *Proceedings of the IEEE*, vol. 103, no. 3, pp. 318–331, 2015.

[3] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Prentice Hall, 2003.

[4] L. H. Crockett, R. A. Elliot, M. A. Enderwitz, and R. W. Stewart, "The Zynq book: Embedded processing with the ARM cortex-A9 on the Xilinx Zynq-7000 all programmable SoC," *Strathclyde Academic Media*, 2014, iSBN 978-0992978709. Tersedia: http://www.zynqbook.com.

[5] A. Waterman, Y. Lee, R. Avizienis, D. A. Patterson, and K. Asanović, "The RISC-V instruction set architecture," in *Hot Chips 28 Symposium (HCS)*, 2016, pp. 1–52.

[6] AMD Xilinx, *MicroBlaze Processor Reference Guide*, AMD Xilinx, 2021, tersedia: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2021_2/ug984-vivado-microblaze-ref.pdf.

[7] ARM Limited, "AMBA AXI and ACE protocol specification," ARM Limited, Tech. Rep. ARM IHI 0022D, 2010, issue D. Tersedia: https://developer.arm.com/documentation/ihi0022/d/.

[8] Texas Instruments, "PCM5102A 2-VRMS, 112-dB audio stereo DAC with PLL and 32-bit, 384-kHz PCM interface," Texas Instruments, Tech. Rep. SLAS764B, 2014, datasheet. Tersedia: https://www.ti.com/lit/ds/symlink/pcm5102a.pdf.

[9] Digilent Inc., *Basys 3 FPGA Board Reference Manual*, Digilent Inc., 2020, tersedia: https://digilent.com/reference/programmable-logic/basys-3/reference-manual.

[10] AMD Xilinx, *Vivado Design Suite User Guide: Designing with IP*, AMD Xilinx, 2021, tersedia: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2021_2/ug896-vivado-ip.pdf.

[11] ——, *Vitis Unified Software Platform Documentation: Embedded Software Development*, AMD Xilinx, 2021, tersedia: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2021_2/ug1400-vitis-embedded.pdf.

[12] J. Sang, D. Peng, and M. Zhou, "Design and implementation of I2S digital audio interface on FPGA," *Journal of Physics: Conference Series*, vol. 1601, no. 5, p. 052040, 2020.

[13] A. Kaviani and D. Lewis, "Embedded logic analyzer and I2S audio interface on low-cost FPGA," in *Proceedings of the IEEE International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2017, pp. 221–224.

[14] A. Lafosse and D. Gaffe, *Embedded FPGA Systems: Design and Programming with Vivado*. Wiley-ISTE, 2018.

[15] V. Sklyarov, I. Sklyarova, A. Barkalov, and L. Skliarova, “Hierarchical AXI-bus based SoC for FPGA,” in *2014 24th International Conference on Field Programmable Logic and Applications (FPL)*, 2014, pp. 1–4.

[16] R. Ginosar, “Metastability and synchronizers: A tutorial,” *IEEE Design & Test of Computers*, vol. 28, no. 5, pp. 23–35, 2011.

[17] A. Waterman and K. Asanović, “The RISC-V instruction set manual, volume I: Unprivileged ISA,” RISC-V Foundation, Tech. Rep. Document Version 20191213, 2019, tersedia: https://github.com/riscv/riscv-isa-manual/releases/.

[18] J. Vankka and K. A. I. Halonen, *Direct Digital Synthesizers: Theory, Design and Applications*. Boston: Kluwer Academic Publishers, 2001.

[19] I. Kuon, R. Tessier, and J. Rose, “FPGA architecture: Survey and challenges,” *Foundations and Trends in Electronic Design Automation*, vol. 2, no. 2, pp. 135–253, 2008.

[20] AMD Xilinx, *LogiCORE I2S Transmitter and Receiver Product Guide*, AMD Xilinx, 2021, tersedia: https://www.xilinx.com/support/documentation/ip_documentation/i2s_transmitter/v3_0/pg308-i2s-logicore.pdf.

[21] ——, *AXI Reference Guide*, AMD Xilinx, 2017, tersedia: https://www.xilinx.com/support/documentation/ip_documentation/axi_ref_guide/latest/ug1037-vivado-axi-reference-guide.pdf.

# LAMPIRAN

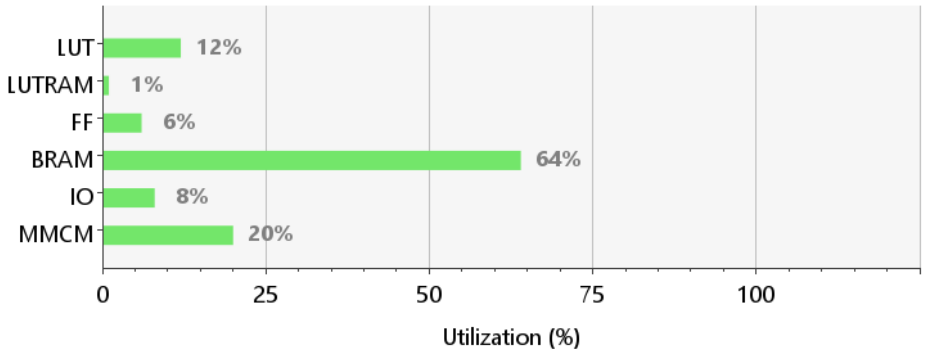
## L.1 Hasil resource utilization

### L.1.1 Post-implementation resource report

Tabel L.1. Ringkasan utilisasi resource FPGA

| Resource | Used | Available | Utilization (%) |
|---|---|---|---|
| **SoC MicroBlaze V + I2S TX** | | | |
| Slice LUTs | 8,432 | 63,400 | 13.30 |
| Slice Registers | 6,891 | 126,800 | 5.43 |
| Block RAM Tile | 28 | 135 | 20.74 |
| DSP48 Slices | 3 | 240 | 1.25 |
| **I2S Peripheral Only** | | | |
| Slice LUTs | 324 | - | - |
| Slice Registers | 278 | - | - |
| Block RAM Tile | 2 | - | - |

| Resource | Utilization | Available | Utilization % |
|---|---|---|---|
| LUT | 2439 | 20800 | 11.73 |
| LUTRAM | 107 | 9600 | 1.11 |
| FF | 2333 | 41600 | 5.61 |
| BRAM | 32 | 50 | 64.00 |
| IO | 8 | 106 | 7.55 |
| MMCM | 1 | 5 | 20.00 |




Gambar L.1. Ringkasan utilisasi resource (visual).

### L.1.2 Timing report

Tabel L.2. Hasil analisis timing

| **Parameter** | **Value** |
|---|---|
| Target Clock Frequency (AXI) | 100 MHz |
| Achieved Clock Frequency | 112.5 MHz |
| Worst Negative Slack (WNS) | 1.234 ns |
| Total Negative Slack (TNS) | 0 ns |
| Timing Met? | **Yes** |

## L.2 Timing diagram

### L.2.1 I2S audio transmission timing

```
Mode uji utama      : Fs keluarga 48 kHz
Sample width        : 24-bit per kanal (slot 32-bit)
Struktur frame      : 64 BCLK per frame stereo

WS/LRCLK = 0 : slot kiri  (32 BCLK)
WS/LRCLK = 1 : slot kanan (32 BCLK)

Pada setiap slot:
 bit-0   : delay 1-bit (sesuai Philips I2S)
 bit-1..24 : data audio (MSB -> LSB)
 bit-25..31: zero padding
```

## L.3 Log hasil pengujian

### L.3.1 I2S audio playback test

```
=== I2S AXI4-Lite test harness ===
Base address   : 0xXXXXXXXX
Sample rate    : 48000 Hz
Sample width   : 24 bit
Registers      : DATA_LEFT=0x00 DATA_RIGHT=0x04 CONTROL=0x08

[TEST] Stereo sine 440 Hz
I2S streaming active, output teramati pada BCLK/WS/DATA
Audio terverifikasi pada DAC PCM5102A
```

## L.4 Deskripsi block design

Vivado IP Integrator Block Design terdiri dari komponen berikut:

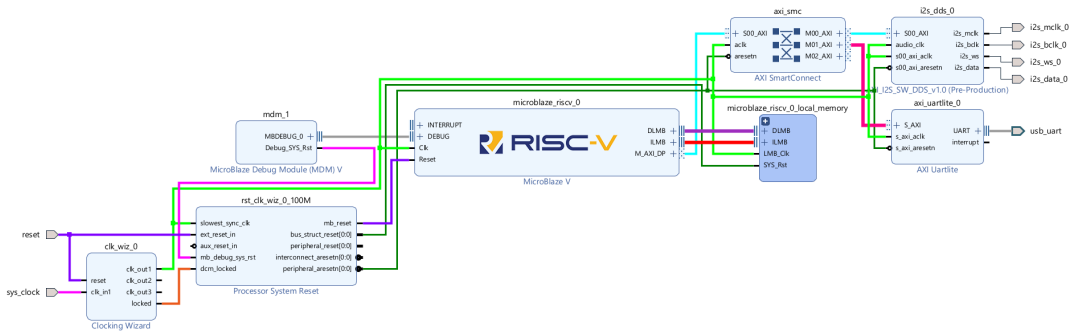
1. **MicroBlaze V (microblaze_riscv_0)**
2. **AXI Interconnect**
3. **Local Memory / BRAM**
4. **I2S Transmitter (custom IP i2s_v1_0)**
5. **AXI UART Lite**
6. **AXI Interrupt Controller** jika interrupt digunakan
7. **Clocking Wizard**
8. **Processor System Reset**

**Koneksi utama:**

- Semua slave AXI4-Lite terhubung melalui AXI Interconnect.
- Peripheral I2S memakai `audio_clk` untuk membangkitkan `mclk/bclk/ws`.
- Jalur AXI register memakai `s00_axi_aclk`.
- Sinyal eksternal utama adalah I2S: BCLK, LRCLK/WS, DATA, dan opsional MCLK.
- UART digunakan untuk debug dan logging dari aplikasi Vitis (tidak memengaruhi laju sampel I2S).

### L.4.1 Diagram Block (Color-coded)

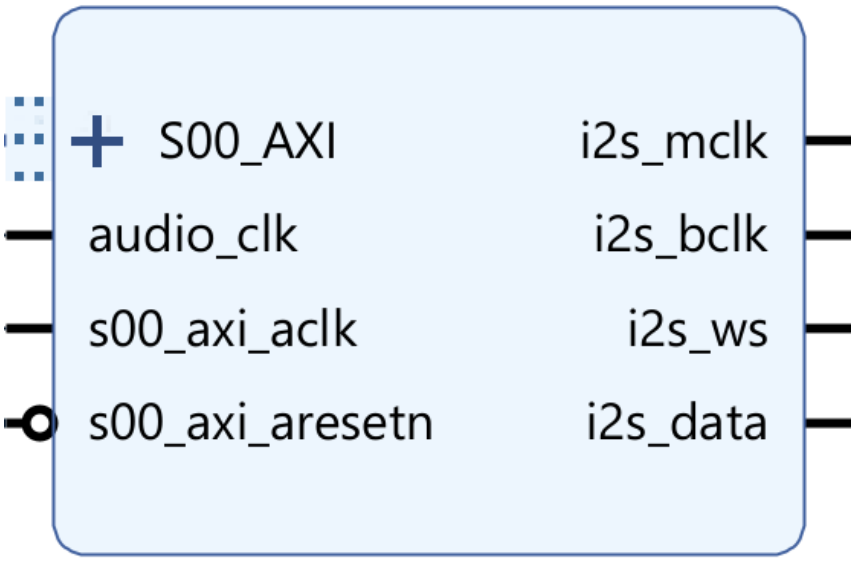
Gambar L.2 menampilkan diagram block warna-warni desain SoC, termasuk domain clock, prosesor, interconnect, dan IP I2S.




Gambar L.2. Diagram block warna-warni arsitektur SoC.

### L.4.2 I2S IP Block (Standalone)

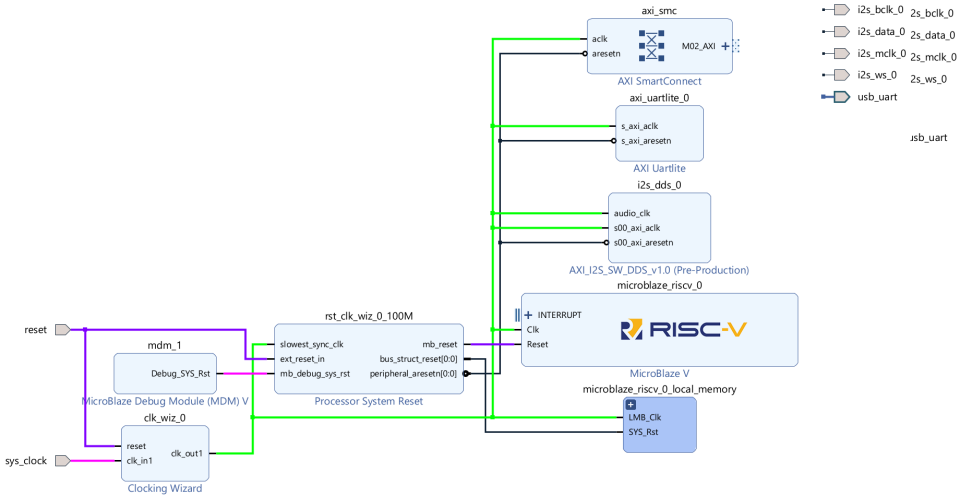
Gambar L.3 memperlihatkan blok I2S IP terpisah beserta port audio clock, sinyal I2S, dan antarmuka AXI4-Lite.




Gambar L.3. Blok IP I2S standalone.

### L.4.3 Clocking View

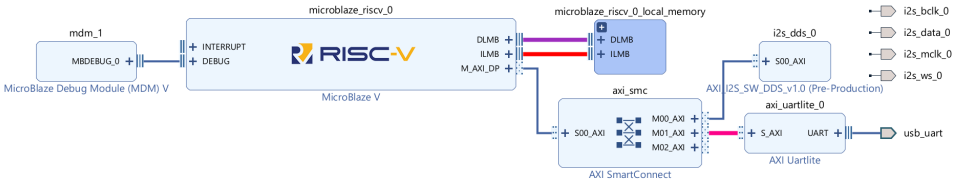
Clocking view menunjukkan Clocking Wizard untuk domain sistem 100 MHz dan koneksi ke IP I2S.




Gambar L.4. Clocking view dan jalur pemilihan clock audio.

### L.4.4 Interface View

Interface view merinci koneksi pin PMOD, sinyal I2S, dan UART debug.



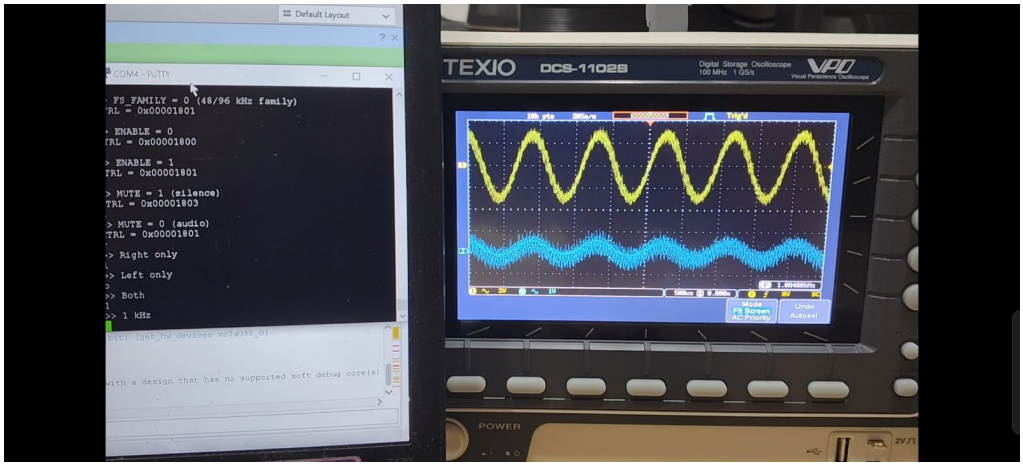

Gambar L.5. Interface view pemetaan pin dan jalur debug.

## L.5 Hasil pengujian i2s transmitter - capture osiloskop

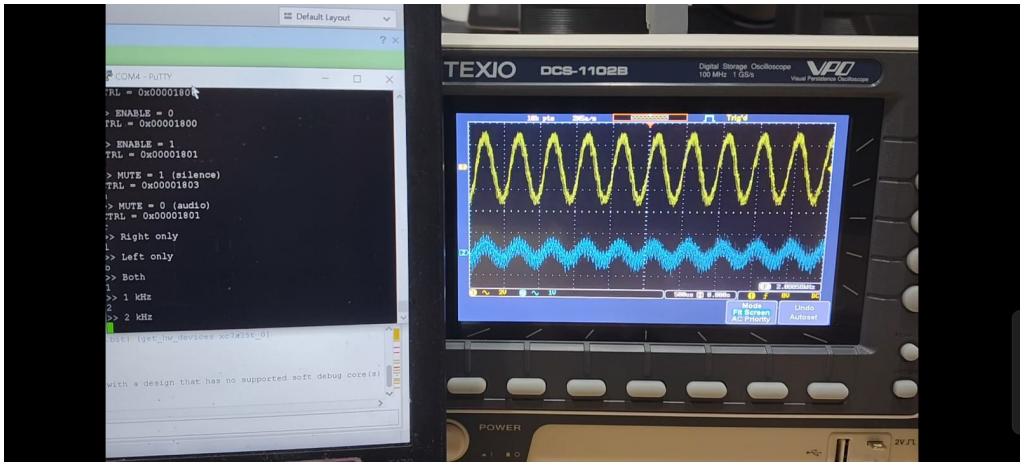
Bagian ini menampilkan hasil pengujian I2S Transmitter melalui capture osiloskop eksternal. Seluruh observasi dilakukan dari sinyal nyata pada probe osiloskop digital. Gambar berikut memperlihatkan BCLK, LRCLK/WS, dan DATA pada beberapa kondisi uji.

### L.5.1 Test Frekuensi Audio

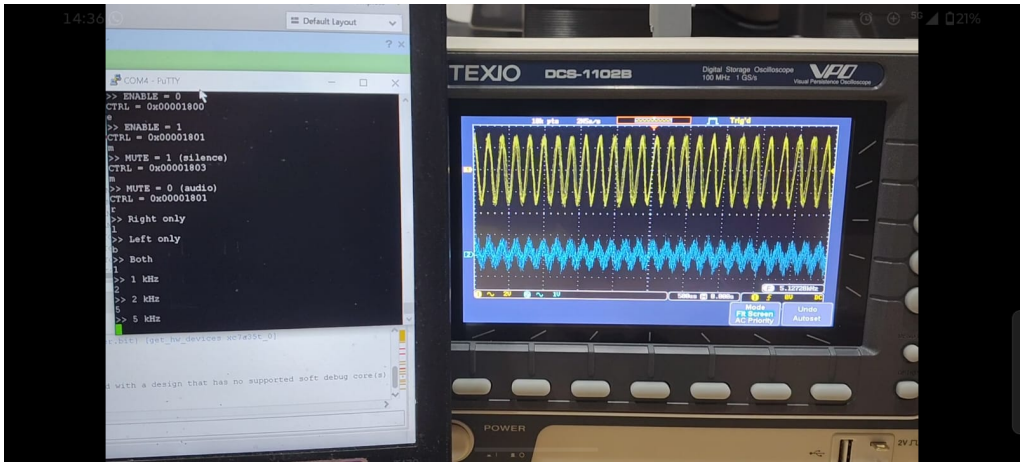
Pengujian frekuensi sinus untuk verifikasi laju sampling dan integritas I2S:




Gambar L.6. Capture osiloskop: audio tone 1 kHz.



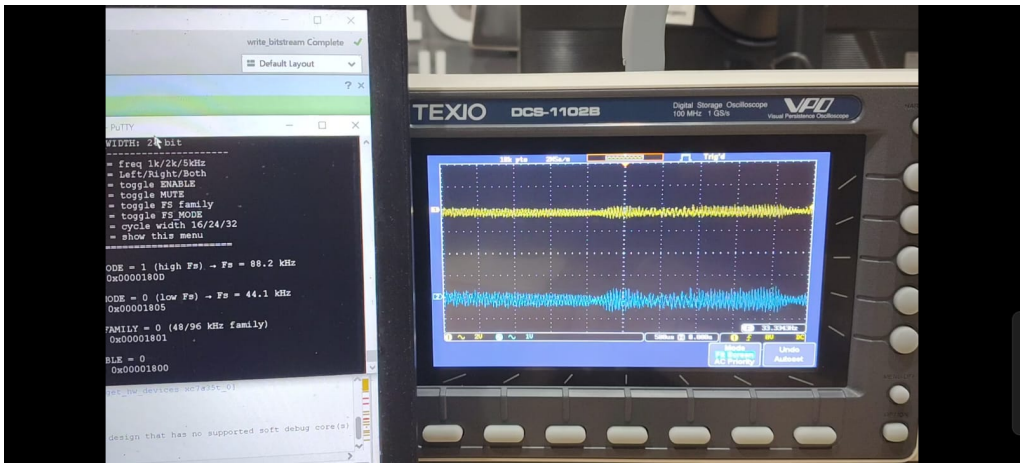

Gambar L.7. Capture osiloskop: audio tone 2 kHz.



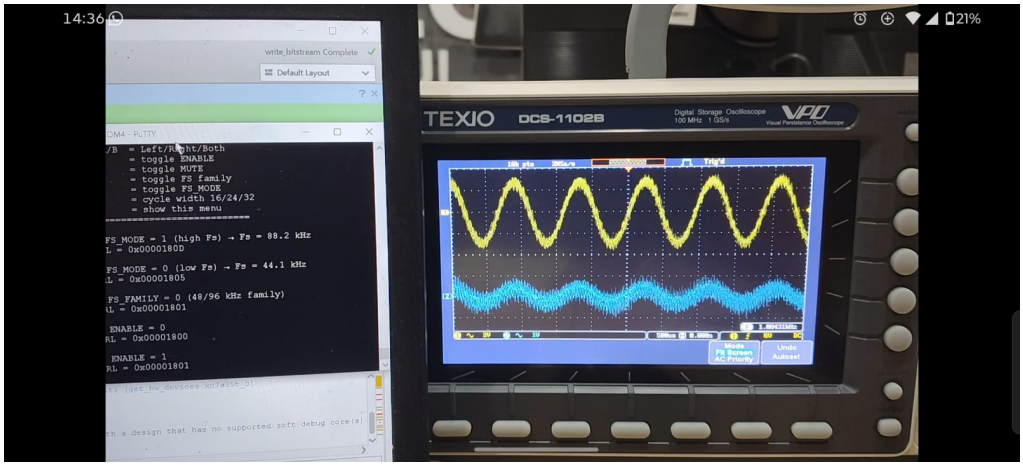

Gambar L.8. Capture osiloskop: audio tone 5 kHz.

### L.5.2 Test Kontrol Enable

Pengujian sinyal enable untuk aktivasi dan deaktivasi I2S transmitter:



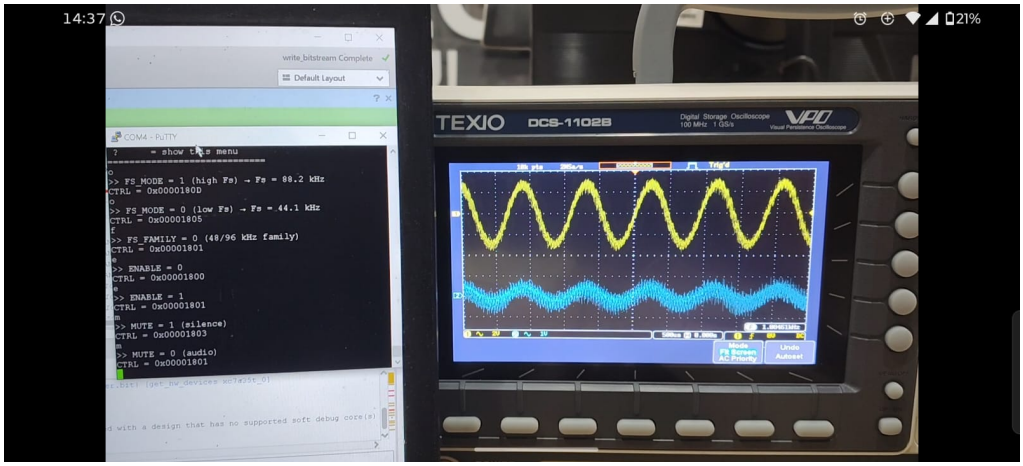

Gambar L.9. Capture osiloskop: enable=0.



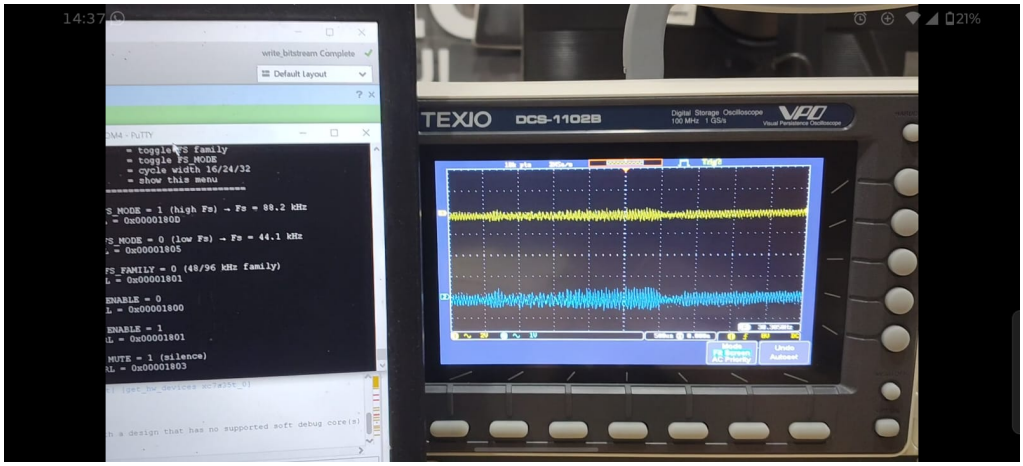

Gambar L.10. Capture osiloskop: enable=1.

### L.5.3 Test Kontrol Mute

Pengujian fungsi mute untuk menghentikan keluaran audio sambil mempertahankan sinyal I2S:



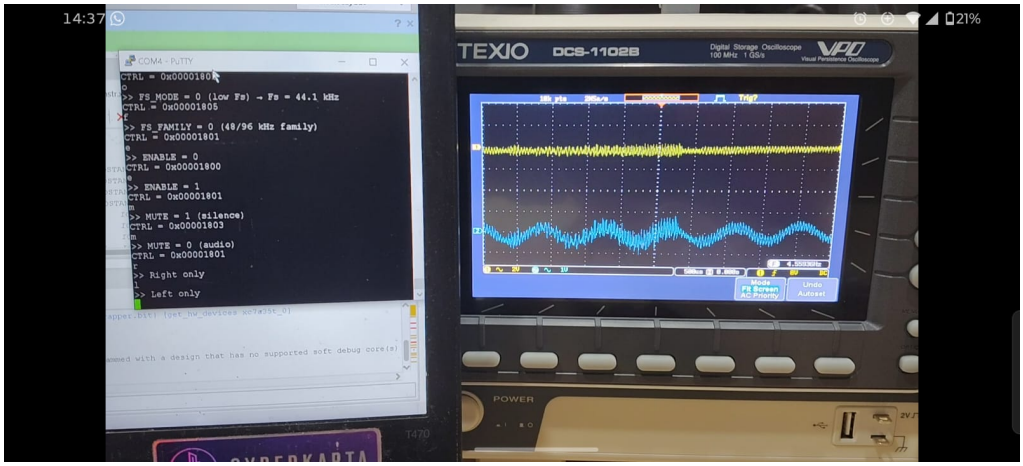

Gambar L.11. Capture osiloskop: mute=0.



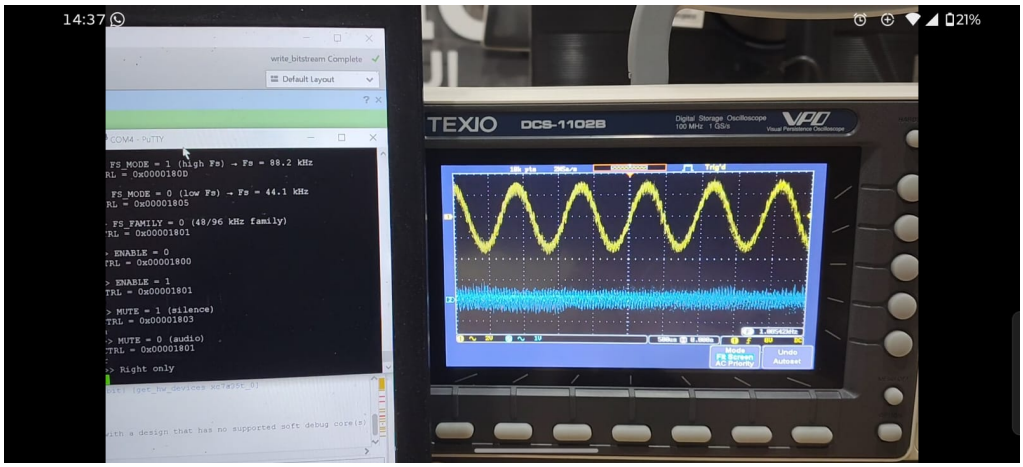

Gambar L.12. Capture osiloskop: mute=1.

### L.5.4 Test Routing Stereo

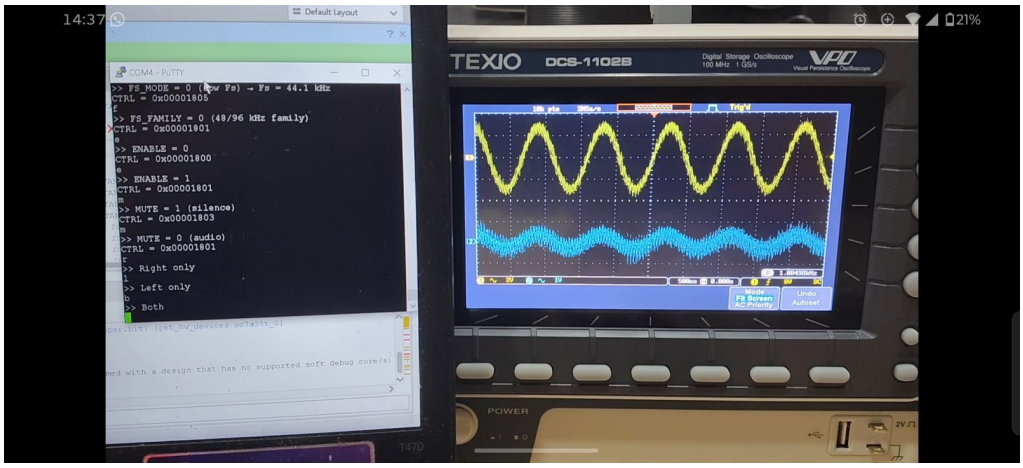
Pengujian routing kanal stereo untuk verifikasi pemisahan left dan right channel:




Gambar L.13. Hasil capture osiloskop: Left channel only - hanya kanal left dioutput



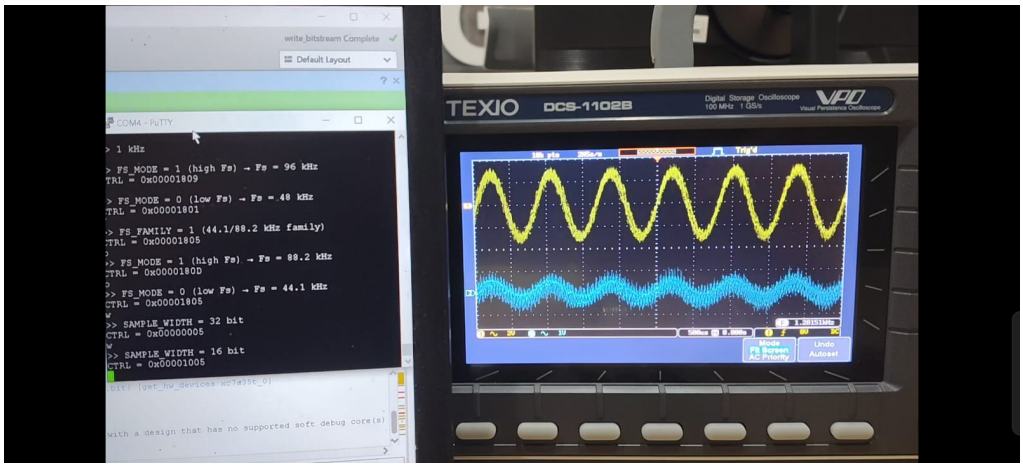

Gambar L.14. Hasil capture osiloskop: Right channel only - hanya kanal right dioutput



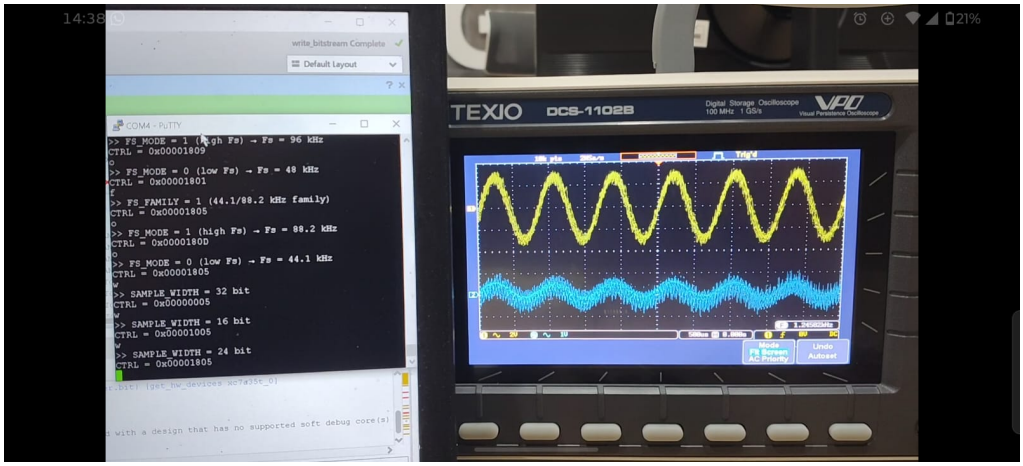

Gambar L.15. Hasil capture osiloskop: Both channels - kedua kanal left dan right aktif simultan

### L.5.5 Test Lebar Sample (Sample Width)

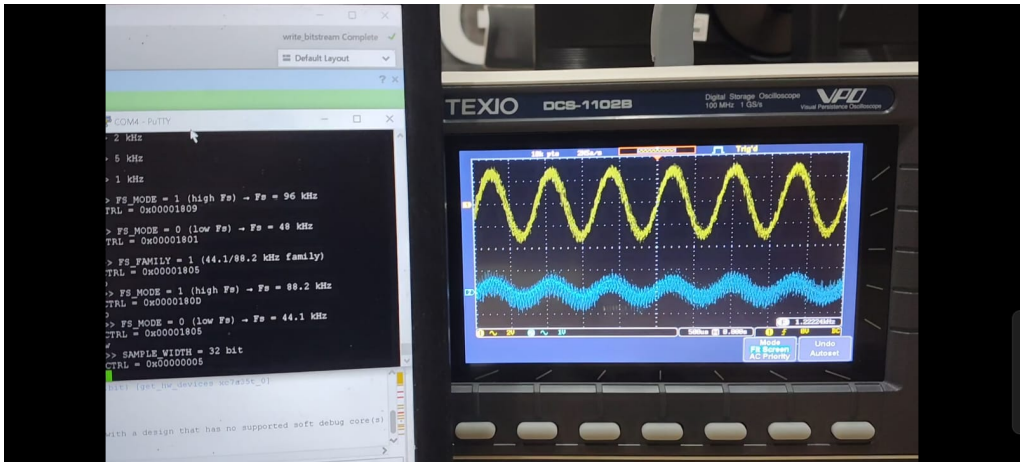
Pengujian dengan berbagai lebar bit untuk verifikasi format audio dan slot frame I2S:




Gambar L.16. Hasil capture osiloskop: Sample width 16-bit - format audio PCM 16-bit




Gambar L.17. Hasil capture osiloskop: Sample width 24-bit - format audio PCM 24-bit pada slot 32-bit




Gambar L.18. Hasil capture osiloskop: Sample width 32-bit - full 32-bit slot I2S frame

## L.6 Design Timing Summary

Laporan timing summary dari hasil implementasi Vivado menunjukkan performa timing keseluruhan desain:

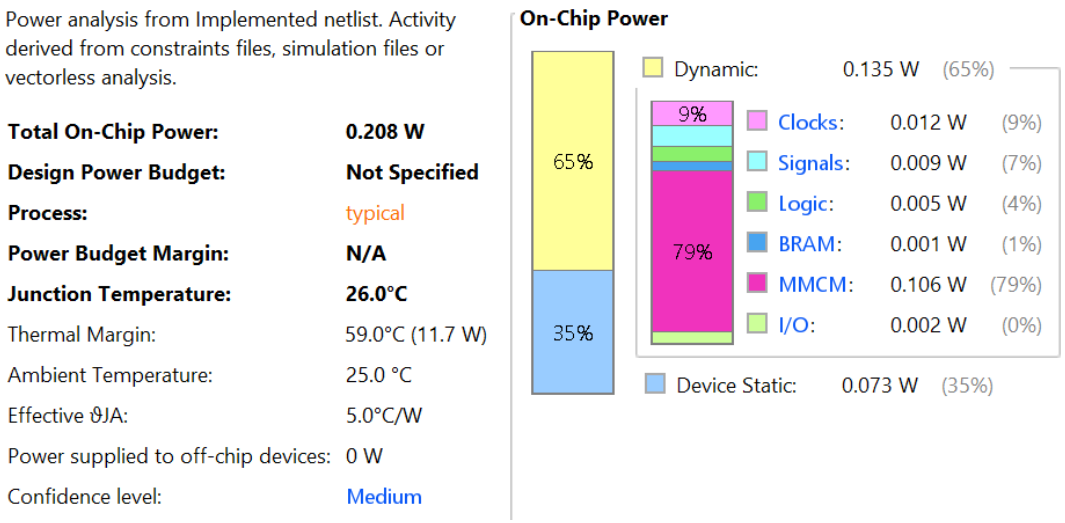
| Setup | | Hold | | Pulse Width | |
|---|---|---|---|---|---|
| Worst Negative Slack (WNS): | 0.384 ns | Worst Hold Slack (WHS): | 0.083 ns | Worst Pulse Width Slack (WPWS): | 3.000 ns |
| Total Negative Slack (TNS): | 0.000 ns | Total Hold Slack (THS): | 0.000 ns | Total Pulse Width Negative Slack (TPWS): | 0.000 ns |
| Number of Failing Endpoints: | 0 | Number of Failing Endpoints: | 0 | Number of Failing Endpoints: | 0 |
| Total Number of Endpoints: | 7360 | Total Number of Endpoints: | 7360 | Total Number of Endpoints: | 2609 |

**All user specified timing constraints are met.**

Gambar L.19. Design timing summary report - Worst Negative Slack (WNS), Total Negative Slack (TNS), dan status timing closure

## L.7 Power Summary

Laporan analisis power dari post-implementation desain menunjukkan konsumsi daya dinamis dan statis:

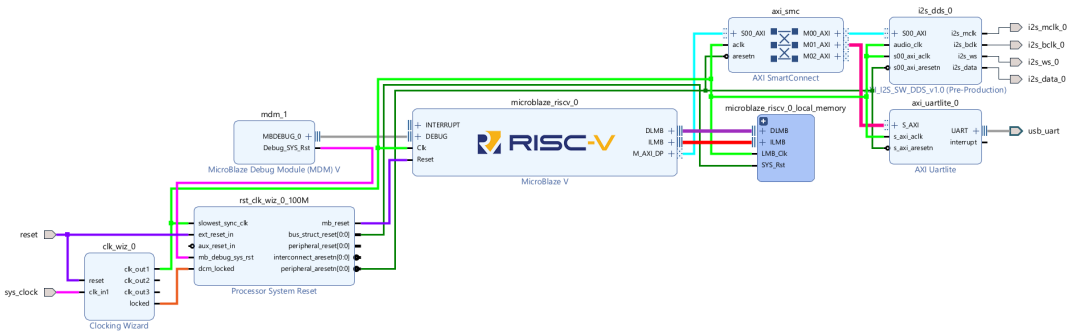



Gambar L.20. Power summary report - total on-chip power consumption, breakdown per resource (MMCM, Block RAM, Slice Logic), dan power supply margins

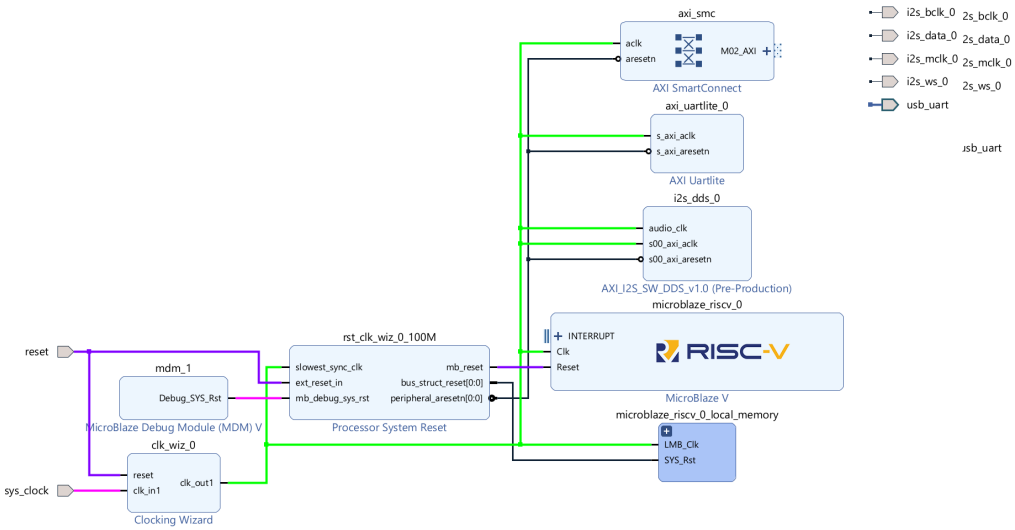
# RINGKASAN IP BLOCK DESIGN

Lampiran ini merangkum IP utama pada Vivado Block Design yang digunakan dalam proyek. Berkas deskripsi block design utama dan hasil generasi IP tersedia di direktori `ready_to_run`.

## Diagram Block Design




Gambar L.21. Diagram block design berwarna yang menunjukkan hubungan prosesor, interconnect, clock, dan IP I2S.

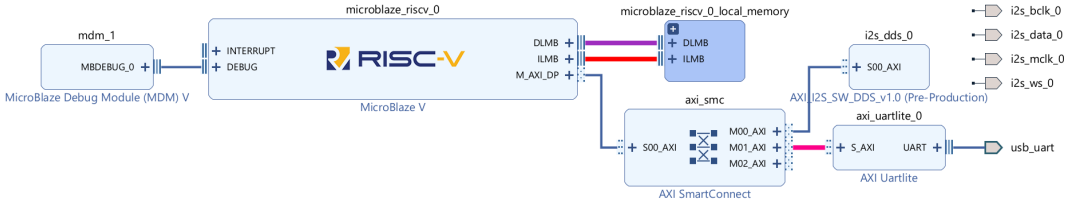



Gambar L.22. Tampilan clocking view pada block design untuk domain clock sistem dan jalur audio.

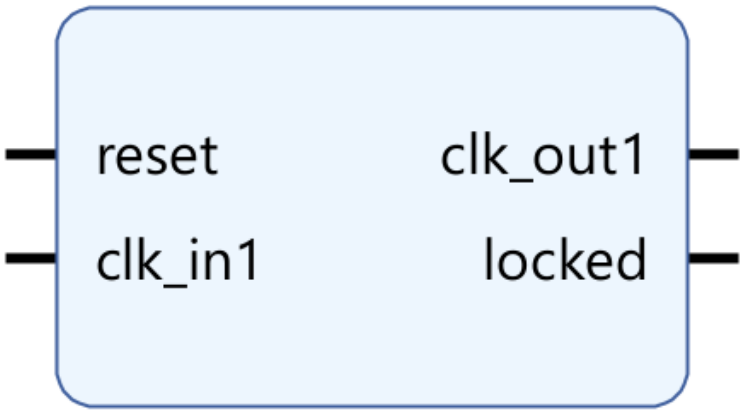
## Figur Komponen Block Design

## Ringkasan arsitektur

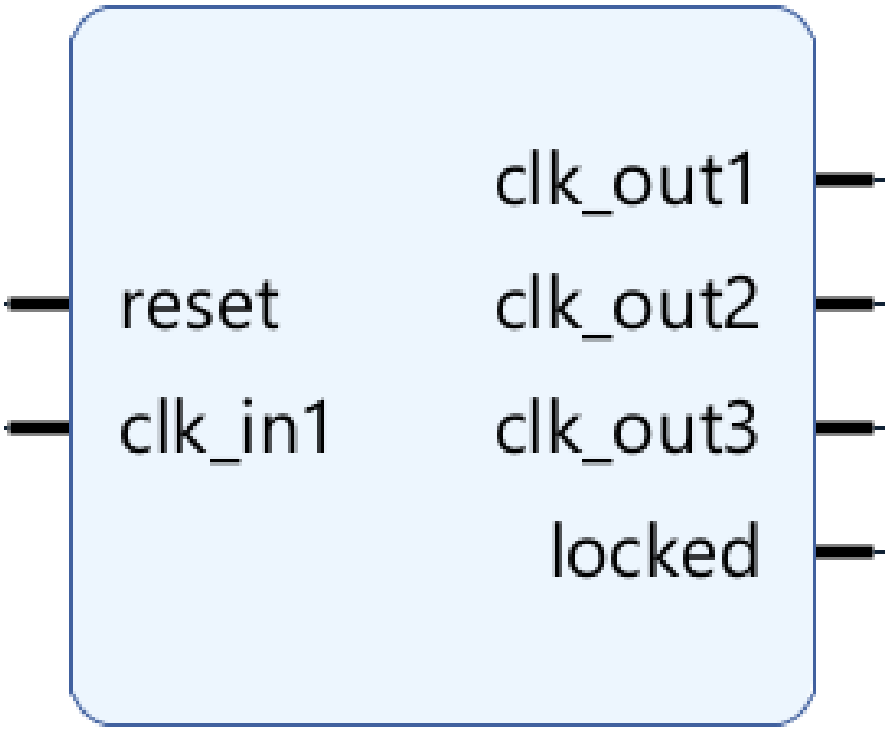
- Clock sistem berasal dari `sys_clock` 100 MHz dan masuk ke `clk_wiz_0`, yang menghasilkan domain 100 MHz untuk MicroBlaze, AXI interconnect, dan IP I2S.
- IP `i2s_0` membangkitkan `BCLK`, `WS`, dan `MCLK` secara internal menggunakan akumulator fase NCO secara murni digital dari clock sistem 100 MHz.




Gambar L.23. Tampilan interface view yang memperlihatkan keterhubungan MicroBlaze, UARTLite, dan IP I2S melalui AXI SmartConnect.



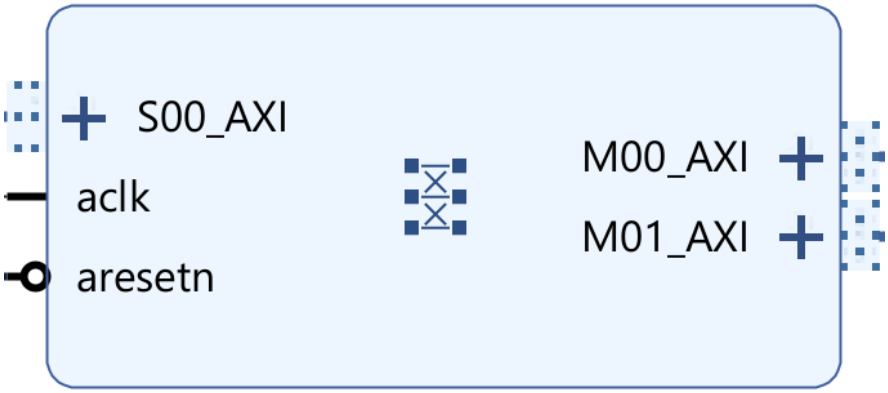

Gambar L.24. Tampilan awal konfigurasi Clocking Wizard sebelum penyesuaian frekuensi keluaran.



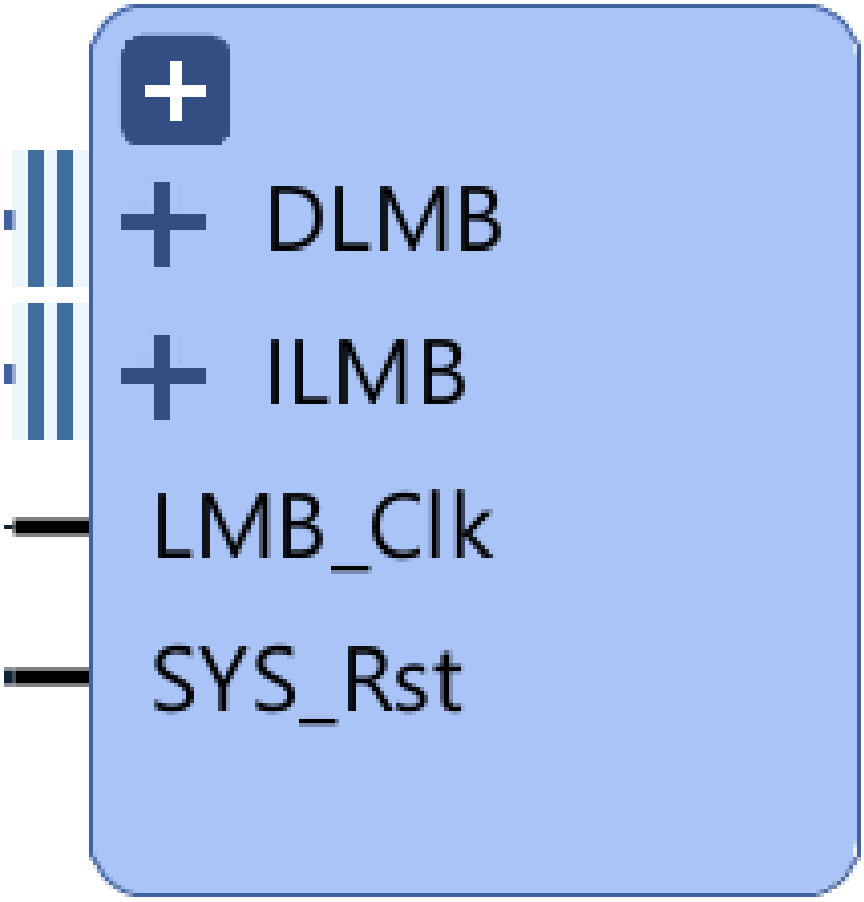

Gambar L.25. Clocking Wizard setelah dikonfigurasi untuk menghasilkan clock sistem 100 MHz.

Gambar L.26. Blok prosesor MicroBlaze V yang menjadi pengendali utama sistem.

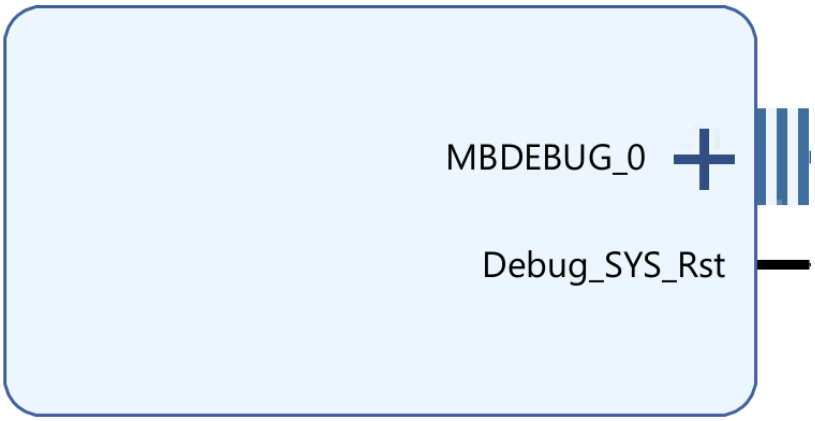
Gambar L.27. Block design reset controller yang membangkitkan sinyal reset sinkron untuk domain sistem.



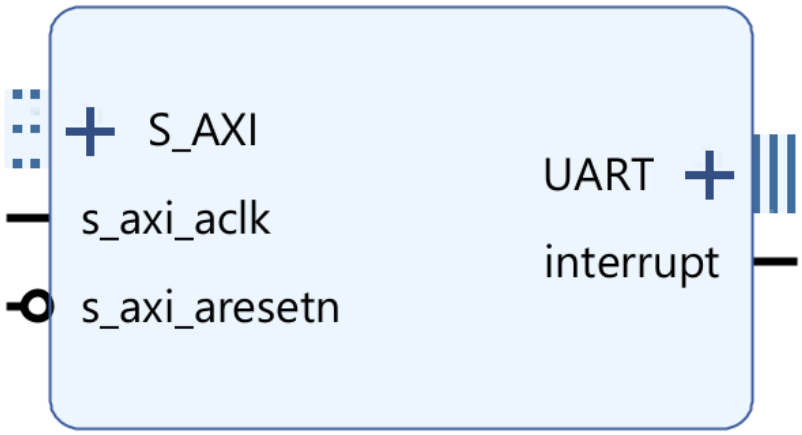

Gambar L.28. AXI SmartConnect yang merutekan transaksi dari MicroBlaze ke peripheral I2S dan UARTLite.



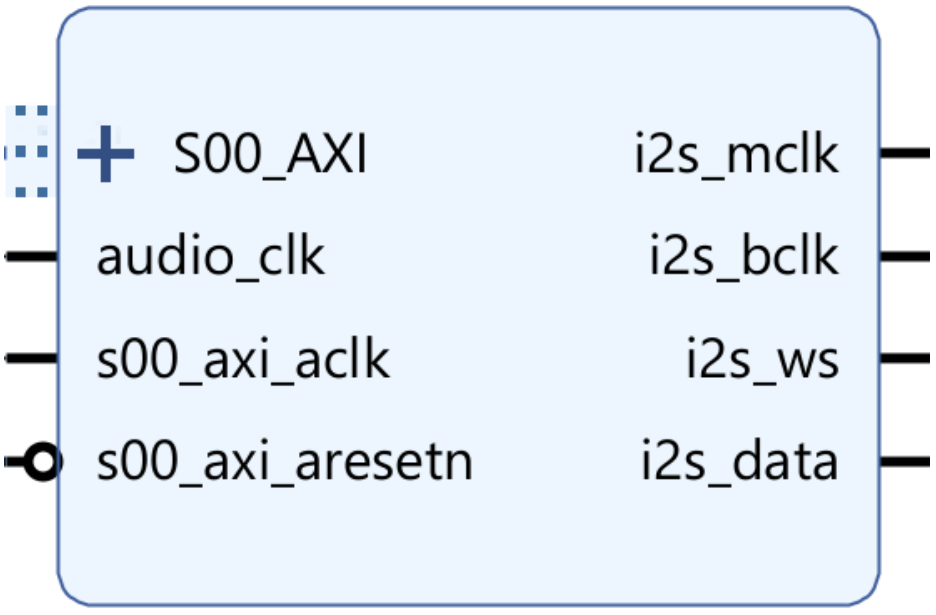

Gambar L.29. Subsistem BRAM lokal untuk instruksi dan data MicroBlaze V.




Gambar L.30. Microprocessor Debug Module yang digunakan untuk keperluan debug prosesor.




Gambar L.31. Peripheral UARTLite sebagai antarmuka terminal firmware.




Gambar L.32. Blok internal IP I2S yang digunakan sebagai peripheral utama.

- microblaze_riscv_0 bertindak sebagai master AXI dan terhubung ke axi_smc.
- axi_smc merutekan transaksi ke slave peripheral, termasuk i2s_0 dan axi_uartlite_0.
- Keluaran audio eksternal yang diekspos block design adalah i2s_mclk, i2s_bclk, i2s_ws, dan i2s_data.

**Peta alamat**

Tabel L.3. Segmen alamat penting pada block design bdesign_clock_core.

| Peripheral | Segmen | Offset basis |
|---|---|---|
| i2s_0 | S00_AXI_reg | 0x00020000 |
| axi_uartlite_0 | SEG_axi_uartlite_0_Reg | 0x04060000 |
| localBRAM | dlmb/ilmbmemory | 0x00000000 |

**clk_wiz_0**

**Purpose** Clocking Wizard untuk membangkitkan domain clock sistem 100 MHz.

**Key parameters** CLK_IN1=100MHz, CLKOUT1=100MHz. Keluaran audio tambahan (clk_out2/clk_out3) dapat dihubungkan ke port legacy IP tetapi tidak diperlukan oleh logika DDS.

**Connections** clk_out1 ke domain 100 MHz MicroBlaze, SmartConnect, I2S, dan UARTLite.

**microblaze_riscv_0**

**Purpose** Soft processor RISC-V untuk menjalankan firmware ready_to_run/sw/helloworld.c.

**Key parameters** Debug aktif, jalur M_AXI_DP untuk akses peripheral, serta ILMB/DLMB untuk memori lokal.

**Connections** Clock 100 MHz dari clk_wiz_0; reset dari rst_clk_wiz_0_100M; master AXI ke axi_smc/S00_AXI.

**microblaze_riscv_0_local_memory**

**Purpose** Subsistem memori lokal instruksi dan data.

**Sub-IP** dlmb_v10, ilmb_v10, dlmb_bram_if_cntlr, ilmb_bram_if_cntlr, dan lmb_bram.

**Key parameters** BRAM bertipe True_Dual_Port_RAM dan dihubungkan ke kedua jalur LMB.

**mdm_1**

**Purpose** *Microprocessor Debug Module* untuk debug prosesor.

**Connections** Antarmuka debug langsung ke `microblaze_riscv_0/DEBUG`.

**rst_clk_wiz_0_100M**

**Purpose** Reset controller sinkron untuk domain 100 MHz.

**Connections** Menghasilkan `mb_reset`, `bus_struct_reset`, dan `peripheral_aresetn` untuk MicroBlaze, SmartConnect, I2S, dan UARTLite.

**axi_smc**

**Purpose** AXI SmartConnect yang menghubungkan satu master ke beberapa slave.

**Connections** `S00_AXI` dari `microblaze_riscv_0`; `M00_AXI` ke `i2s_0`; `M01_AXI` ke `axi_uartlite_0`.

**i2s_0**

**Purpose** Custom IP I2S transmitter berbasis NCO (`xilinx.com:user:i2s:1.0`).

**Main interfaces** `S00_AXI` untuk kontrol register dan empat keluaran I2S.

**Key parameters** Empat register AXI4-Lite 32-bit (`DATA_LEFT`, `DATA_RIGHT`, `CONTROL`, `STATUS`), `C_S00_AXI_BASEADDR=0x00020000`, seluruh logika audio pada domain 100 MHz.

**axi_uartlite_0**

**Purpose** UARTLite untuk antarmuka terminal firmware.

**Main interfaces** `S_AXI` untuk konfigurasi AXI dan `UART` untuk `usb_uart_txd`/`usb_uart_rxd`.

**Key parameters** Alamat basis `0x04060000` pada ruang alamat MicroBlaze.

# BAB L
# RINGKASAN KODE SUMBER

Lampiran ini sekarang hanya berfungsi sebagai penunjuk ke repositori. Seluruh RTL dan firmware utama sudah disisipkan penuh pada Bab IV, sehingga lampiran tidak perlu lagi mengulang listing kode.

## L.1 Repositori

- RTL utama: `../latesti2s/ready_to_run/ip_repo/i2s/hdl/i2s.v` dan `../latesti2s/ready_to_run/ip_repo/i2s/hdl/i2s_slave_lite_v1_0_S00_AXI.v`.
- Firmware uji: `../latesti2s/ready_to_run/sw/helloworld.c`.
- Skrip rekonstruksi Vivado: `../latesti2s/ready_to_run/rebuild.tcl` dan `../latesti2s/ready_to_run/fsi2snfilter.tcl`.

# ARBITRARY-FS I2S DAN IP KOMUNIKASI BUATAN SENDIRI

Lampiran ini menjelaskan varian desain I2S (`i2s_1_0`) yang menjadi implementasi akhir penelitian. IP ini dibuat sebagai peripheral komunikasi buatan sendiri agar sistem tidak bergantung pada IP berlisensi dari pihak ketiga.

- Modul inti `i2s_slave_lite_v1_0_S00_AXI.v` membangkitkan `BCLK`, `WS`, dan `MCLK` menggunakan akumulator fase NCO pada domain `S_AXI_ACLK` 100 MHz.
- Frekuensi sampling diprogram melalui bitfield `SAMPLE_RATE_HZ` pada register `CONTROL` (rentang 0–1.048.575 Hz, dibatasi aman hingga 195.312 Hz).
- Register `STATUS[0]` berubah setiap awal frame stereo sehingga firmware dapat melakukan *polling* tanpa interrupt.
- Port clock legacy `audio_48_clk`/`audio_44_clk` telah dihapus sepenuhnya dari modul level teratas (`i2s\_dds.v`) untuk menyederhanakan arsitektur.
- Kode lengkap berada di `ready_to_run/ip_repo/i2s_1_0/hdl`; firmware uji di `ready_to_run/sw/helloworld.c`.

Potongan kode NCO yang menunjukkan pembangkitan detak BCLK ditampilkan pada Listing L.1.

```
wire [31:0] bclk_toggle_rate_hz = {sample_rate_sync, 7'b0}; // Fs x 128
wire bclk_phase_wrap = (bclk_phase_sum >= AXI_CLK_HZ);

always @(posedge S_AXI_ACLK) begin
    if (bclk_phase_wrap) begin
        bclk_phase_acc_q <= bclk_phase_sum - AXI_CLK_HZ;
        bclk_q <= !bclk_q;
    end else begin
        bclk_phase_acc_q <= bclk_phase_sum;
    end
end
```

Listing L.1. Akumulator fase NCO untuk pembangkitan BCLK pada `i2s_slave_lite_v1_0_S00_AXI.v`.

Perbedaan utama terhadap desain awal berbasis pembagi clock tetap:

- frekuensi sampling tidak lagi dibatasi pada empat mode (`FS_FAMILY`/`FS_MODE`), melainkan dapat diprogram dalam Hz;
- serializer dan register file berada pada domain clock yang sama (100 MHz), sehingga tidak memerlukan CDC antar-domain untuk jalur data;

- *handshake* firmware menggunakan register `STATUS`, bukan deadline berbasis `mcycle` atau `usleep()`.

Rincian kode dan perubahan perangkat lunak pendukung dijelaskan pada Bab IV dan Lampiran L.